JENKINS-TASK-04

1.Setup Jenkins CICD pipeline using freestyle job using Docker containers using below code.

Source Code: https://github.com/betawins/hiring-app.git

1) Stages:

- **Git Clone**
- **SonarQube Integration**
- **Maven Compilation**
- **Nexus Artifactory**
- **Slack Notification**
- **Deploy On tomcat**

The final flow will be:

GitHub → Jenkins Freestyle → Maven Compile/Test → SonarQube → Nexus → Tomcat → Slack

1. What you need before starting

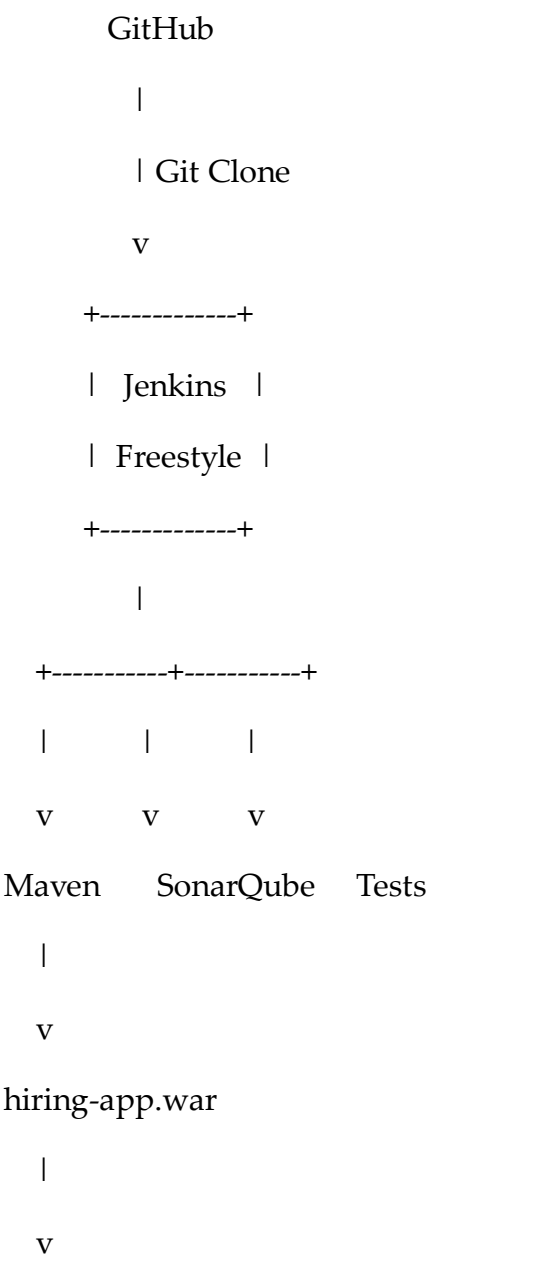
You already have much of the Jenkins/SonarQube setup from your previous work, so don't reinstall everything unnecessarily.

Required components

Component	Where to run	Purpose
Jenkins	EC2	CI/CD automation
Git	Jenkins EC2	Clone source code
Java 17	Jenkins EC2	Maven/application
Maven	Jenkins EC2	Compile/package

Component	Where to run	Purpose
SonarQube	Your existing EC2	Code quality
Nexus Repository	Docker container	Store WAR artifact
Tomcat	Docker container	Deploy application
Slack	Cloud	Notifications
Docker	Jenkins EC2	Run Nexus/Tomcat containers

The architecture should look approximately like:



Nexus Repository

(Docker)

|

v

Tomcat

(Docker)

|

v

Running Application

Jenkins

|

v

Slack

Notification

Important point about "Docker containers"

For this assignment, don't try to put **every single Jenkins build step inside a separate Docker container** unless your instructor specifically requires that.

A much simpler and cleaner implementation is:

- Jenkins → EC2 installation
- Nexus → Docker container
- Tomcat → Docker container
- Jenkins controls Docker using docker commands
- Maven runs on Jenkins
- SonarQube runs on its existing server
- Slack receives build notifications

Sonatype officially supports running Nexus Repository in a containerized environment and provides the sonatype/nexus3 image.

Step 1 — Verify Jenkins server

SSH into your Jenkins EC2:

ssh -i your-key.pem ec2-user@<JENKINS_PUBLIC_IP>

Check Jenkins:

sudo systemctl status jenkins

Check Java:

java -version

Check Maven:

mvn -version

Check Git:

git --version

Check Docker:

docker --version

If Docker is not installed on Amazon Linux 2023:

sudo dnf install docker -y

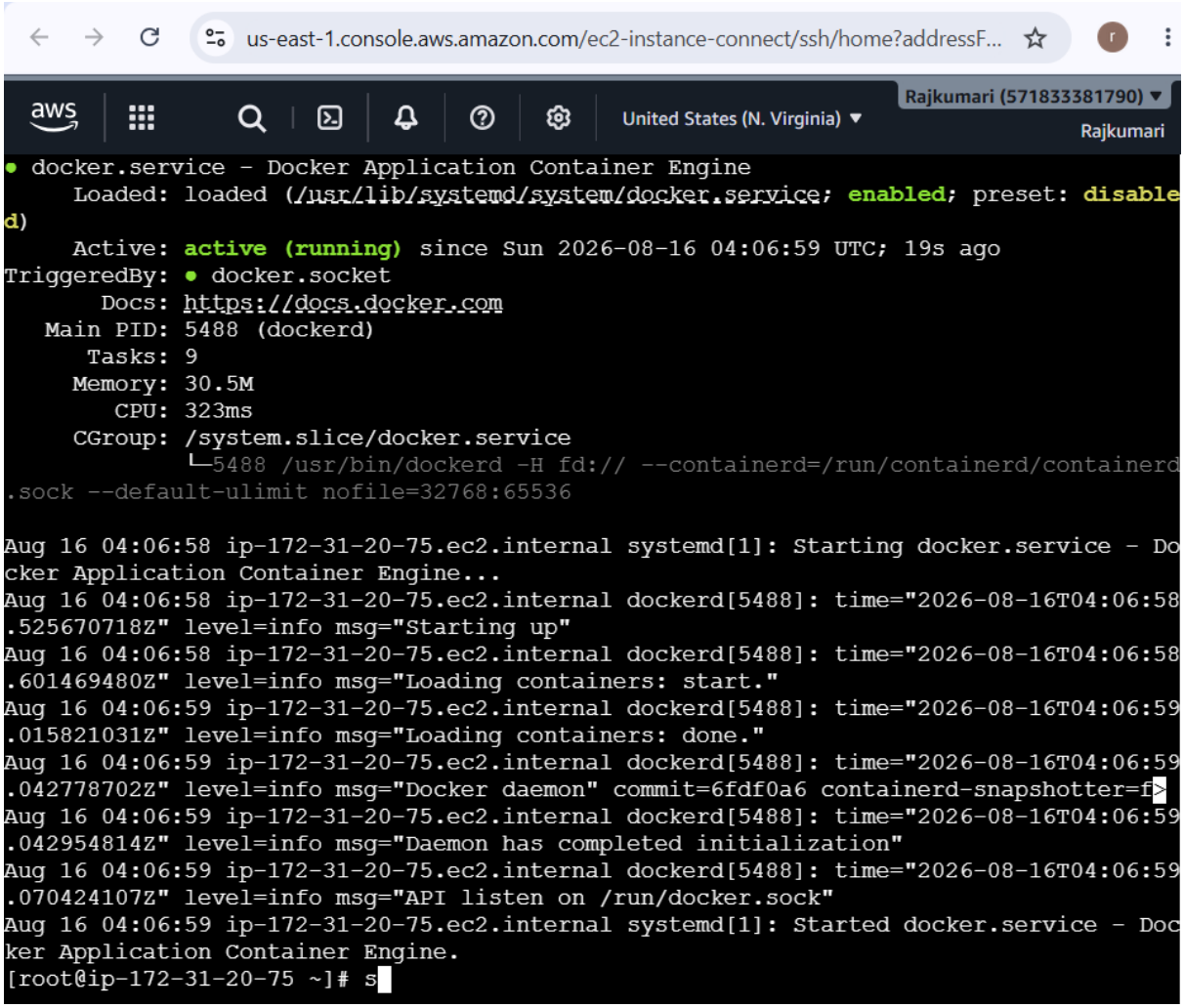
Start Docker:

sudo systemctl start docker

sudo systemctl enable docker

Check:

sudo systemctl status docker



Step 2 — Give Jenkins permission to use Docker

This is important.

Find Jenkins user:

id jenkins

Add Jenkins to Docker group:

sudo usermod -aG docker jenkins

Restart Jenkins:

sudo systemctl restart jenkins

You may also restart Docker:

sudo systemctl restart docker

Then verify:

sudo -u jenkins docker ps

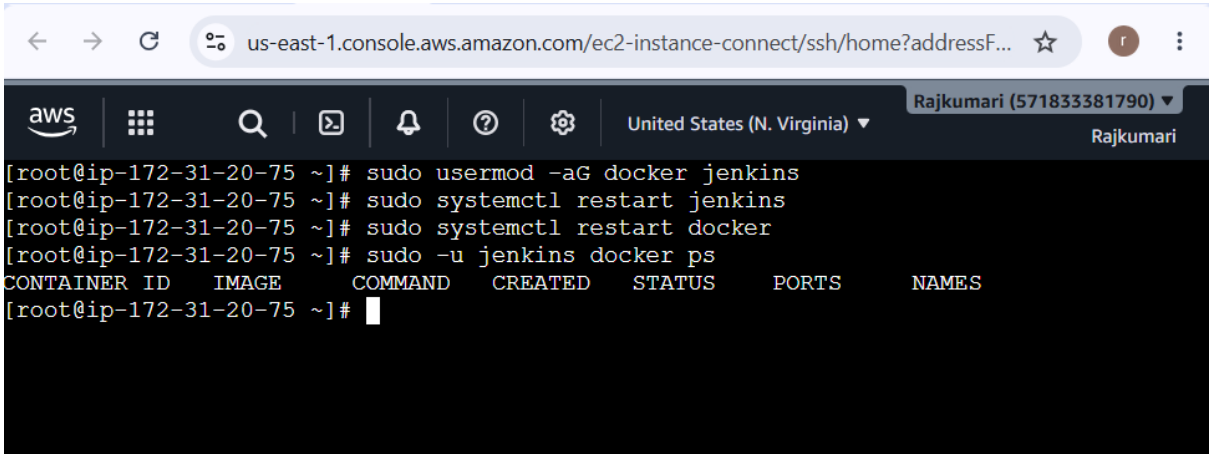
You want:

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

and **not**:

permission denied while trying to connect to the Docker daemon

If permission is still denied, log out of SSH and reconnect, then restart Jenkins.



Step 3 — Create Nexus using Docker

Pull Nexus:

docker pull sonatype/nexus3

Create a persistent volume:

docker volume create nexus-data

Run Nexus:

```
docker run -d \
  --name nexus \
  -p 8081:8081 \
  -v nexus-data:/nexus-data \
  sonatype/nexus3
```

Check:

docker ps

You should see something similar to:

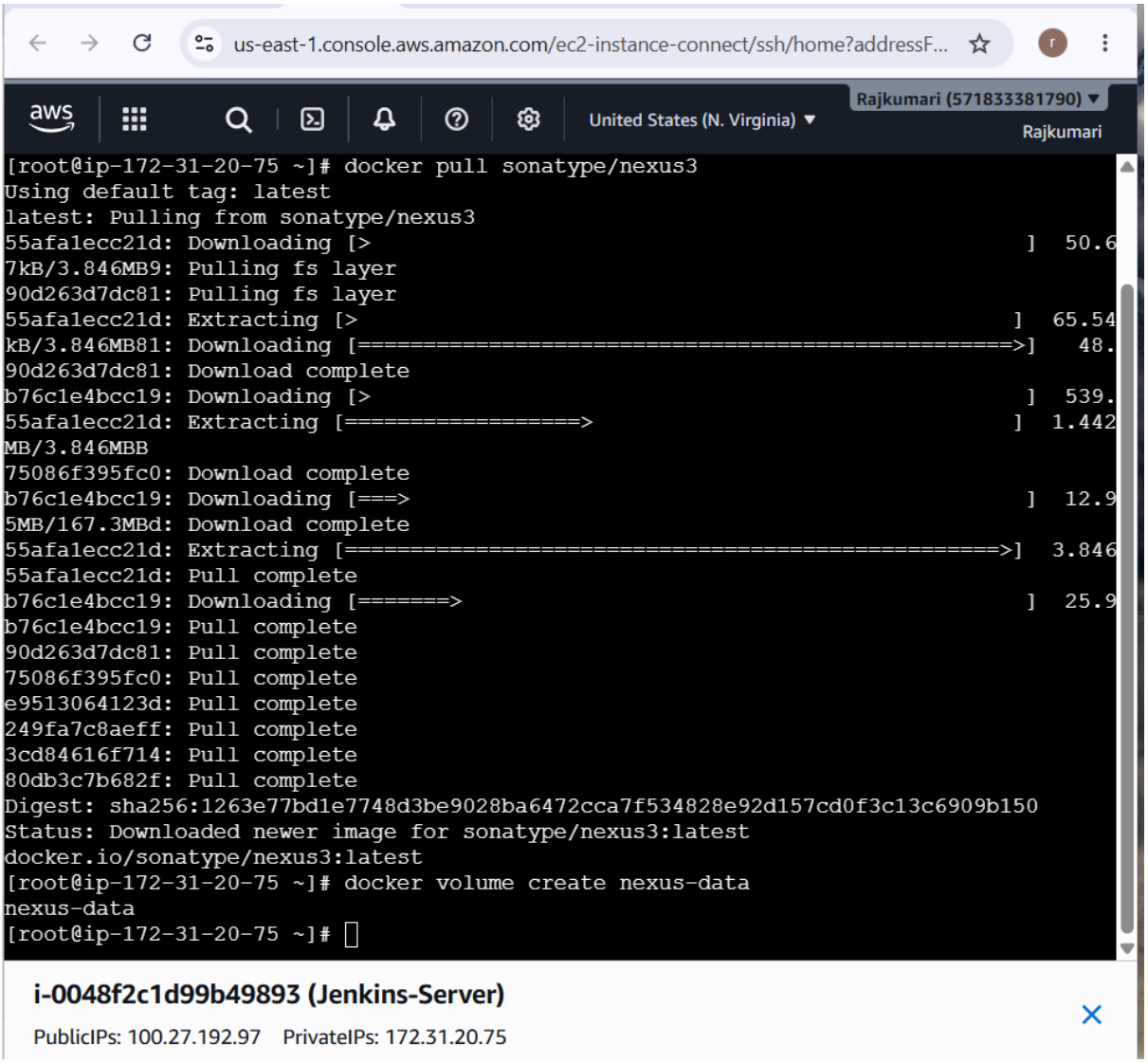
nexus sonatype/nexus3 Up ...

Check logs:

docker logs -f nexus

Wait until Nexus finishes starting.

Sonatype documents Docker/container deployment for Nexus Repository, and the current Nexus 3 Docker image is sonatype/nexus3.



Download Nexus

cd /opt

wget https://download.sonatype.com/nexus/3/nexus-3.93.1-04-linux-x86_64.tar.gz

Verify:

ls -lh nexus-3.93.1-04-linux-x86_64.tar.gz

2. Extract Nexus

sudo tar -xvzf nexus-3.93.1-04-linux-x86_64.tar.gz

Check:

ls -ld /opt/nexus-3.93.1-04

3. Create a dedicated Nexus user

Do **not** run Nexus as root.

sudo useradd -r -m -d /opt/nexus -s /bin/bash nexus

4. Change ownership

sudo chown -R nexus:nexus /opt/nexus-3.93.1-04

Create the Nexus data directory:

sudo mkdir -p /opt/sonatype-work

sudo chown -R nexus:nexus /opt/sonatype-work

5. Configure Nexus to run as the nexus user

Open:

sudo vi /opt/nexus-3.93.1-04/bin/nexus.rc

Find:

#run_as_user=""

Change it to:

run_as_user="nexus"

Save and exit.

6. Create a symbolic link

This makes future Nexus upgrades easier:

sudo ln -s /opt/nexus-3.93.1-04 /opt/nexus

Check:

ls -l /opt/nexus

7. Check Java

Run:

java -version

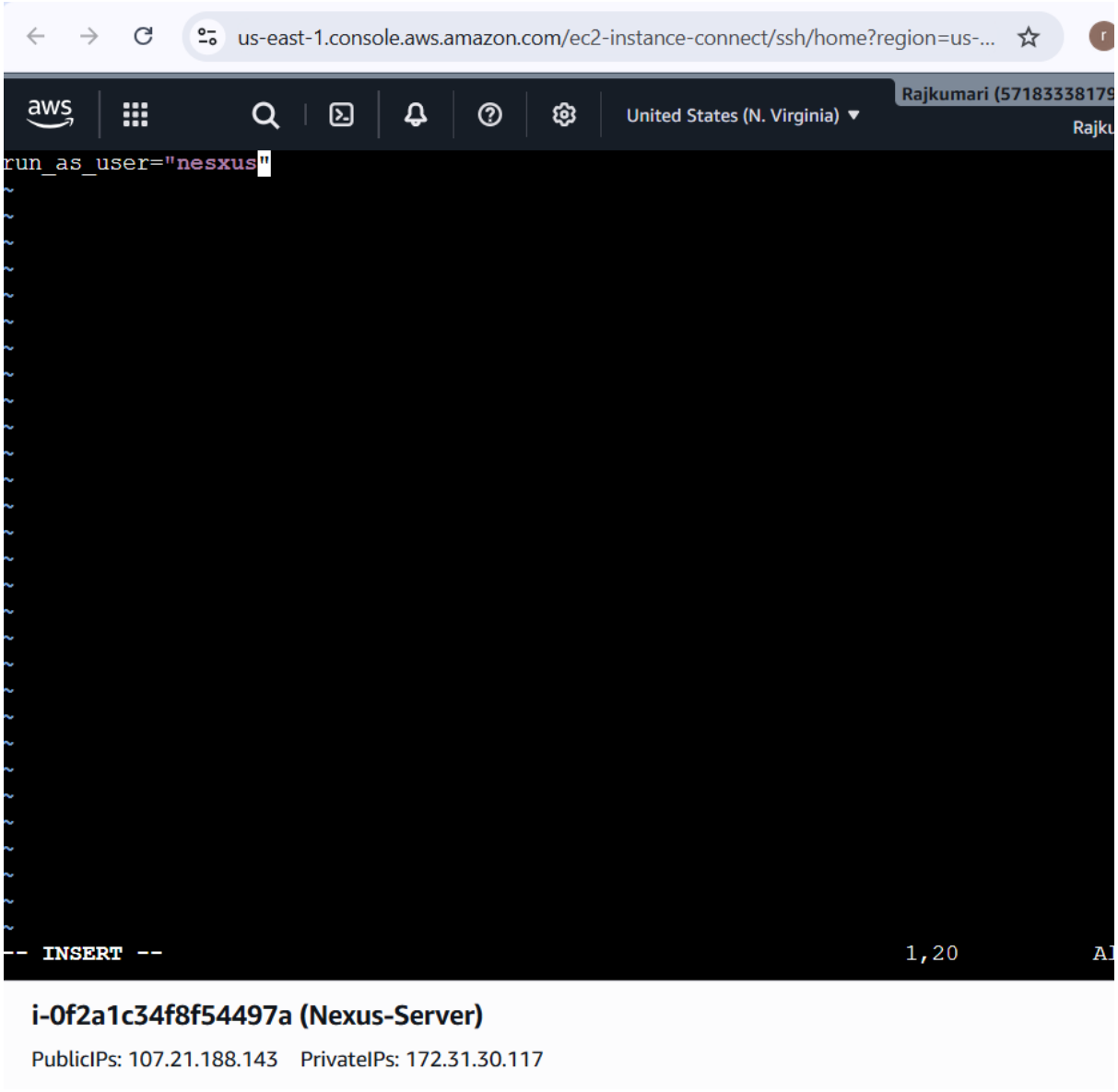
For your Nexus installation, **Java compatibility is important**. If Java is missing or you have the wrong version, don't start Nexus yet.

Paste the output of:

java -version

and:

ls -lh /opt/nexus-3.93.1-04



Download Nexus

cd /opt

wget https://download.sonatype.com/nexus/3/nexus-3.93.1-04-linux-x86_64.tar.gz

Verify:

ls -lh nexus-3.93.1-04-linux-x86_64.tar.gz

2. Extract Nexus

sudo tar -xvzf nexus-3.93.1-04-linux-x86_64.tar.gz

Check:

ls -ld /opt/nexus-3.93.1-04

3. Create a dedicated Nexus user

Do **not** run Nexus as root.

sudo useradd -r -m -d /opt/nexus -s /bin/bash nexus

4. Change ownership

sudo chown -R nexus:nexus /opt/nexus-3.93.1-04

Create the Nexus data directory:

sudo mkdir -p /opt/sonatype-work

sudo chown -R nexus:nexus /opt/sonatype-work

5. Configure Nexus to run as the nexus user

Open:

sudo vi /opt/nexus-3.93.1-04/bin/nexus.rc

Find:

#run_as_user=""

Change it to:

run_as_user="nexus"

Save and exit.

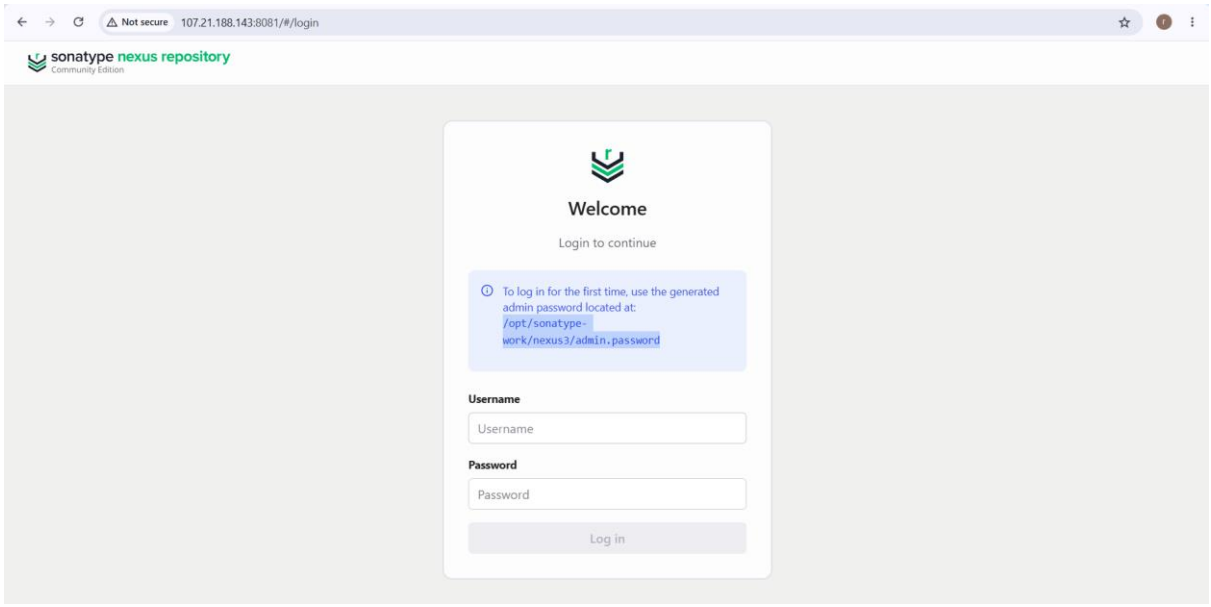
6. Create a symbolic link

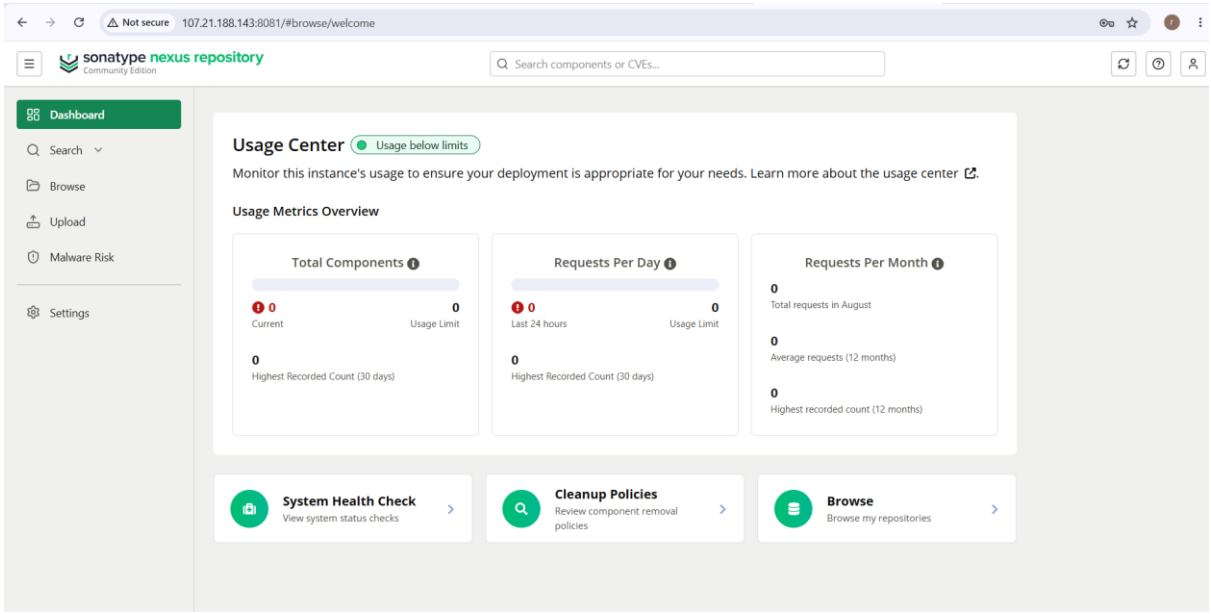
This makes future Nexus upgrades easier:

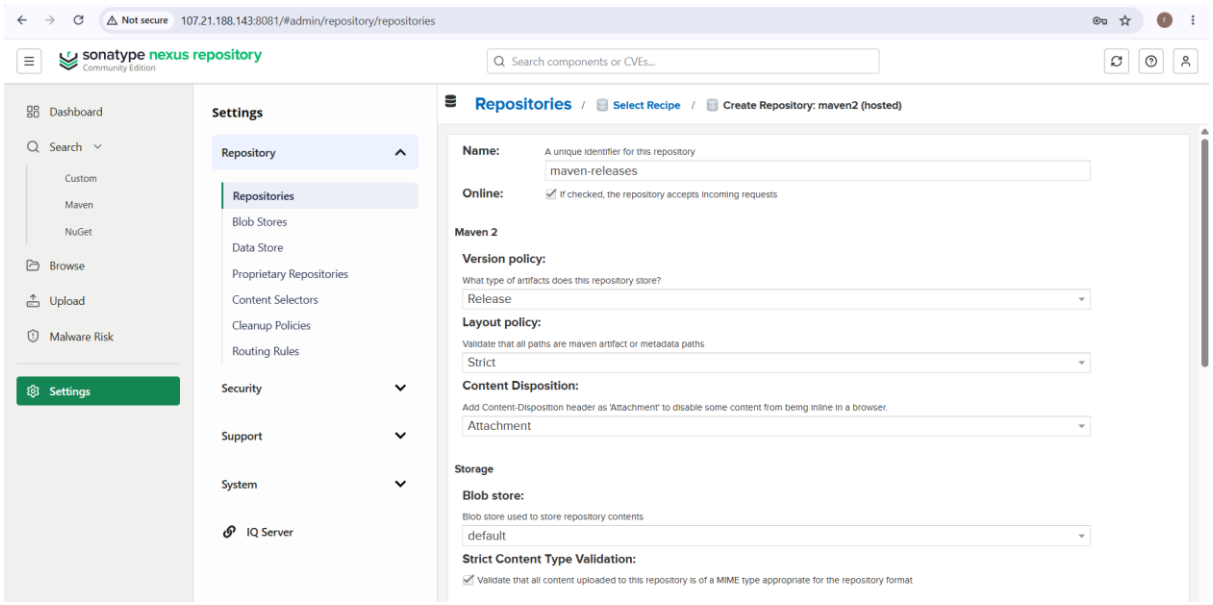
sudo ln -s /opt/nexus-3.93.1-04 /opt/nexus

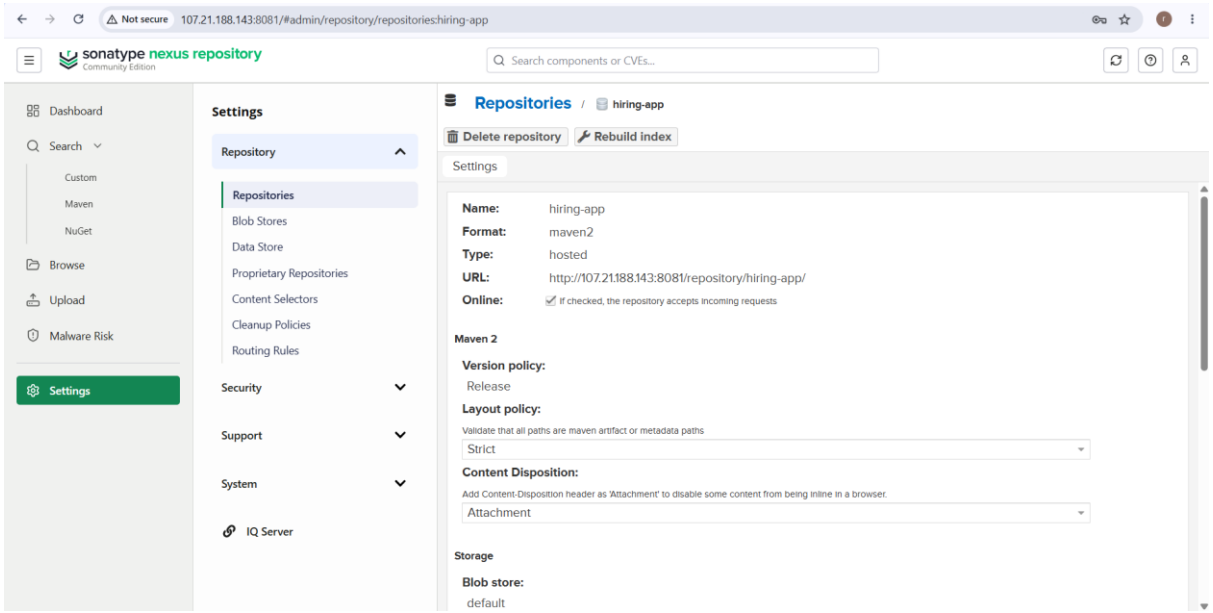
Check:

ls -l /opt/nexus


Not secure
107.21.188.143:8081/#/login
sonatype nexus repository
Community Edition
Welcome
Login to continue
To log in for the first time, use the generated admin password located at:
/opt/sonatype-work/nexus3/admin.password
Username
Username
Password
Password
Log in


Not secure
107.21.188.143:8081/#browse/welcome
sonatype nexus repository
Community Edition
Search components or CVEs...
Dashboard
Search
Browse
Upload
Malware Risk
Settings
Usage Center
Usage below limits
Monitor this instance's usage to ensure your deployment is appropriate for your needs. Learn more about the usage center
Usage Metrics Overview
Total Components
0
Current
0
Usage Limit
0
Highest Recorded Count (30 days)
Requests Per Day
0
Last 24 hours
0
Usage Limit
0
Highest Recorded Count (30 days)
Requests Per Month
0
Total requests in August
0
Average requests (12 months)
0
Highest recorded count (12 months)
System Health Check
View system status checks
Cleanup Policies
Review component removal policies
Browse
Browse my repositories





1. Click Create local user

You should get a form with fields similar to:

- User ID
- First name
- Last name
- Email
- Password
- Status

- Roles

Enter:

Field	Value
User ID	jenkins
First name	Jenkins
Last name	CI/CD
Email	Your email
Password	Create a strong password
Status	Active

2. Assign the appropriate role

For your assignment, Jenkins needs permission to upload artifacts to Nexus.

For a simple lab setup, you can select:

nx-admin

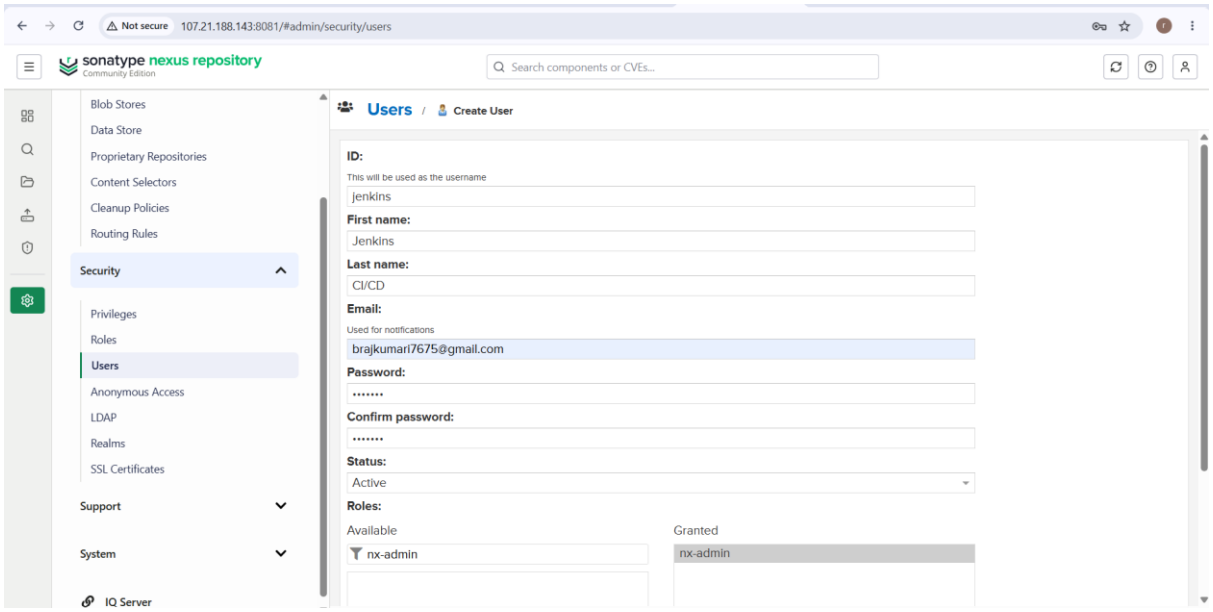
However, **for a real production environment, don't give Jenkins nx-admin**. A dedicated role with only repository read/write permissions is better.

Since this is your training/assignment environment, nx-admin will make the Jenkins → Nexus integration easier.

3. Save the user

Click:

Create local user



Then in Jenkins:

Manage Jenkins → Credentials → Global → Add Credentials

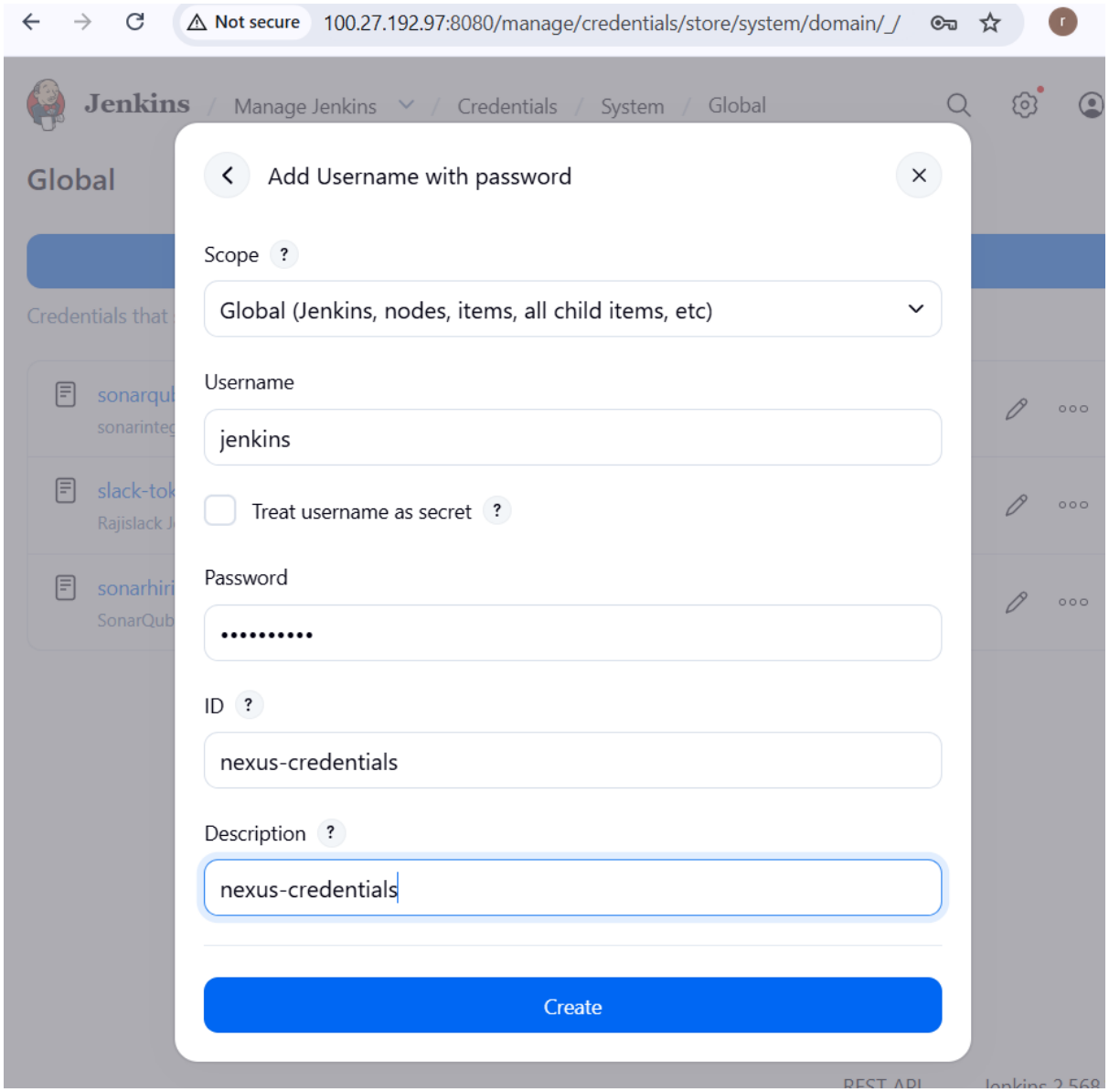
Select:

Kind: Username with password

ID: nexus-credentials

Username: jenkins

Password: <Nexus password>



10. Verify Tomcat server

On your Tomcat server, verify Tomcat is running:

sudo systemctl status tomcat

or:

sudo ss -lntp | grep 8080

Test:

curl http://localhost:8080

From your browser:

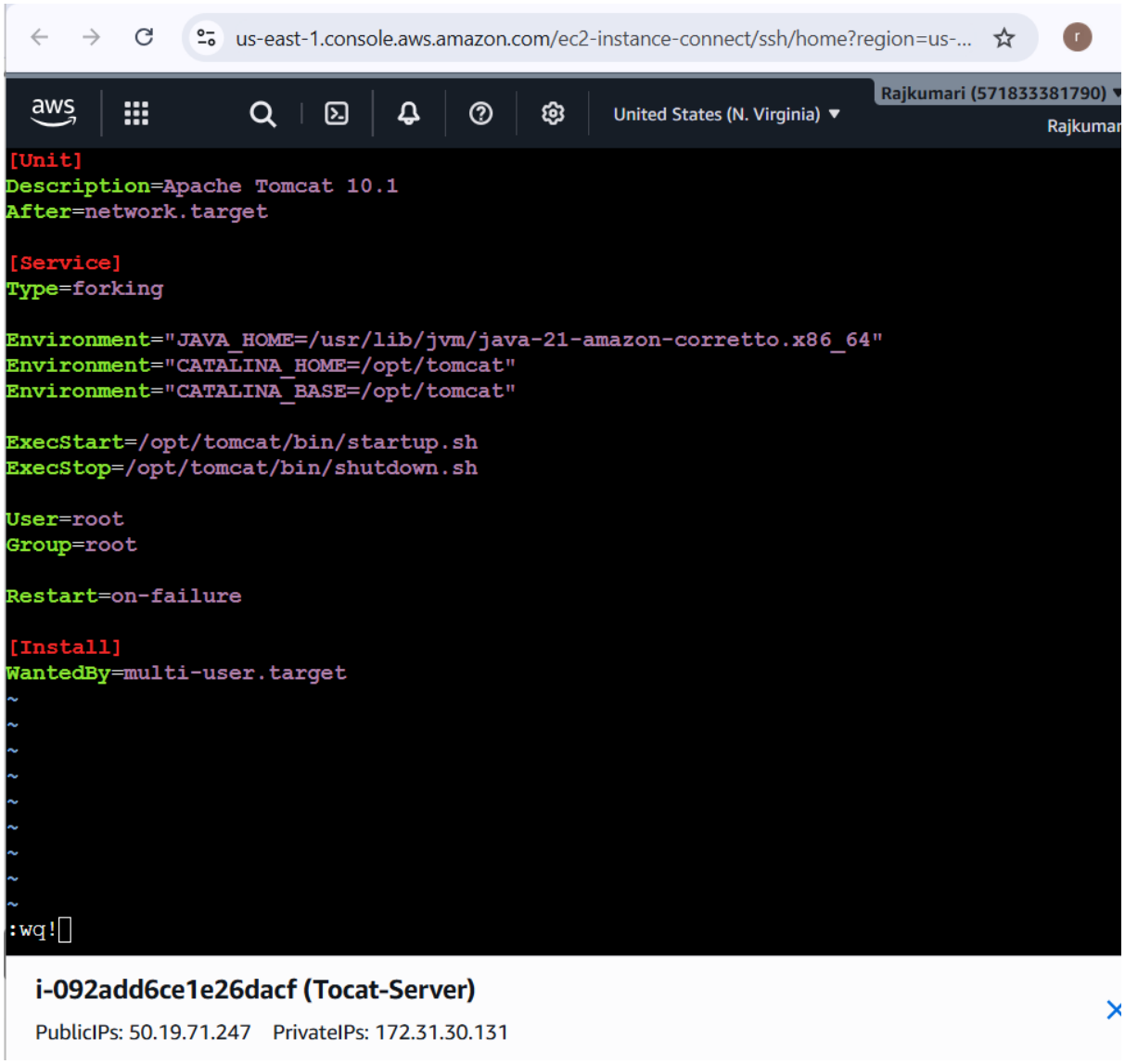
http://<TOMCAT_PUBLIC_IP>:8080

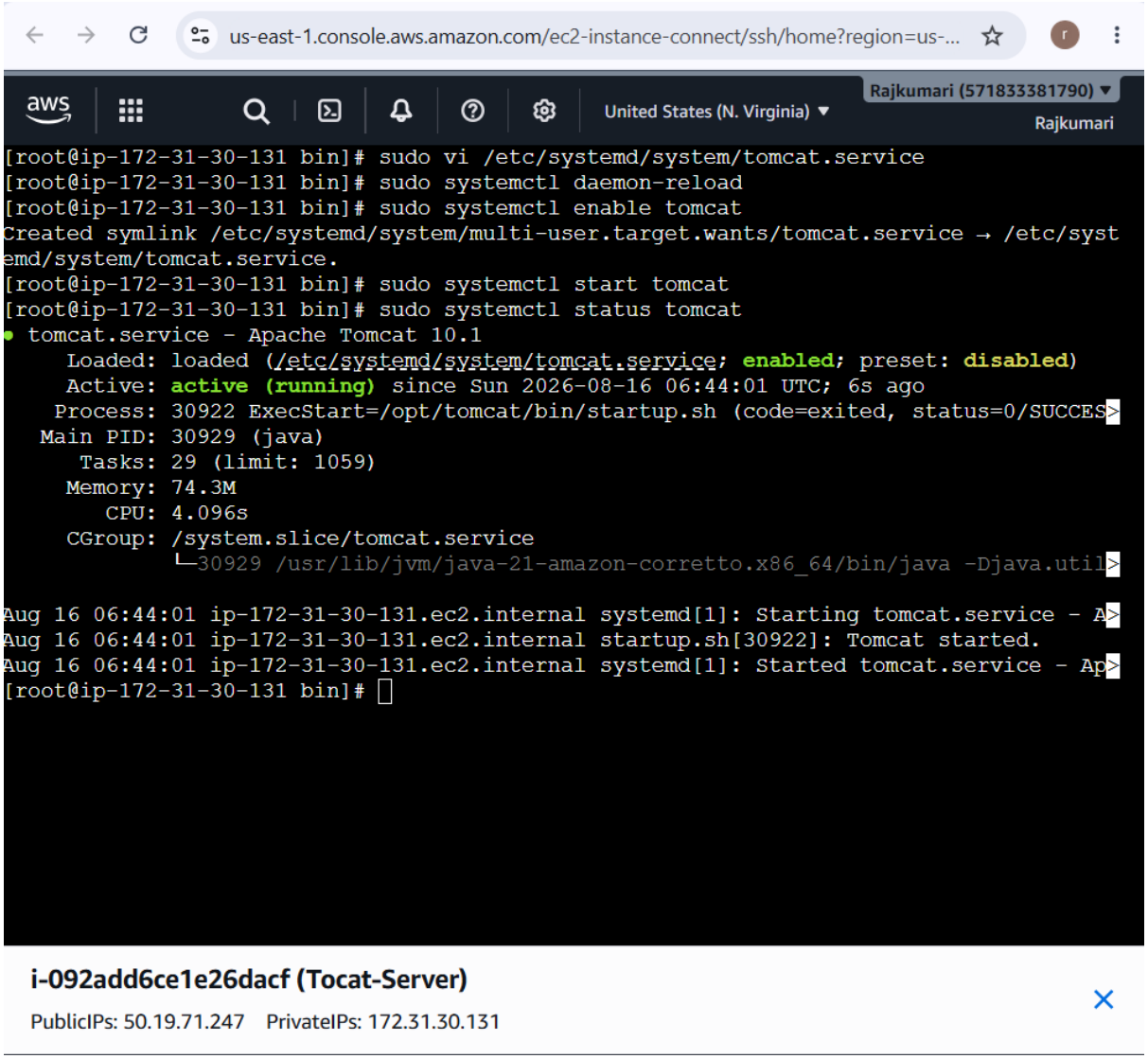
If your assignment specifically requires **Docker containers**, this is where your Tomcat container can be used.

For example:

docker ps

should show your Tomcat container.





Configure Tomcat for Jenkins Deployment

Your assignment requires:

Jenkins → Build WAR → Deploy to Tomcat

So next configure Tomcat Manager.

8. Edit tomcat-users.xml

sudo vi /opt/tomcat/conf/tomcat-users.xml

Before the closing </tomcat-users> tag, add:

<role rolename="manager-gui"/>

<role rolename="manager-script"/>

<user username="jenkins" password="Jenkins@123" roles="manager-gui,manager-script"/>

It should look like:

<tomcat-users>

<role rolename="manager-gui"/>

<role rolename="manager-script"/>

<user username="jenkins"

password="Jenkins@123"

roles="manager-gui,manager-script"/>

</tomcat-users>

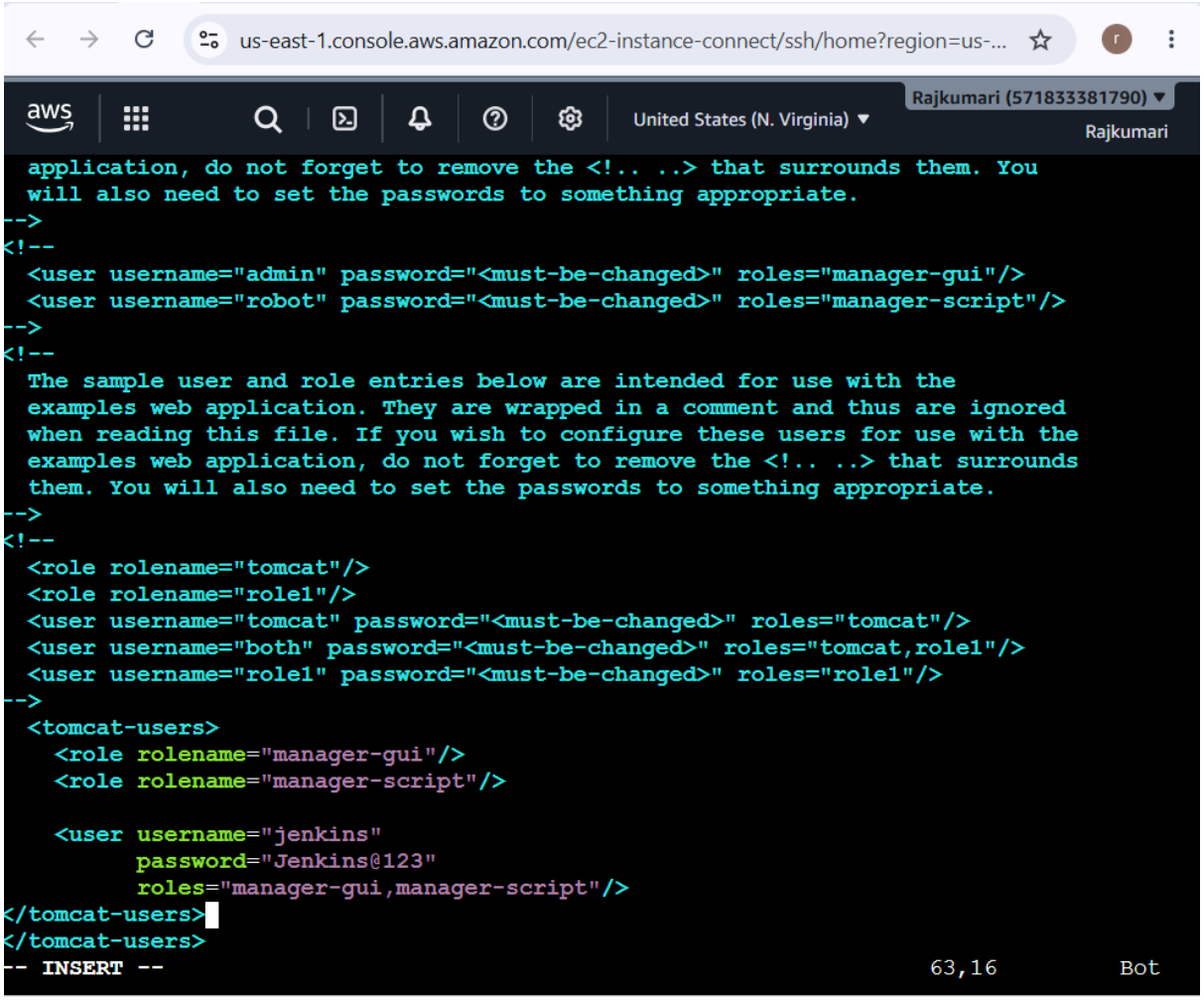
Save:

ESC

:wq

ENTER

For your assignment, you can use this password, but for a real environment use a strong secret and don't store it directly in your Jenkinsfile.



For your Tomcat server, use this Jenkins private IP to restrict Tomcat Manager access to Jenkins.

1. On the Tomcat server

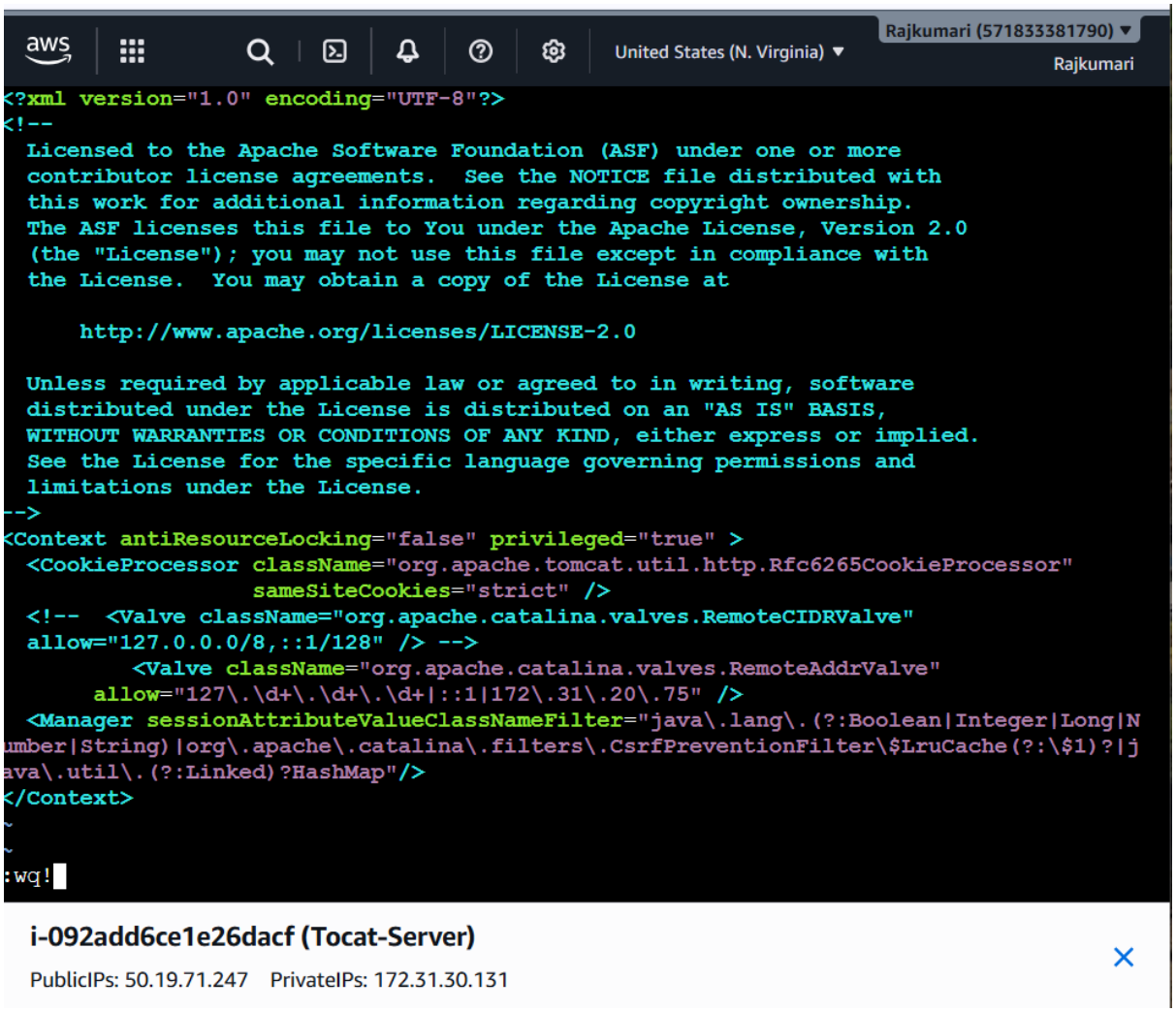
Edit:

sudo vi /opt/tomcat/webapps/manager/META-INF/context.xml

Find the RemoteAddrValve and change it to:

<Valve className="org.apache.catalina.valves.RemoteAddrValve"

allow="127\.\d+\.\d+\.\d+|::1|172\.31\.20\.75" />



6. Set JAVA_HOME

Check the installed directory:

ls -l /usr/lib/jvm/

Then create the environment variable:

sudo vi /etc/profile.d/java.sh

Add:

export JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto

export PATH=$JAVA_HOME/bin:$PATH

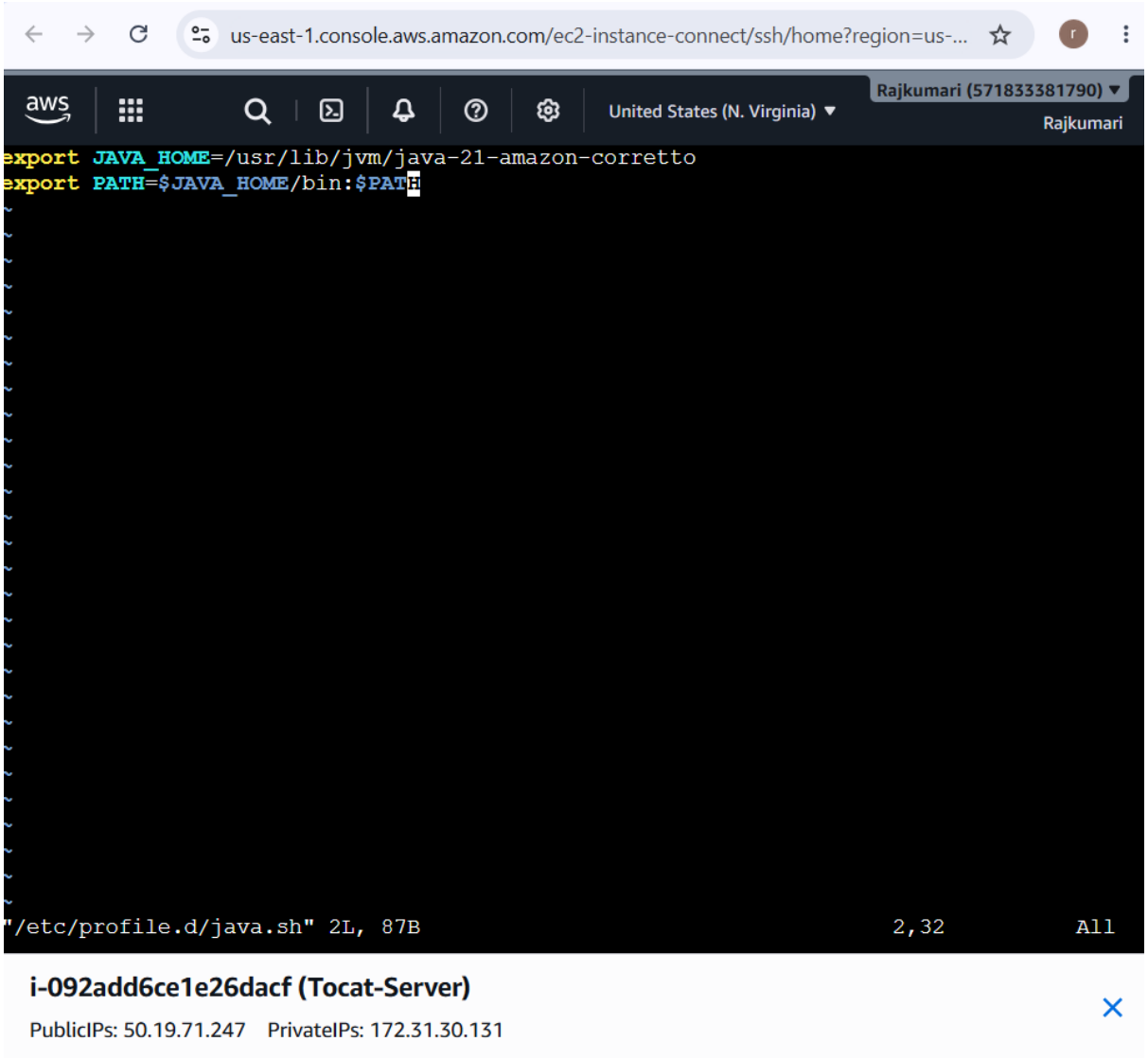
Save and exit, then run:

source /etc/profile.d/java.sh

Verify:

echo $JAVA_HOME

java -version



1. Install Nexus-related plugin

Go to:

Jenkins → Manage Jenkins → Plugins → Available plugins

Search for:

Nexus Artifact Uploader

Install it.

This allows Jenkins to upload the generated .war/.jar artifact to your Nexus repository.

After installation, restart Jenkins if Jenkins asks you to.

2. Install Tomcat-related plugin

Search for:

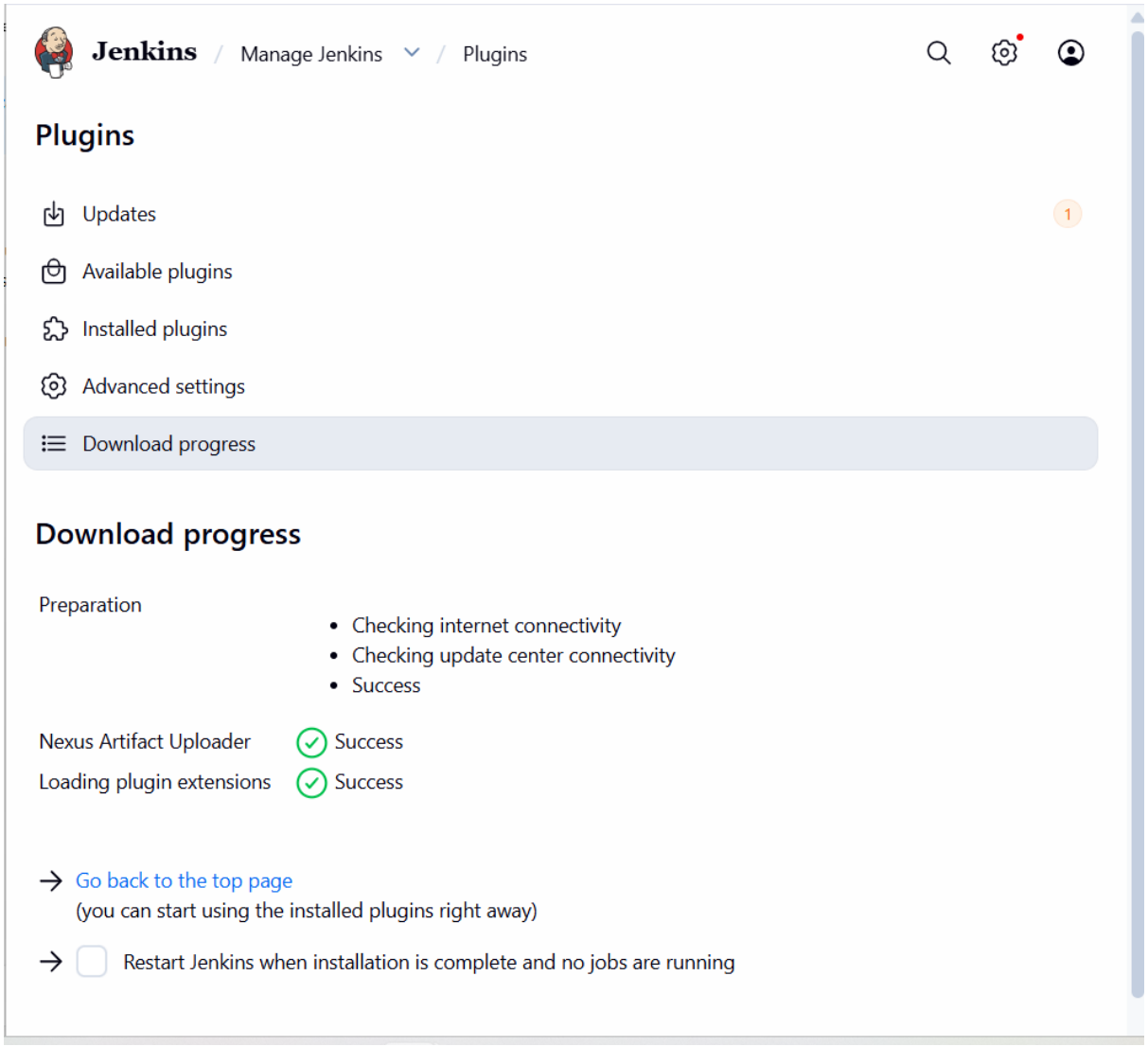
Deploy to container

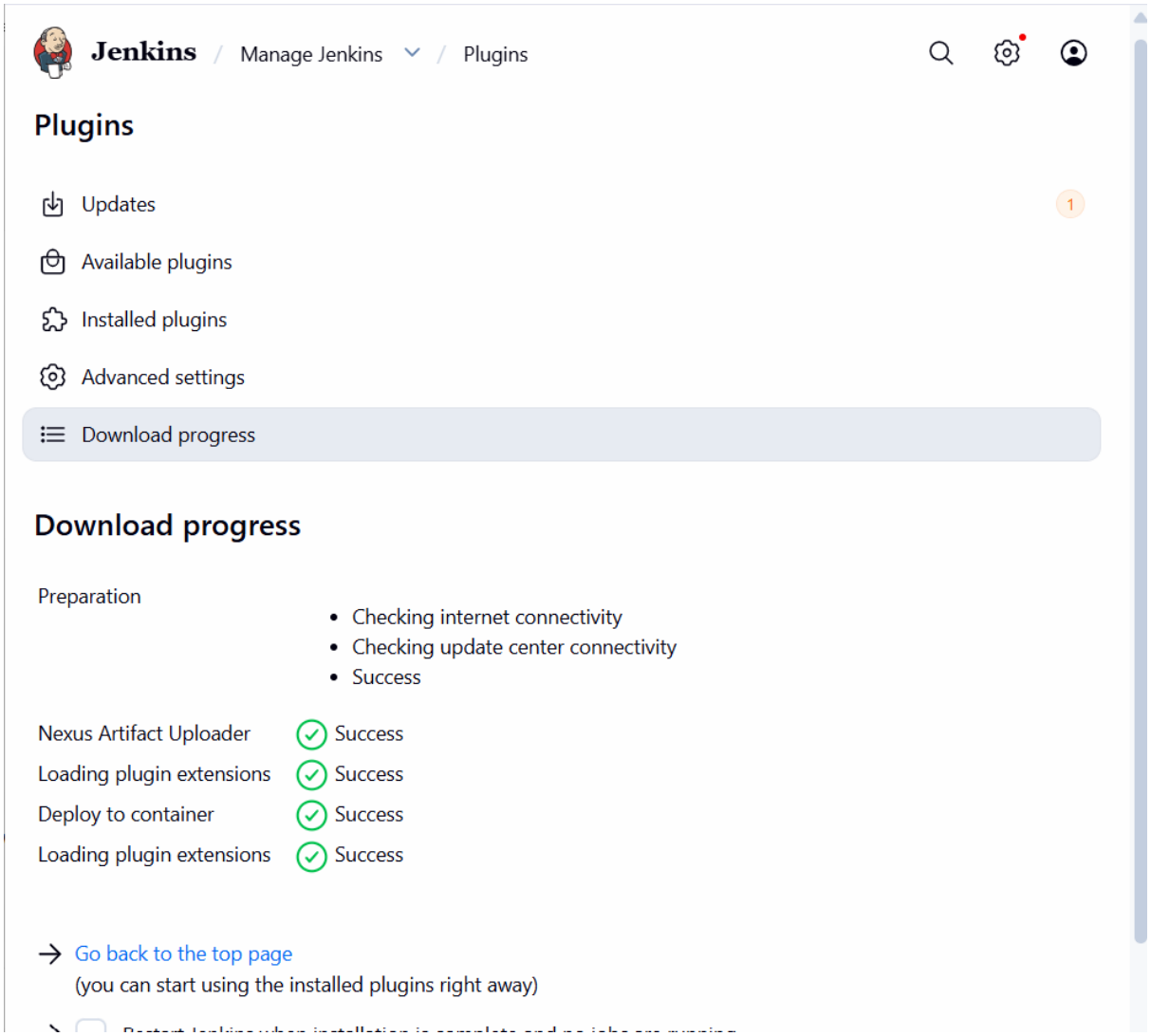
Install:

Deploy to container Plugin

This plugin supports deployment of WAR files to containers such as Tomcat.

After installation, restart Jenkins if required.


Jenkins / Manage Jenkins / Plugins
Plugins
Updates
1
Available plugins
Installed plugins
Advanced settings
Download progress
Download progress
Preparation
Checking internet connectivity
Checking update center connectivity
Success
Nexus Artifact Uploader
Success
Loading plugin extensions
Success
Go back to the top page
(you can start using the installed plugins right away)
Restart Jenkins when installation is complete and no jobs are running



Now create Maven settings.xml

This is the **next step**.

On your **Jenkins server**, run:

sudo mkdir -p /var/lib/jenkins/.m2

Then:

sudo vi /var/lib/jenkins/.m2/settings.xml

Put this inside:

<settings>

  <servers>

    <server>

```
        <id>hiring-app</id>
        <username>${env.NEXUS_USERNAME}</username>
        <password>${env.NEXUS_PASSWORD}</password>
      </server>
    </servers>
</settings>
```

Notice this important part:

<id>hiring-app</id>

It **must exactly match** the <id> in your pom.xml.

Your setup is therefore:

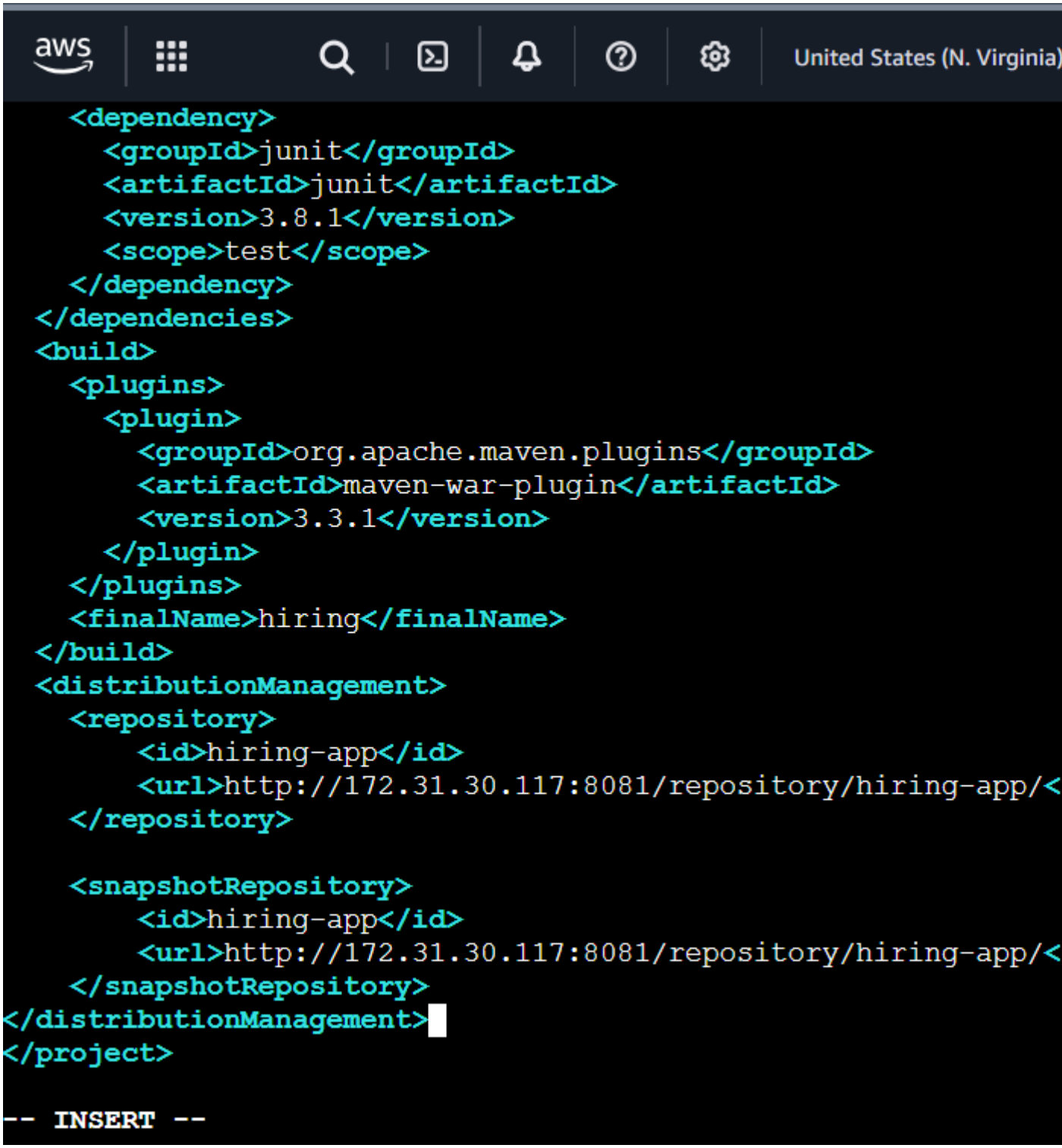
```
pom.xml
  |
  | id = hiring-app
  ↓
settings.xml
  |
  | id = hiring-app
  ↓
Nexus
  |
  ↓
hiring-app repository
```

Save it:

Esc

:wq

Enter



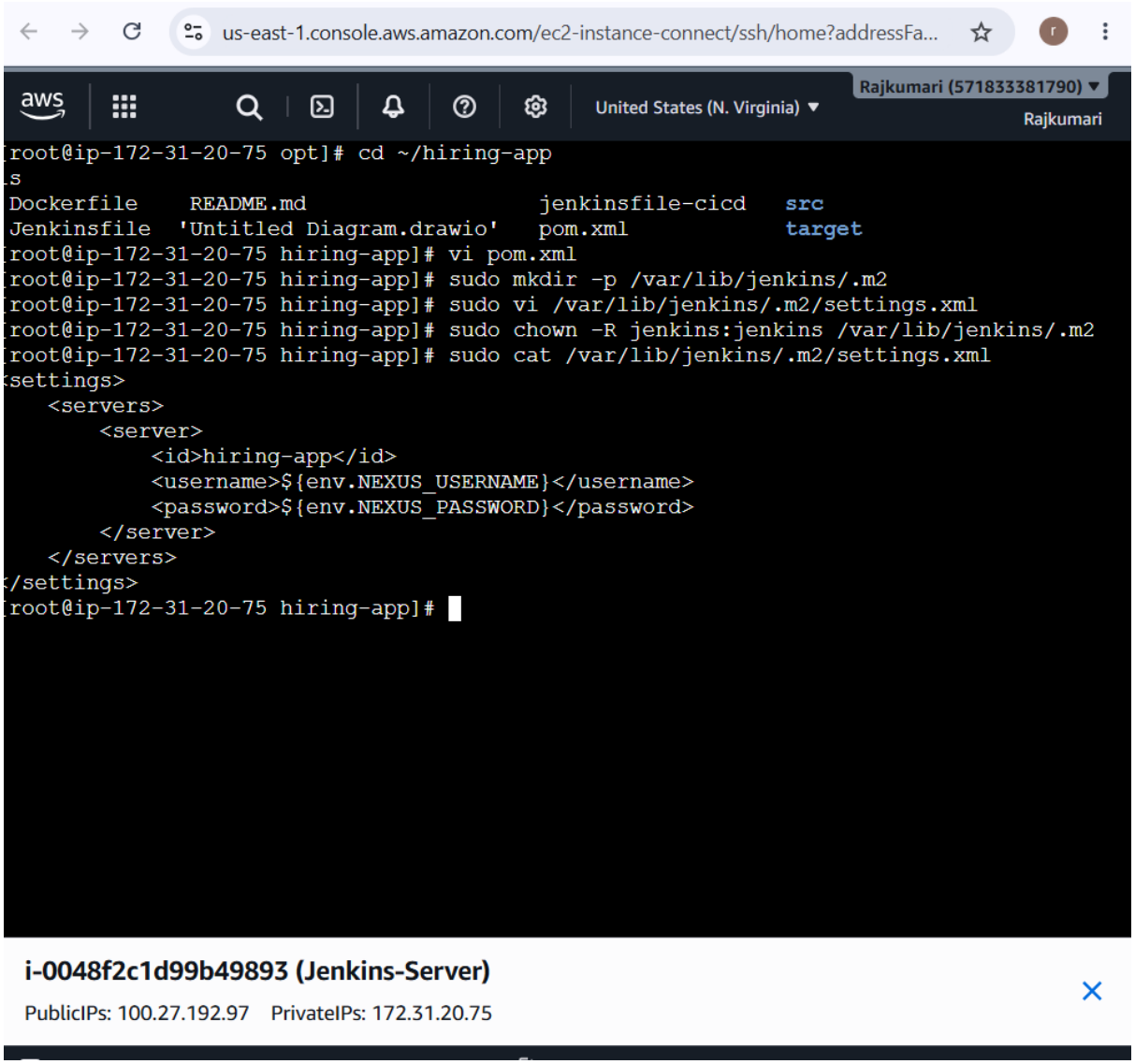
3. Set ownership

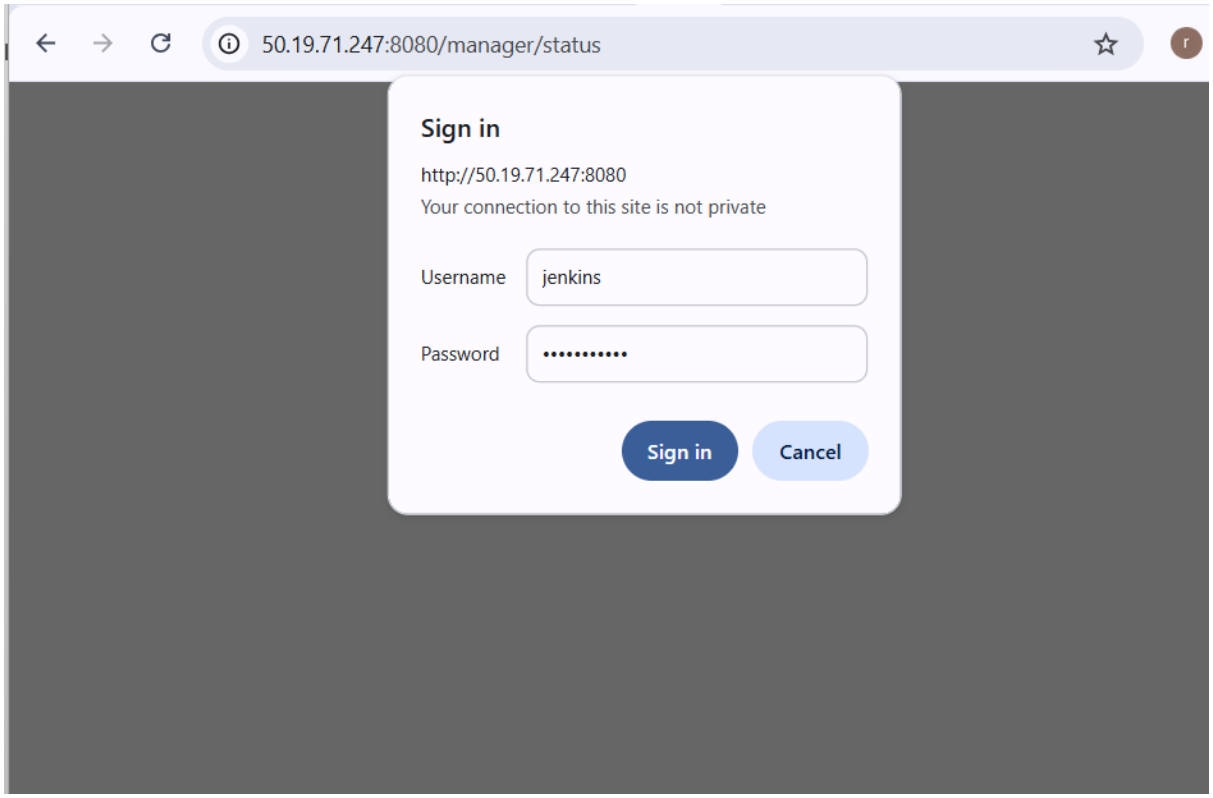
Run:

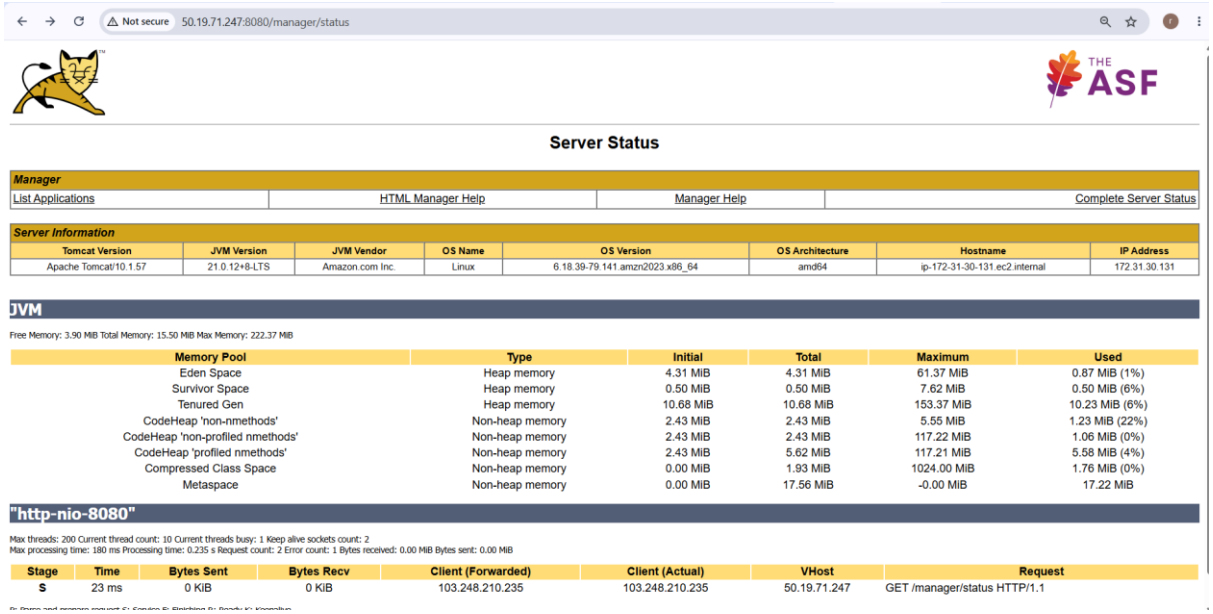
sudo chown -R jenkins:jenkins /var/lib/jenkins/.m2

Then verify:

sudo cat /var/lib/jenkins/.m2/settings.xml


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFa...
aws
United States (N. Virginia)
Rajkumari (571833381790)
Rajkumari
root@ip-172-31-20-75 opt]# cd ~/hiring-app
s
Dockerfile README.md jenkinsfile-cicd src
Jenkinsfile 'Untitled Diagram.drawio' pom.xml target
root@ip-172-31-20-75 hiring-app]# vi pom.xml
root@ip-172-31-20-75 hiring-app]# sudo mkdir -p /var/lib/jenkins/.m2
root@ip-172-31-20-75 hiring-app]# sudo vi /var/lib/jenkins/.m2/settings.xml
root@ip-172-31-20-75 hiring-app]# sudo chown -R jenkins:jenkins /var/lib/jenkins/.m2
root@ip-172-31-20-75 hiring-app]# sudo cat /var/lib/jenkins/.m2/settings.xml
<settings>
<servers>
<server>
<id>hiring-app</id>
<username>${env.NEXUS_USERNAME}</username>
<password>${env.NEXUS_PASSWORD}</password>
</server>
</servers>
</settings>
root@ip-172-31-20-75 hiring-app]#
i-0048f2c1d99b49893 (Jenkins-Server)
PublicIPs: 100.27.192.97 PrivateIPs: 172.31.20.75





Step 1 — Install Jenkins Tomcat deployment plugin

In Jenkins:

Manage Jenkins → Plugins → Available plugins

Search:

Deploy to container

Install:

Deploy to container Plugin

If Jenkins asks for a restart, restart Jenkins.

Step 2 — Add Tomcat credentials to Jenkins

Go to:

Manage Jenkins → Credentials

Then:

System → Global credentials → Add Credentials

Select:

Kind: Username with password

Enter:

Username: jenkins

Password: Jenkins@123

ID: tomcat-credentials

Description: Tomcat deployment

Click **Create**.

Step 3 — Configure your Freestyle job

Go to:

Jenkins → hiring-app → Configure

You already have your Git configuration:

Repository:

https://github.com/rajkumari-hub/hiring-app.git

Branch:

*/main

And your existing Maven build:

mvn clean deploy -s /var/lib/jenkins/.m2/settings.xml

This already uploads your WAR to Nexus.

Now add Tomcat deployment

Scroll to:

Post-build Actions

Click:

Add post-build action

Select:

Deploy war/ear to a container

You should get a configuration section.

WAR/EAR files

Enter:

target/hiring.war

Containers

Select:

Tomcat 10.x

Credentials

Select:

tomcat-credentials

Tomcat URL

Enter:

http://172.31.30.131:8080

Then **Save**.

Step 1 — Don't use "Deploy war/ear to a container"

In your Freestyle job:

Jenkins → hiring-app → Configure

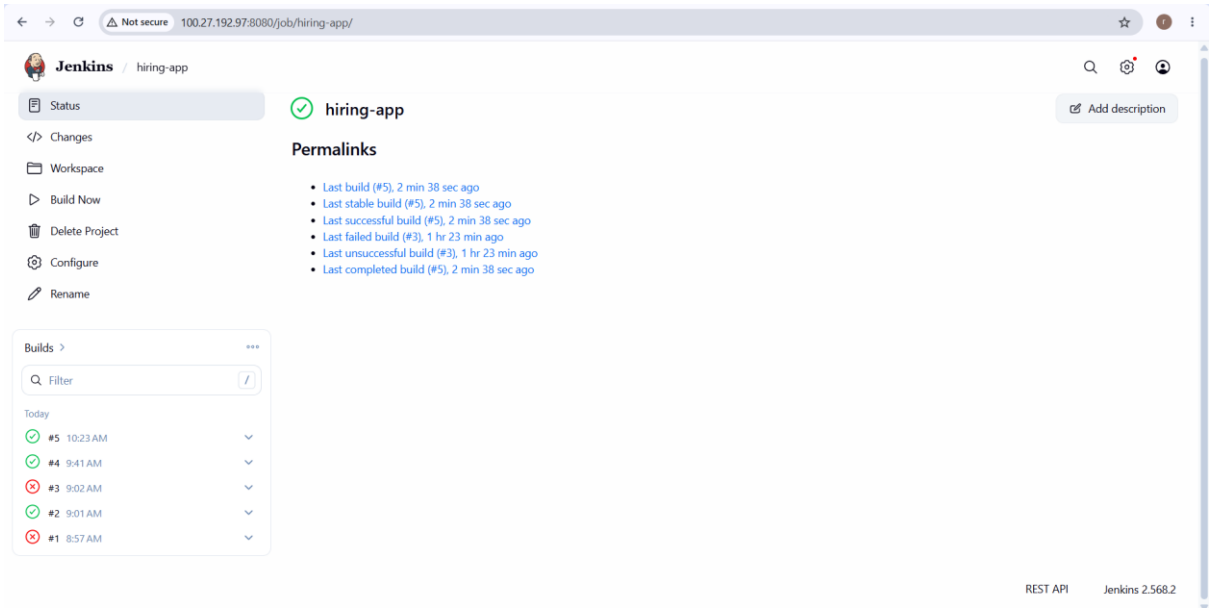
You can leave the plugin section unused.

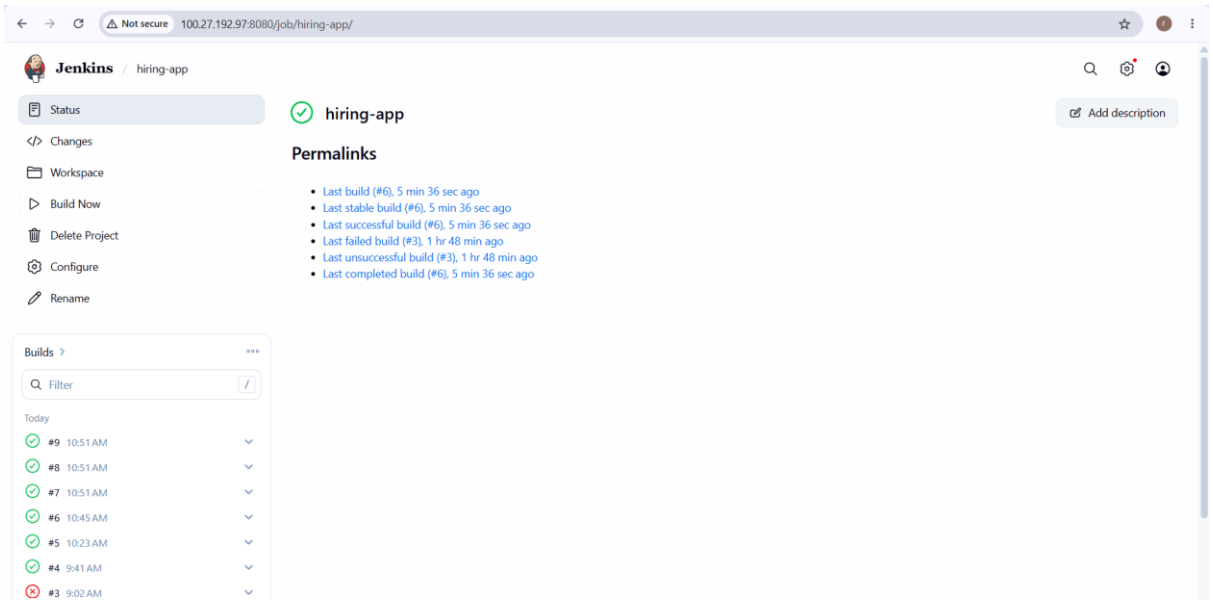
Go to:

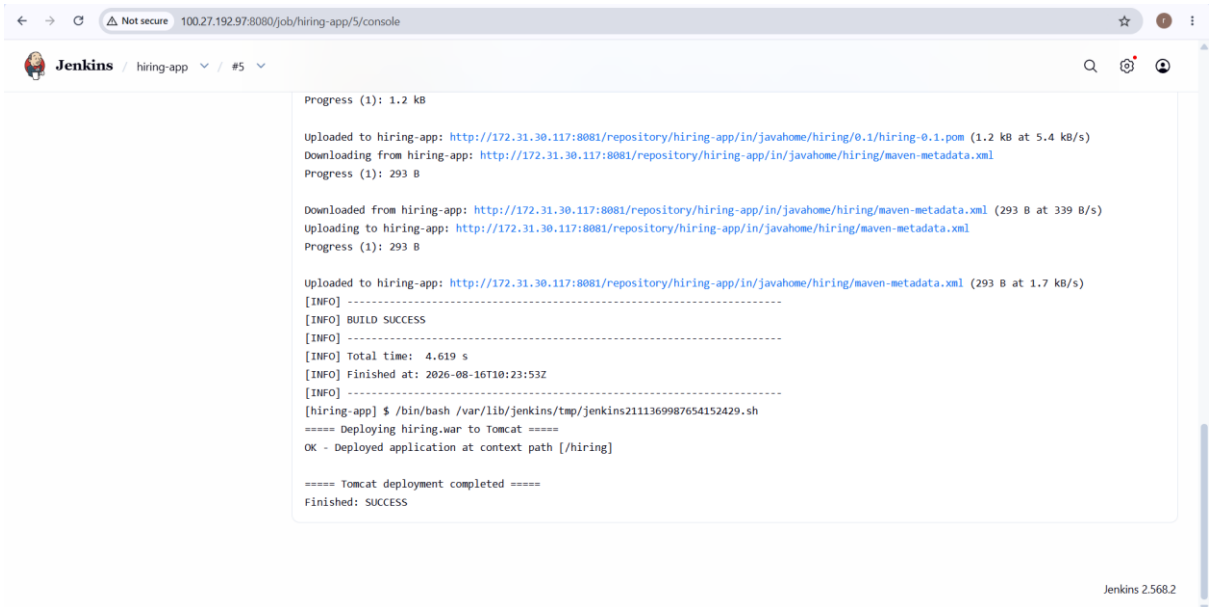
Build → Add build step → Execute shell

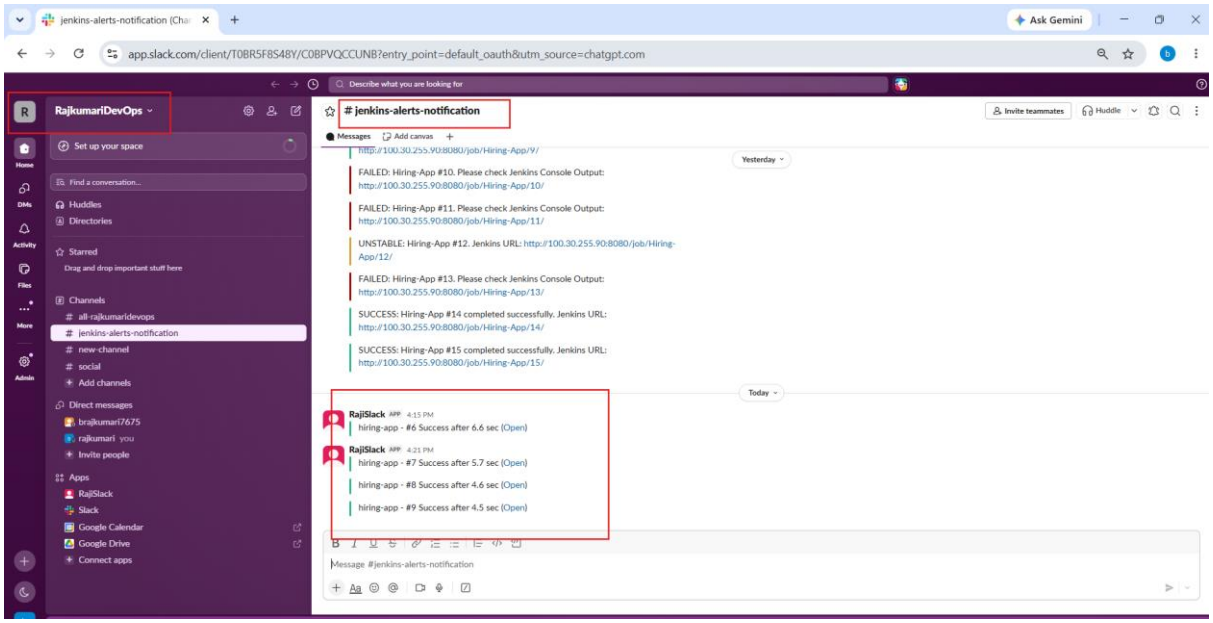
Add this:

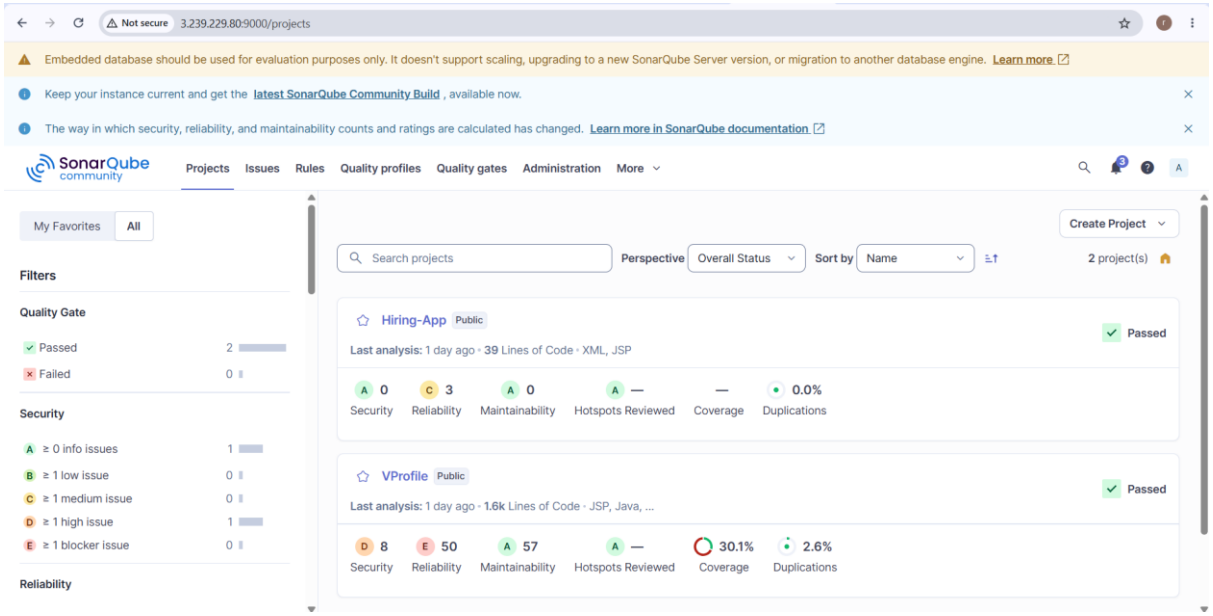
```
#!/bin/bash


TOMCAT_URL="http://172.31.30.131:8080"
TOMCAT_USER="jenkins"
TOMCAT_PASSWORD="Jenkins@123"
WAR_FILE="target/hiring.war"


echo "===== Deploying hiring.war to Tomcat ====="


curl --fail --silent --show-error \
    -u "${TOMCAT_USER}:${TOMCAT_PASSWORD}" \
    -T "${WAR_FILE}" \
    "${TOMCAT_URL}/manager/text/deploy?path=/hiring&update=true"


echo
echo "===== Tomcat deployment completed ====="
```

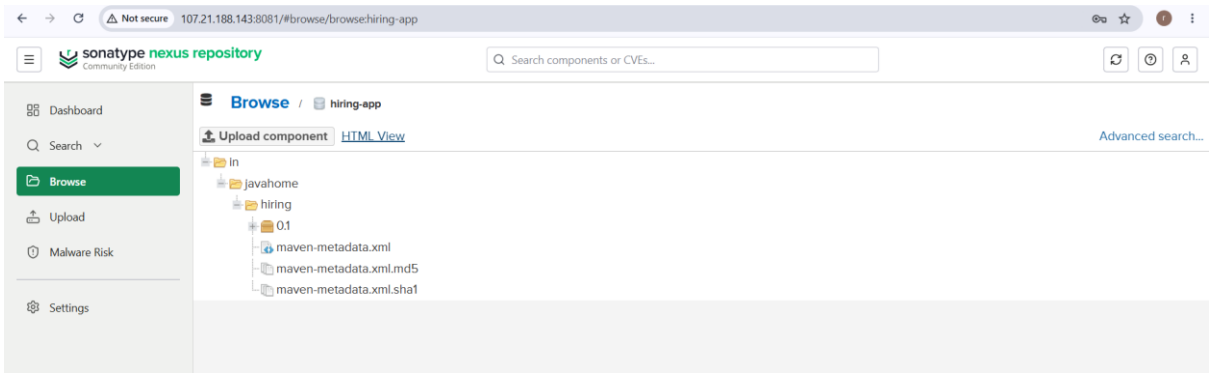

Not secure 100.27.192.97:8080/job/hiring-app/
Jenkins / hiring-app
Status
Changes
Workspace
Build Now
Delete Project
Configure
Rename
hiring-app
Add description
Permalinks
Last build (#5), 2 min 38 sec ago
Last stable build (#5), 2 min 38 sec ago
Last successful build (#5), 2 min 38 sec ago
Last failed build (#3), 1 hr 23 min ago
Last unsuccessful build (#3), 1 hr 23 min ago
Last completed build (#5), 2 min 38 sec ago
Builds
Filter
Today
#5 10:23 AM
#4 9:41 AM
#3 9:02 AM
#2 9:01 AM
#1 8:57 AM
REST API
Jenkins 2.568.2

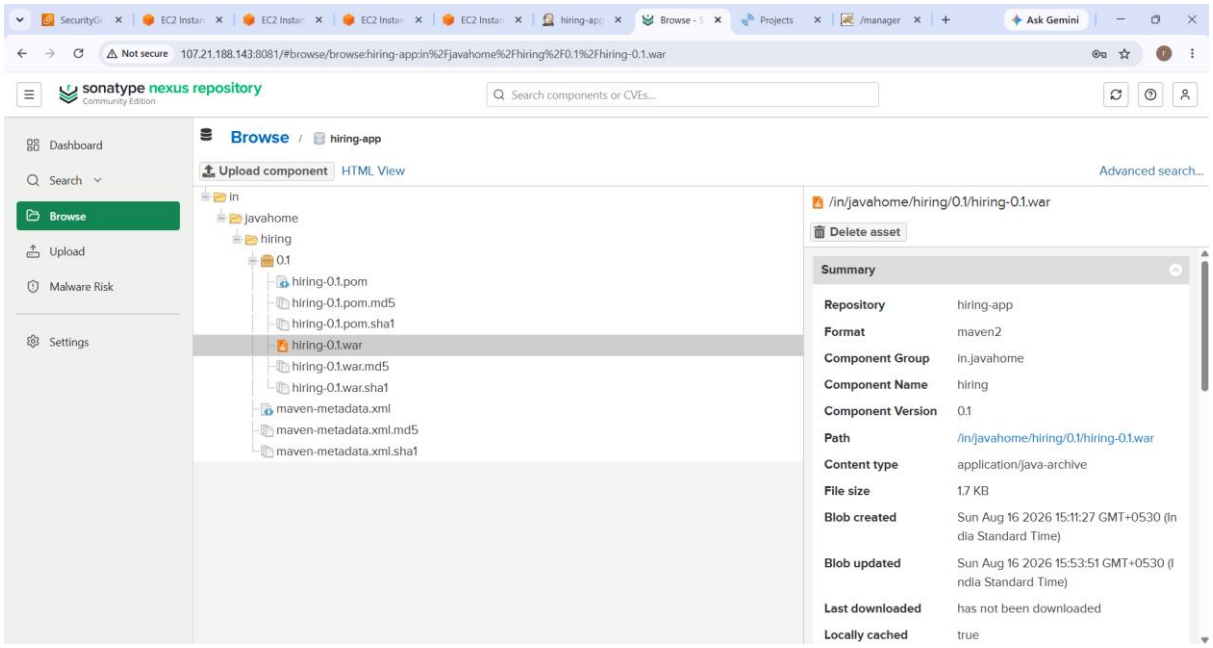

Not secure 100.27.192.97:8080/job/hiring-app/
Jenkins / hiring-app
Status
Changes
Workspace
Build Now
Delete Project
Configure
Rename
hiring-app
Add description
Permalinks
Last build (#6), 5 min 36 sec ago
Last stable build (#6), 5 min 36 sec ago
Last successful build (#6), 5 min 36 sec ago
Last failed build (#3), 1 hr 48 min ago
Last unsuccessful build (#3), 1 hr 48 min ago
Last completed build (#6), 5 min 36 sec ago
Builds
Filter
Today
#9 10:51 AM
#8 10:51 AM
#7 10:51 AM
#6 10:45 AM
#5 10:23 AM
#4 9:41 AM
#3 9:02 AM

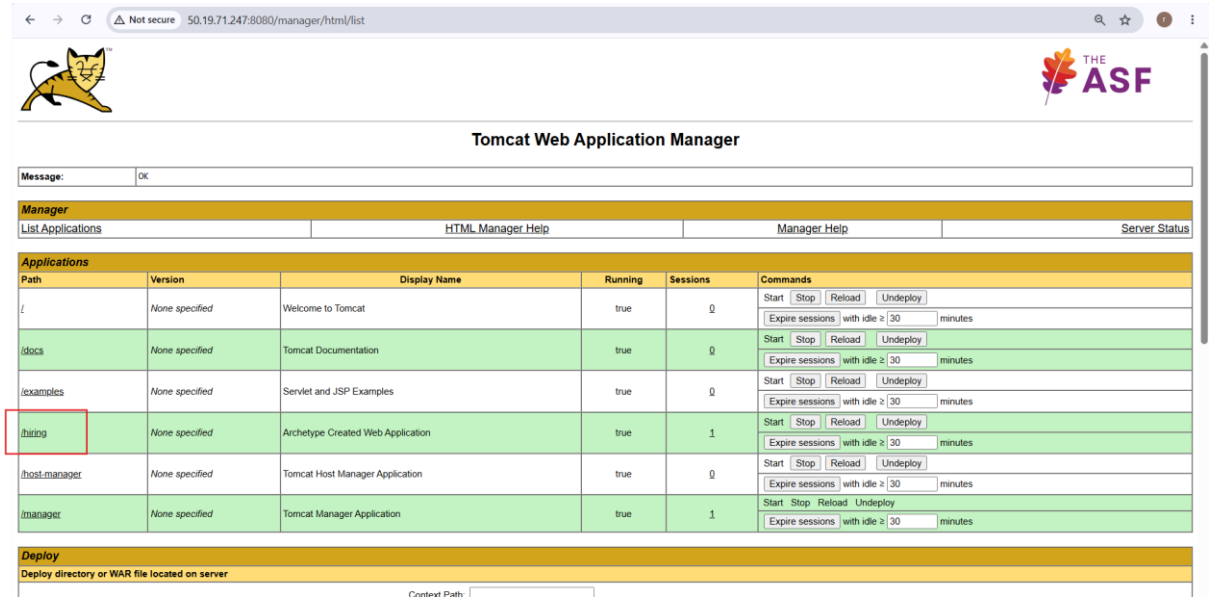

Not secure 100.27.192.97:8080/job/hiring-app/5/console
Jenkins / hiring-app / #5
Progress (1): 1.2 kB
Uploaded to hiring-app: http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/0.1/hiring-0.1.pom (1.2 kB at 5.4 kB/s)
Downloading from hiring-app: http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml
Progress (1): 293 B
Downloaded from hiring-app: http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml (293 B at 339 B/s)
Uploading to hiring-app: http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml
Progress (1): 293 B
Uploaded to hiring-app: http://172.31.30.117:8081/repository/hiring-app/in/javahome/hiring/maven-metadata.xml (293 B at 1.7 kB/s)
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 4.619 s
[INFO] Finished at: 2026-08-16T10:23:53Z
[INFO] --
[hiring-app] $ /bin/bash /var/lib/jenkins/tmp/jenkins2111369987654152429.sh
===== Deploying hiring.war to Tomcat =====
OK - Deployed application at context path [/hiring]
===== Tomcat deployment completed =====
Finished: SUCCESS
Jenkins 2.568.2


jenkins-alerts-notification
app.slack.com/client/T0BR5F8S48Y/C0BPVQCCUNB?entry_point=default_oauth&utm_source=chatgpt.com
RajkumariDevOps
jenkins-alerts-notification
FAILED: Hiring-App #10. Please check Jenkins Console Output:
FAILED: Hiring-App #11. Please check Jenkins Console Output:
UNSTABLE: Hiring-App #12. Jenkins URL:
FAILED: Hiring-App #13. Please check Jenkins Console Output:
SUCCESS: Hiring-App #14 completed successfully. Jenkins URL:
SUCCESS: Hiring-App #15 completed successfully. Jenkins URL:
RajiSlack
hiring-app - #6 Success after 6.6 sec (Open)
hiring-app - #7 Success after 5.7 sec (Open)
hiring-app - #8 Success after 4.6 sec (Open)
hiring-app - #9 Success after 4.5 sec (Open)


Not secure 3.239.229.80:9000/projects
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
Keep your instance current and get the latest SonarQube Community Build, available now.
The way in which security, reliability, and maintainability counts and ratings are calculated has changed. Learn more in SonarQube documentation
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
My Favorites All
Filters
Quality Gate
Passed 2
Failed 0
Security
Reliability
Create Project
Search projects
Perspective Overall Status
Sort by Name
2 project(s)
Hiring-App Public
Last analysis: 1 day ago · 39 Lines of Code · XML, JSP
Passed
VProfile Public
Last analysis: 1 day ago · 1.6k Lines of Code · JSP, Java, ...
Passed
Coverage
Duplications


Not secure 107.21.188.143:8081/#browse/browse:hiring-app
sonatype nexus repository
Community Edition
Search components or CVEs...
Dashboard
Search
Browse
Upload
Malware Risk
Settings
Browse / hiring-app
Upload component
HTML View
Advanced search...
in
javahome
hiring
0.1
maven-metadata.xml
maven-metadata.xml.md5
maven-metadata.xml.sha1


Not secure 107.21.188.143:8081/#browse/browse:hiring-app:in%2Fjavahome%2Fhiring%2F0.1%2Fhiring-0.1.war
sonatype nexus repository
Community Edition
Search components or CVEs...
Dashboard
Search
Browse
Upload
Malware Risk
Settings
Browse / hiring-app
Upload component
HTML View
Advanced search...
in
javahome
hiring
0.1
hiring-0.1.pom
hiring-0.1.pom.md5
hiring-0.1.pom.sha1
hiring-0.1.war
hiring-0.1.war.md5
hiring-0.1.war.sha1
maven-metadata.xml
maven-metadata.xml.md5
maven-metadata.xml.sha1
/in/javahome/hiring/0.1/hiring-0.1.war
Delete asset
Summary
Repository hiring-app
Format maven2
Component Group in.javahome
Component Name hiring
Component Version 0.1
Path /in/javahome/hiring/0.1/hiring-0.1.war
Content type application/java-archive
File size 1.7 KB
Blob created Sun Aug 16 2026 15:11:27 GMT+0530 (India Standard Time)
Blob updated Sun Aug 16 2026 15:53:51 GMT+0530 (India Standard Time)
Last downloaded has not been downloaded
Locally cached true


Not secure 50.19.71.247:8080/manager/html/list
THE ASF
Tomcat Web Application Manager
Message: OK
Manager
List Applications | HTML Manager Help | Manager Help | Server Status
Applications
Path | Version | Display Name | Running | Sessions | Commands
/ None specified Welcome to Tomcat true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs None specified Tomcat Documentation true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples None specified Servlet and JSP Examples true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiring None specified Archetype Created Web Application true 1 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager None specified Tomcat Host Manager Application true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager None specified Tomcat Manager Application true 1 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
Deploy
Deploy directory or WAR file located on server
Context Path:


Not secure 50.19.71.247:8080/hiring/?utm_source=chatgpt.com
Welcome to Techie Horizon

2) Setup a Jenkins CICD pipeline using Declarative pipeline using feature-1.1 branch.

Source Code: https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages: Git Clone

SonarQube Integration

Maven Compilation

Nexus Artifactory

Slack Notification

Deploy On tomcat

The required flow is:

GitHub (feature-1.1) → Git Clone → SonarQube → Maven Build → Nexus → Slack → Tomcat

Jenkins Declarative Pipeline must use the top-level pipeline {} structure, and Maven can be configured through Jenkins' tools directive.

Before starting: values you need

Keep these ready:

Item	Example
Git repository	https://github.com/betawins/sabear_simplecutomerapp.git
Branch	feature-1.1

Item	Example
Jenkins	Your Jenkins EC2
SonarQube	http://SONAR_PRIVATE_IP:9000
Nexus	http://NEXUS_PRIVATE_IP:8081
Tomcat	http://TOMCAT_PRIVATE_IP:8080
Slack workspace	rajislack
Slack channel	#jenkins-alerts-notification

Part 1 — Prepare Jenkins

Step 1 — Verify Jenkins server

SSH into your Jenkins EC2:

ssh -i your-key.pem ec2-user@<JENKINS-PUBLIC-IP>

Check:

java -version

mvn -version

git --version

You should get versions for all three.

For your existing setup, you already have Java/Maven/Git installed, so don't reinstall them if these commands work.

Step 2 — Check the Git branch

Run:

git ls-remote --heads https://github.com/betawins/sabear_simplecutomerapp.git

You should see:

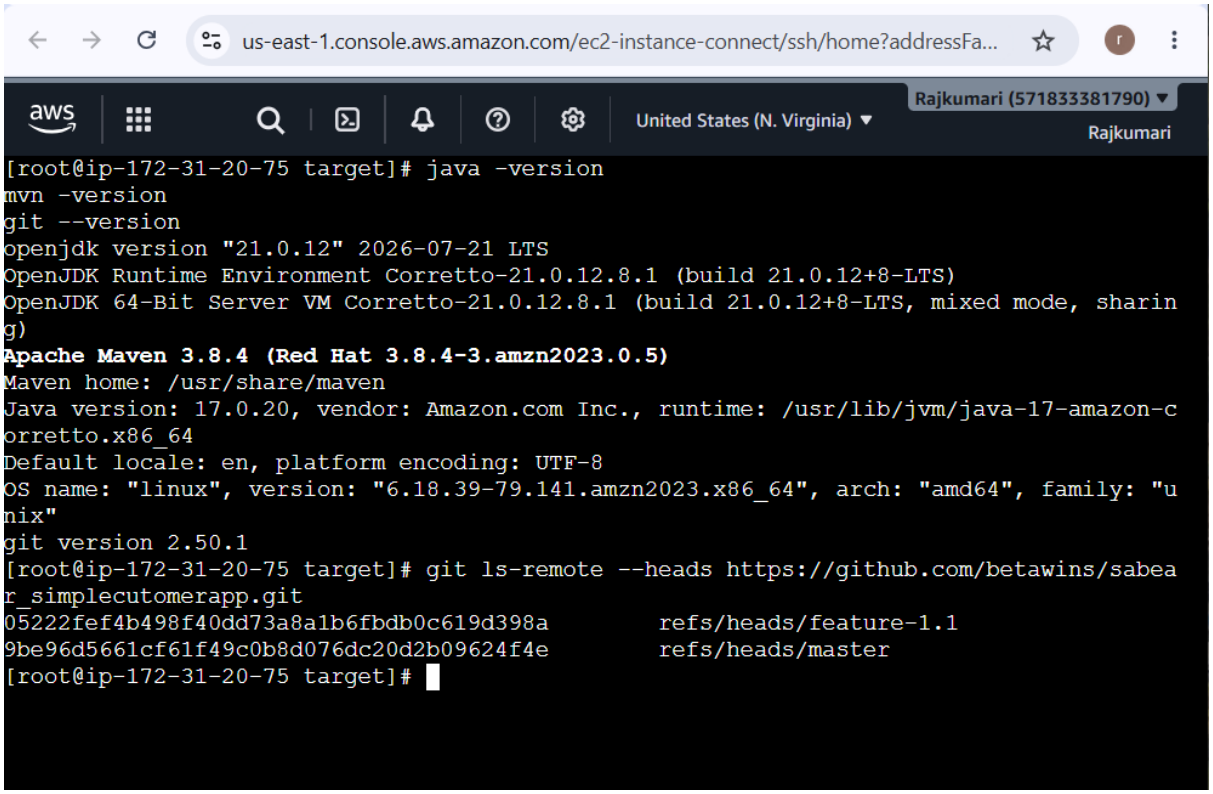
refs/heads/feature-1.1

This confirms the branch exists.

Your Jenkins pipeline will specifically clone:

feature-1.1

—not main and not master.



Step 3 — Install Jenkins plugins

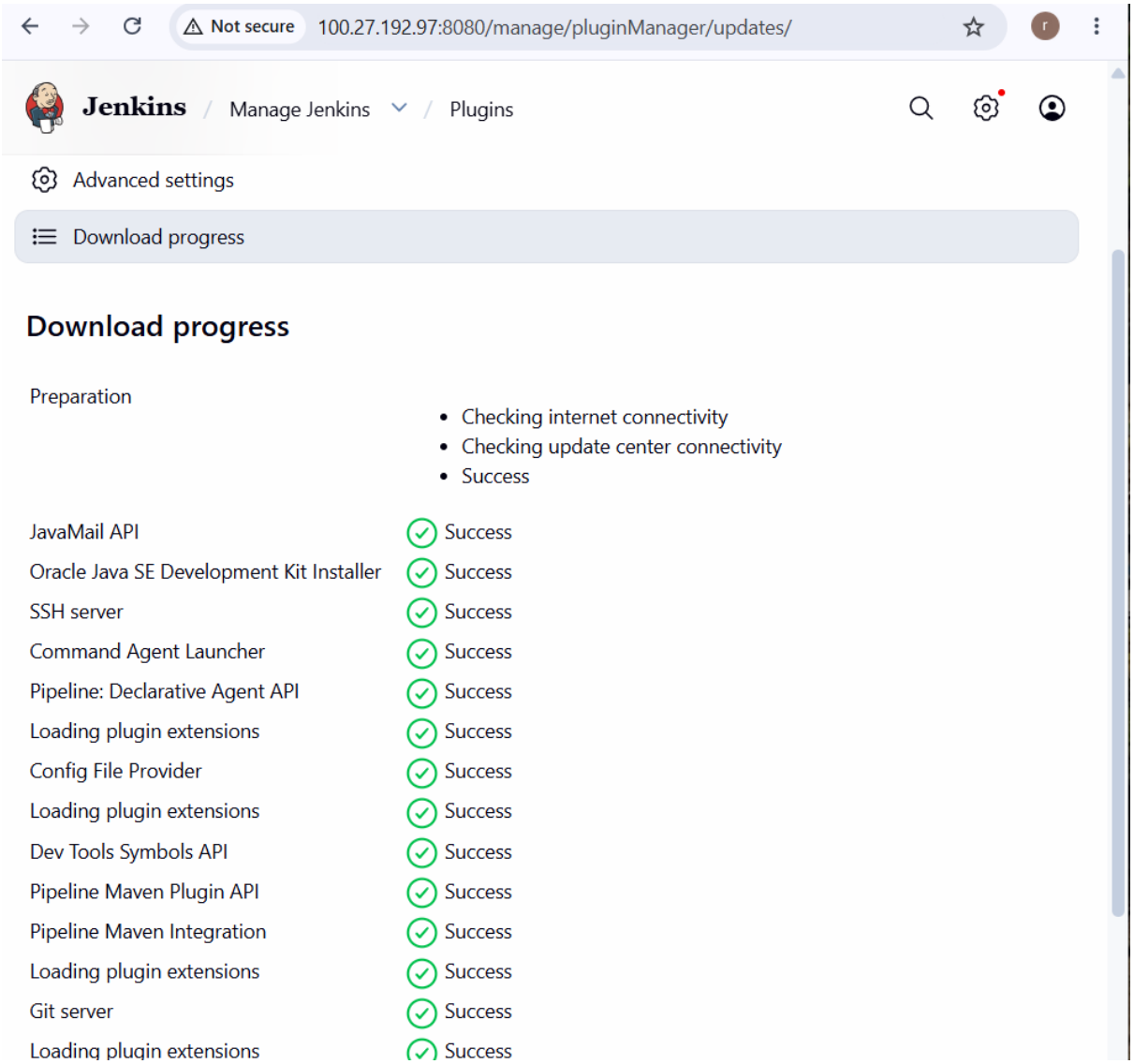
Go to:

Jenkins Dashboard → Manage Jenkins → Plugins → Available plugins

Make sure these are installed:

- Pipeline
- Pipeline: Declarative
- Git
- Git client
- SonarQube Scanner for Jenkins
- Credentials Binding
- Slack Notification
- Config File Provider
- Pipeline Maven Integration

Restart Jenkins if Jenkins asks you to.



Step 4 — Configure Maven

Go to:

Manage Jenkins → Tools

Find:

Maven installations

Click:

Add Maven

Use:

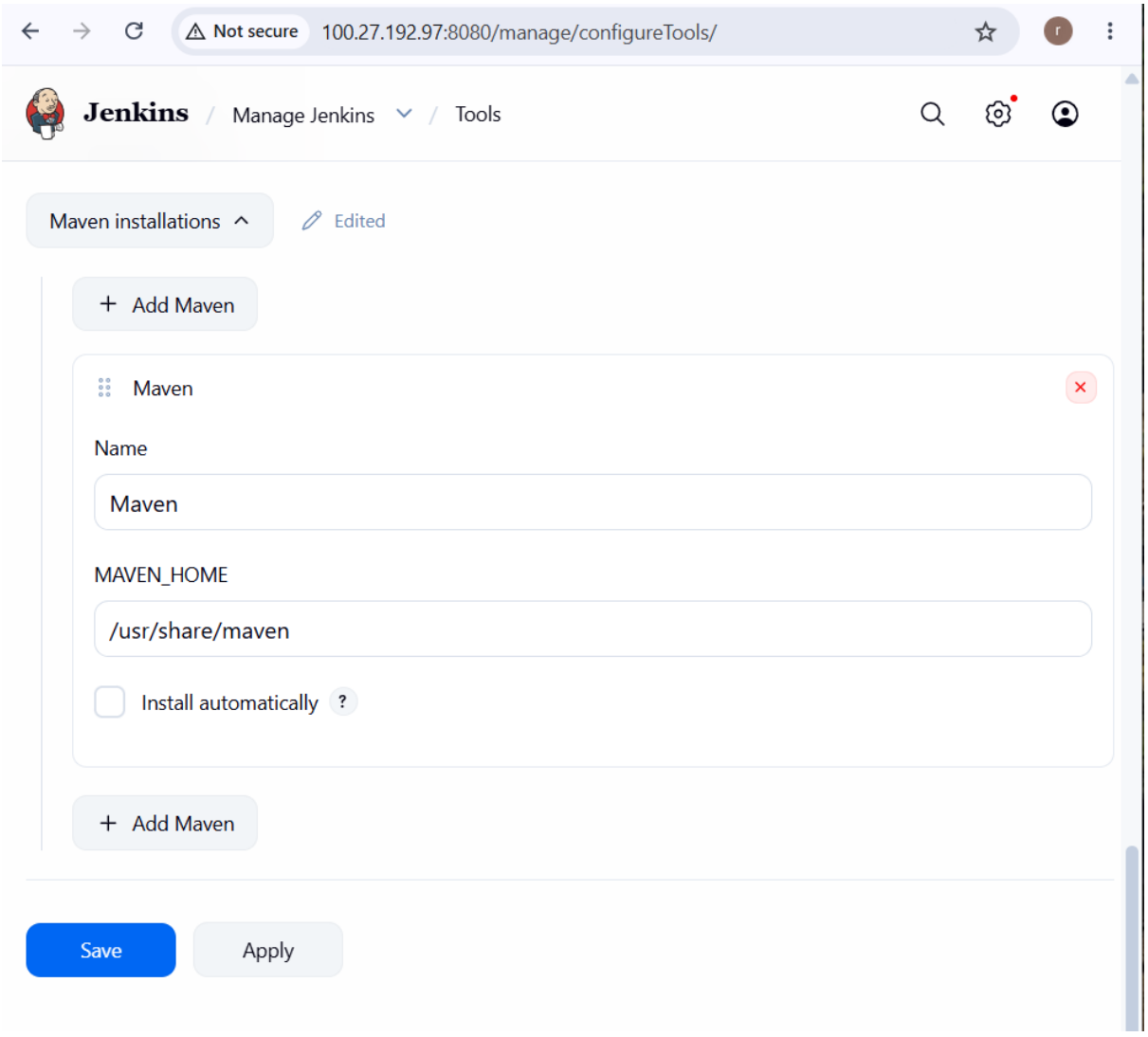
Name: Maven-3.8.4

If you are using the existing system Maven, you can also configure its installation path.

Your pipeline can then use:

```
tools {
    maven 'Maven-3.8.4'
}
```

Jenkins supports Maven under the Declarative Pipeline tools directive, provided the tool is configured under **Manage Jenkins → Tools**.



Part 2 — Configure SonarQube

Step 5 — Verify SonarQube

From Jenkins:

curl http://<SONARQUBE-PRIVATE-IP>:9000

You should receive SonarQube HTML output.

Also make sure your AWS Security Group allows:

Jenkins-SG → SonarQube-SG

TCP 9000

Step 6 — Configure SonarQube in Jenkins

Go to:

Manage Jenkins → System

Find:

SonarQube servers

Click:

Add SonarQube

Use:

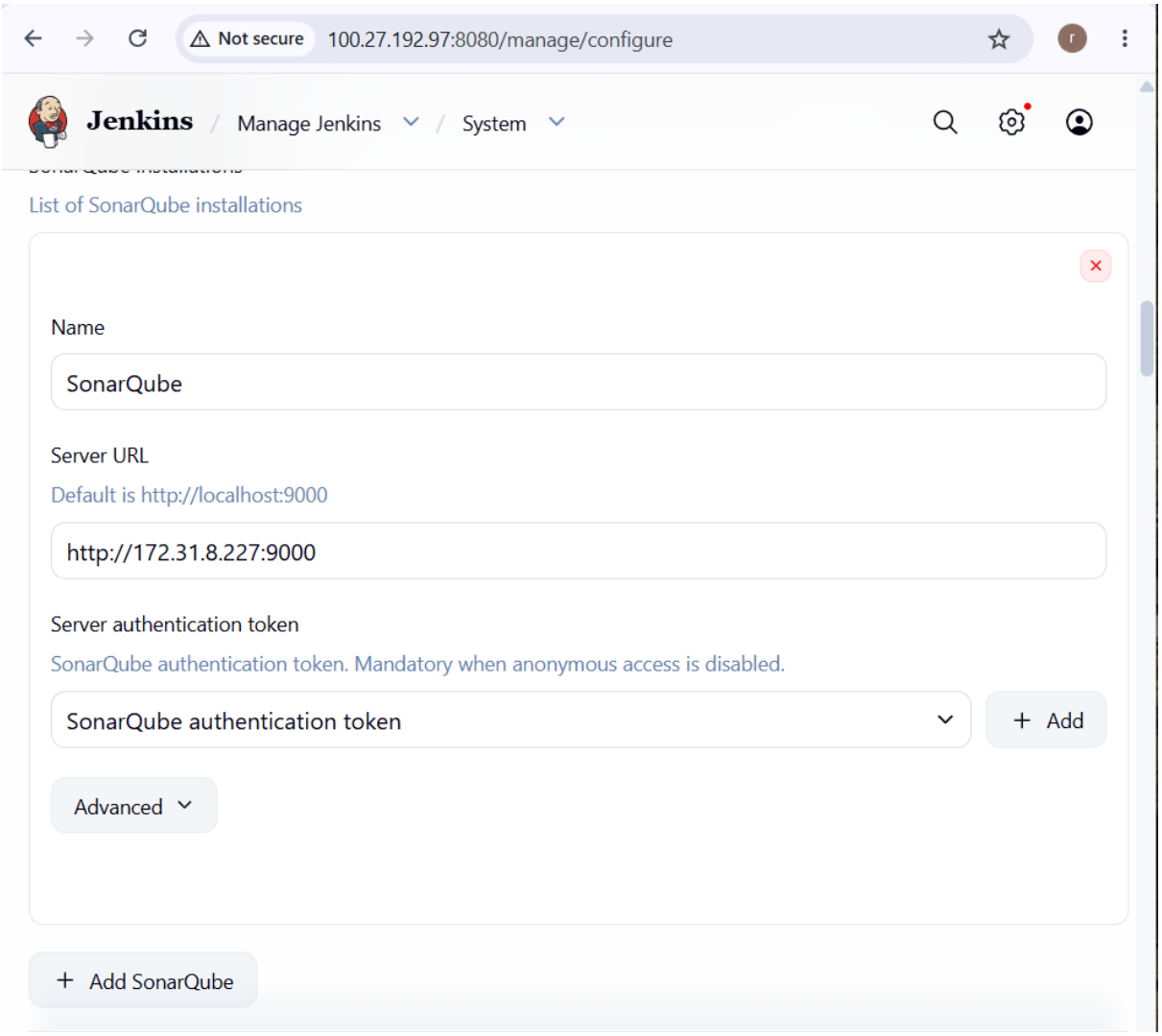
Name: SonarQube

Server URL: http://<SONARQUBE-PRIVATE-IP>:9000

Add the SonarQube authentication token through Jenkins Credentials if your SonarQube setup requires it.

The name **must match** what you later put in:

withSonarQubeEnv('SonarQube')



Part 3 — Configure Nexus

Step 7 — Verify Nexus from Jenkins

From Jenkins:

curl http://<NEXUS-PRIVATE-IP>:8081

You should receive Nexus response/HTML.

Make sure AWS Security Group allows:

Jenkins-SG

↓

Nexus-SG

TCP 8081

Step 8 — Create Nexus repository

Open:

http://<NEXUS-PUBLIC-IP>:8081

Log in to Nexus.

Go to:

Settings → Repositories → Create repository

For a release application, create:

maven2 (hosted)

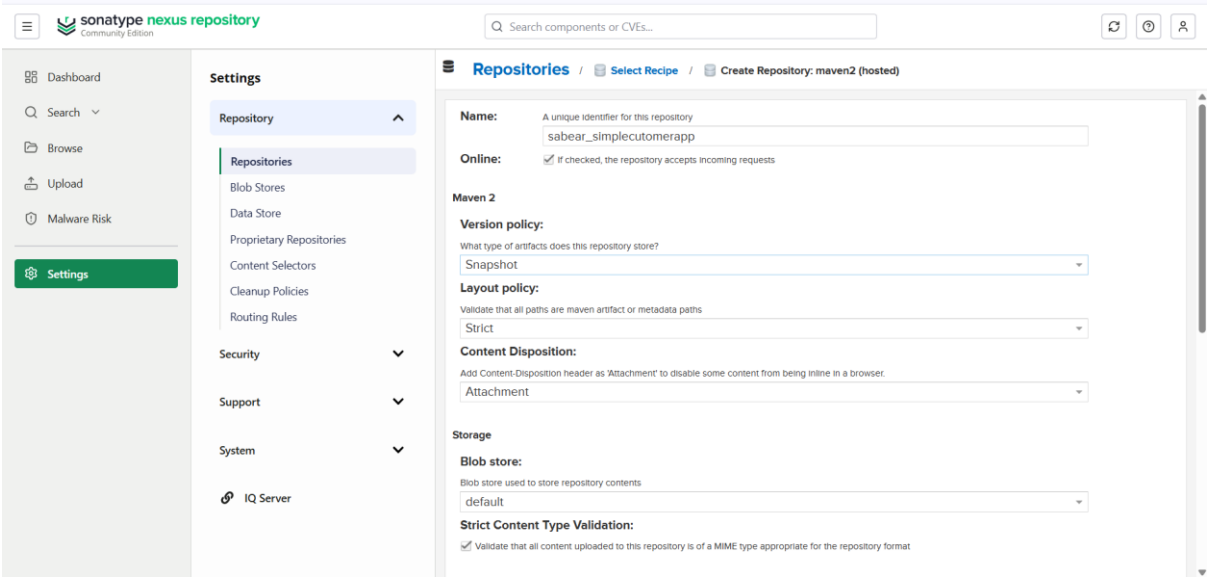
For example:

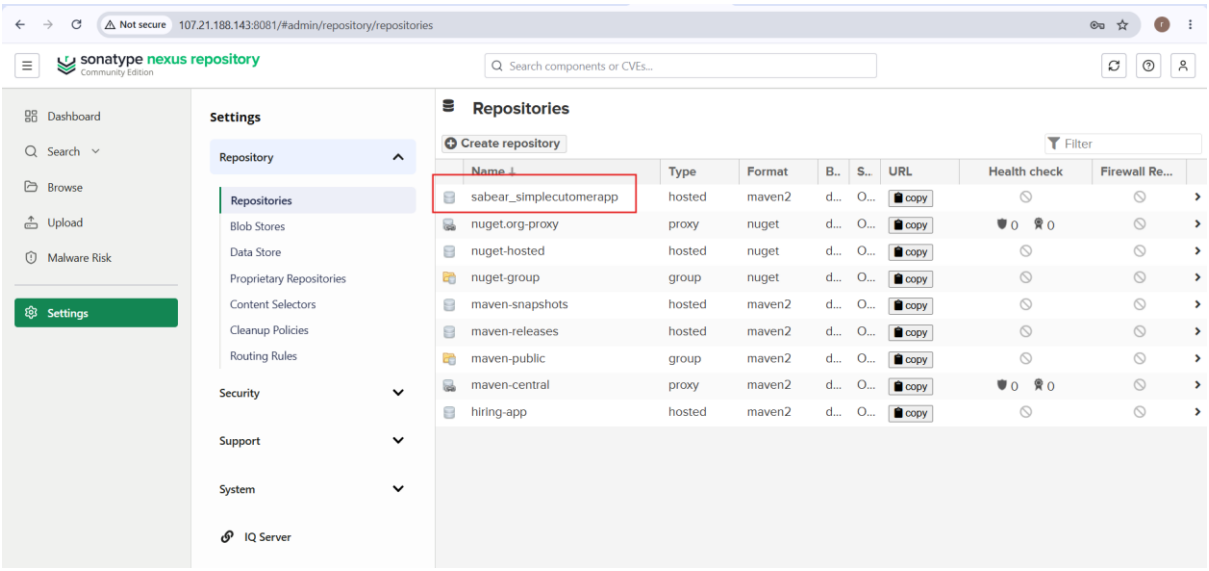
Name: hiring-releases

Version policy: Snapshot

Deployment policy: Allow redeploy

If your project's version ends in -SNAPSHOT, create/use a Maven snapshots repository instead.





Step 9 — Create Nexus Jenkins credential

In Jenkins:

Manage Jenkins → Credentials → System → Global credentials → Add Credentials

Select:

Kind: Username with password

Username: <NEXUS_USERNAME>

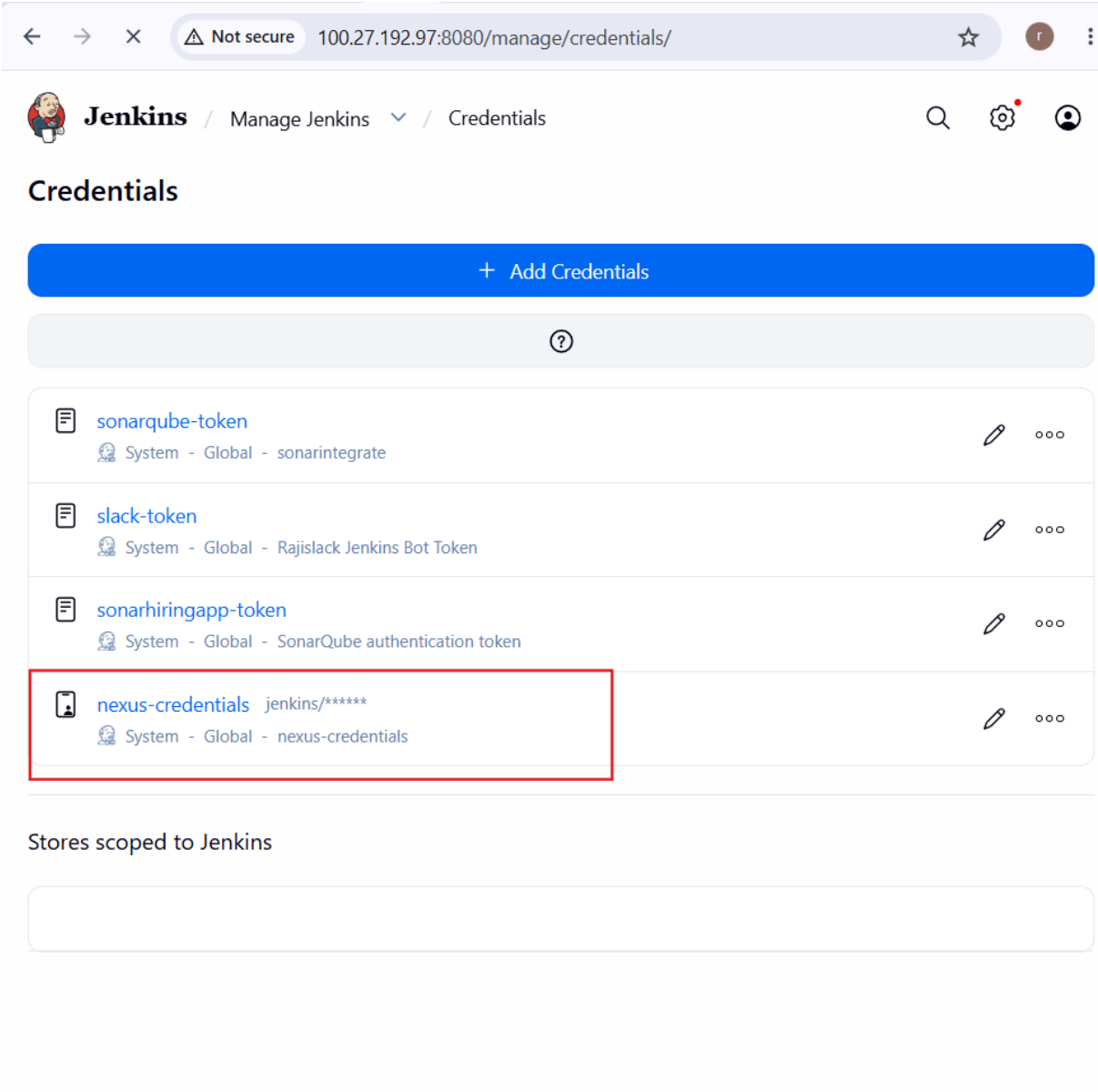
Password: <NEXUS_PASSWORD>

ID: nexus-credentials

Save it.

Don't put the Nexus password directly into your Jenkinsfile.

Jenkins' credentials binding mechanism is designed to expose credentials only within the scope where they're needed.



Step 10 — Check the project's pom.xml

This is **very important** before running the Nexus stage.

Clone the project temporarily:

cd /tmp

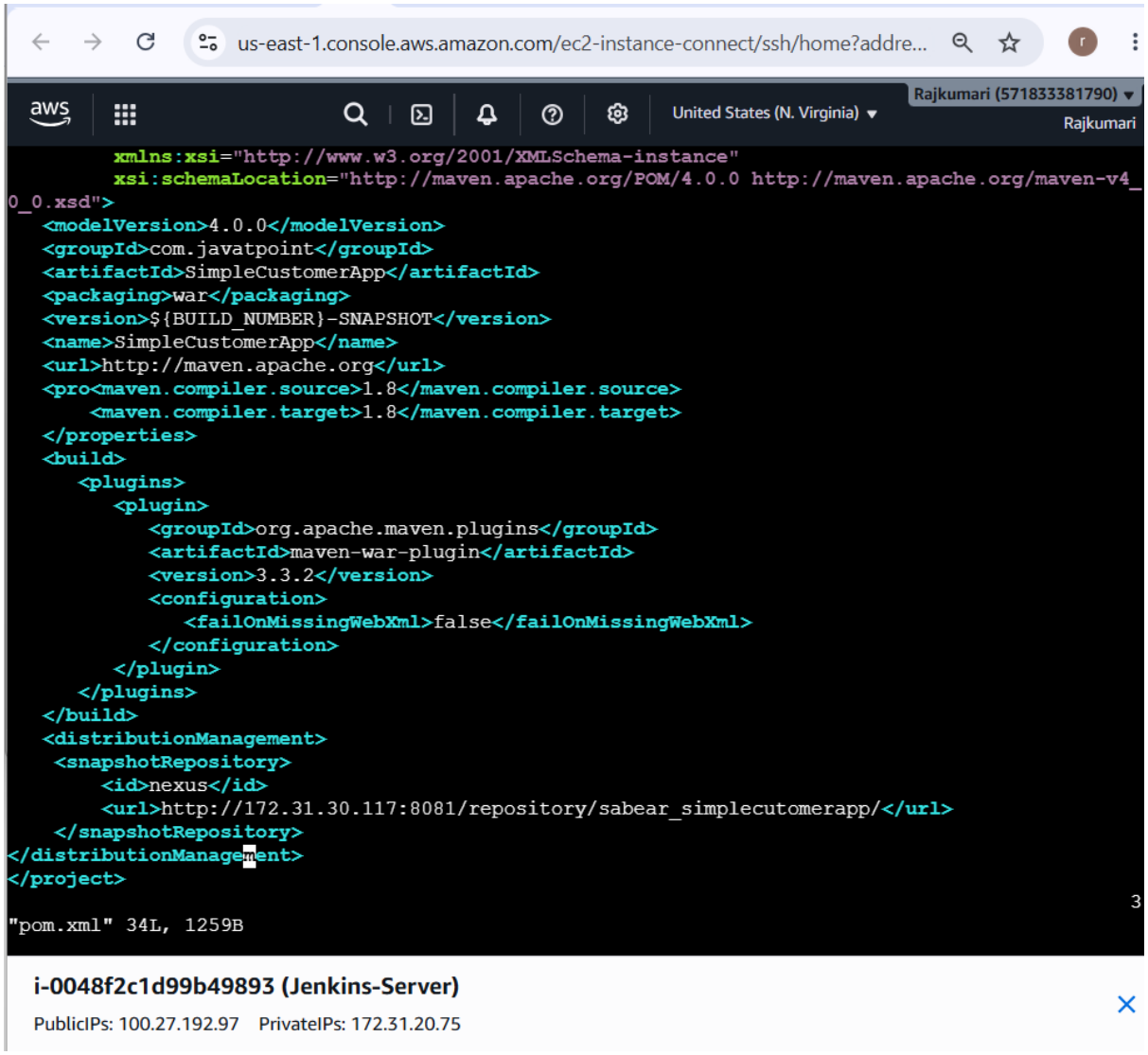
git clone -b feature-1.1 https://github.com/betawins/sabear_simplecutomerapp.git

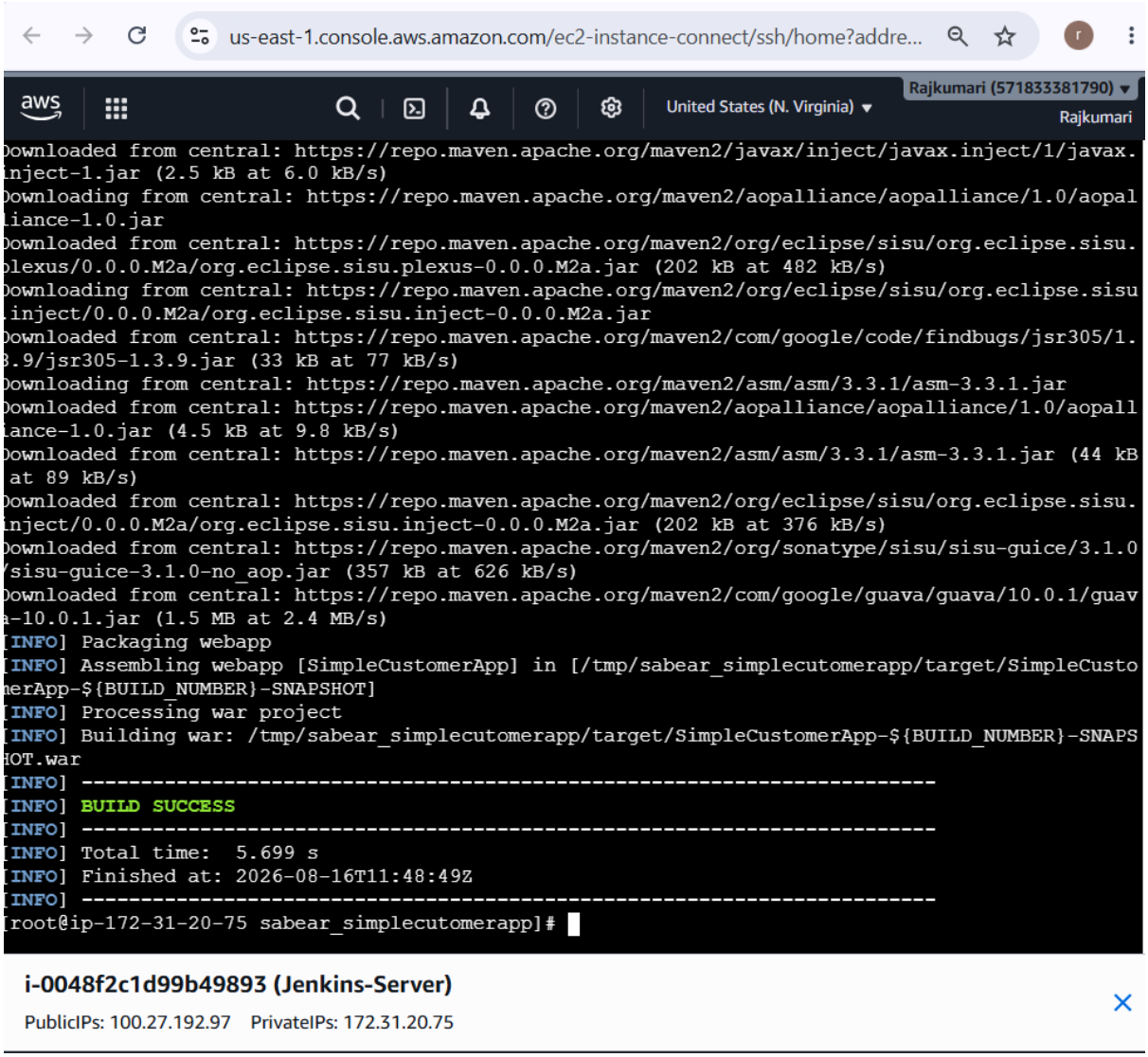
cd sabear_simplecutomerapp

Check:

grep -n "distributionManagement" pom.xml

If the project doesn't already have Nexus distributionManagement, you'll need to configure it.





Part 4 — Configure Maven → Nexus authentication

Step 11 — Create Maven settings.xml

Create a Maven settings file containing:

```
<settings>
  <servers>
    <server>
      <id>nexus</id>
      <username>YOUR_NEXUS_USERNAME</username>
      <password>YOUR_NEXUS_PASSWORD</password>
    </server>
```

```
  </servers>
</settings>
```

For your real assignment, don't commit this file containing a password to GitHub.

Instead, use Jenkins Credentials + Config File Provider.

Step 12 — Add settings.xml to Jenkins

Go to:

Manage Jenkins → Managed files

Create a Maven settings file.

For example:

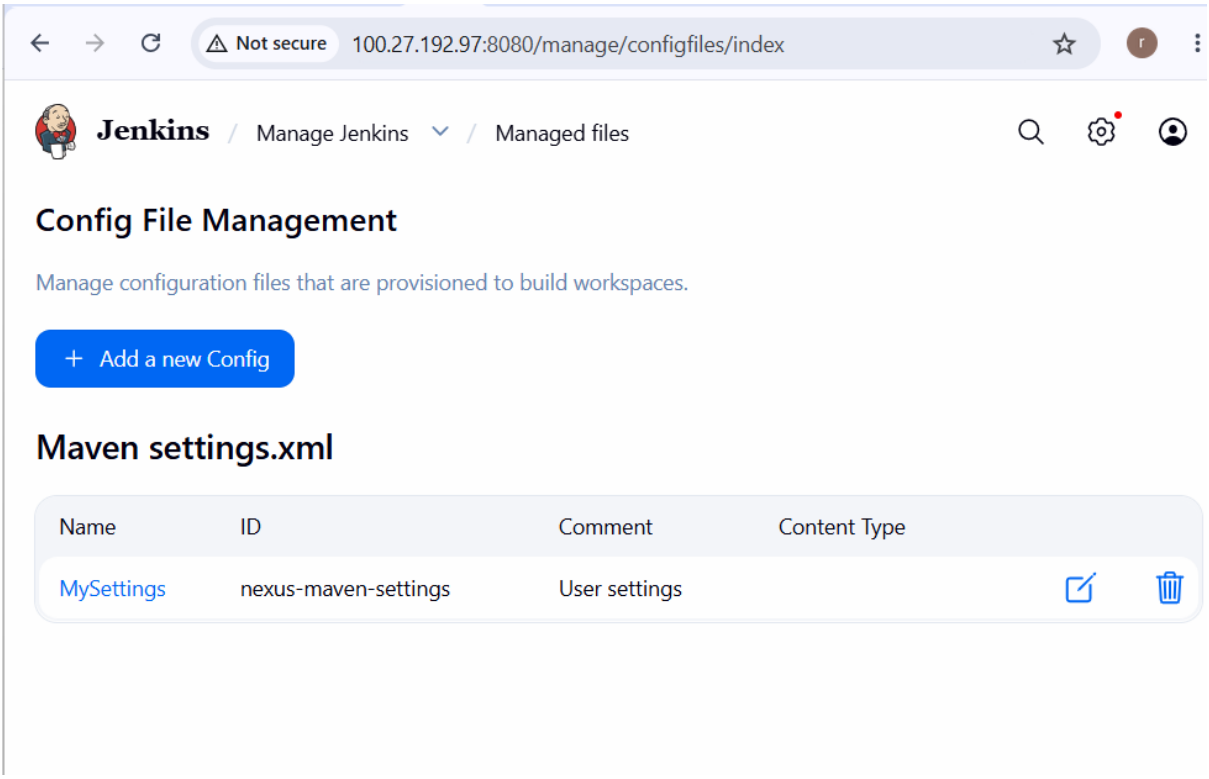
ID: nexus-settings

Configure the Nexus credentials appropriately.

Your pipeline can then use:

```
withMaven(
  mavenSettingsConfig: 'nexus-settings'
)
```

This keeps Nexus credentials outside your Jenkinsfile.



Part 5 — Prepare Tomcat

Step 13 — Verify Tomcat

SSH into your Tomcat server:

ssh -i your-key.pem ec2-user@<TOMCAT-PUBLIC-IP>

Check:

java -version

Then:

sudo systemctl status tomcat

Test:

curl http://localhost:8080

You should get the Tomcat page/response.

Step 14 — Configure Tomcat Manager

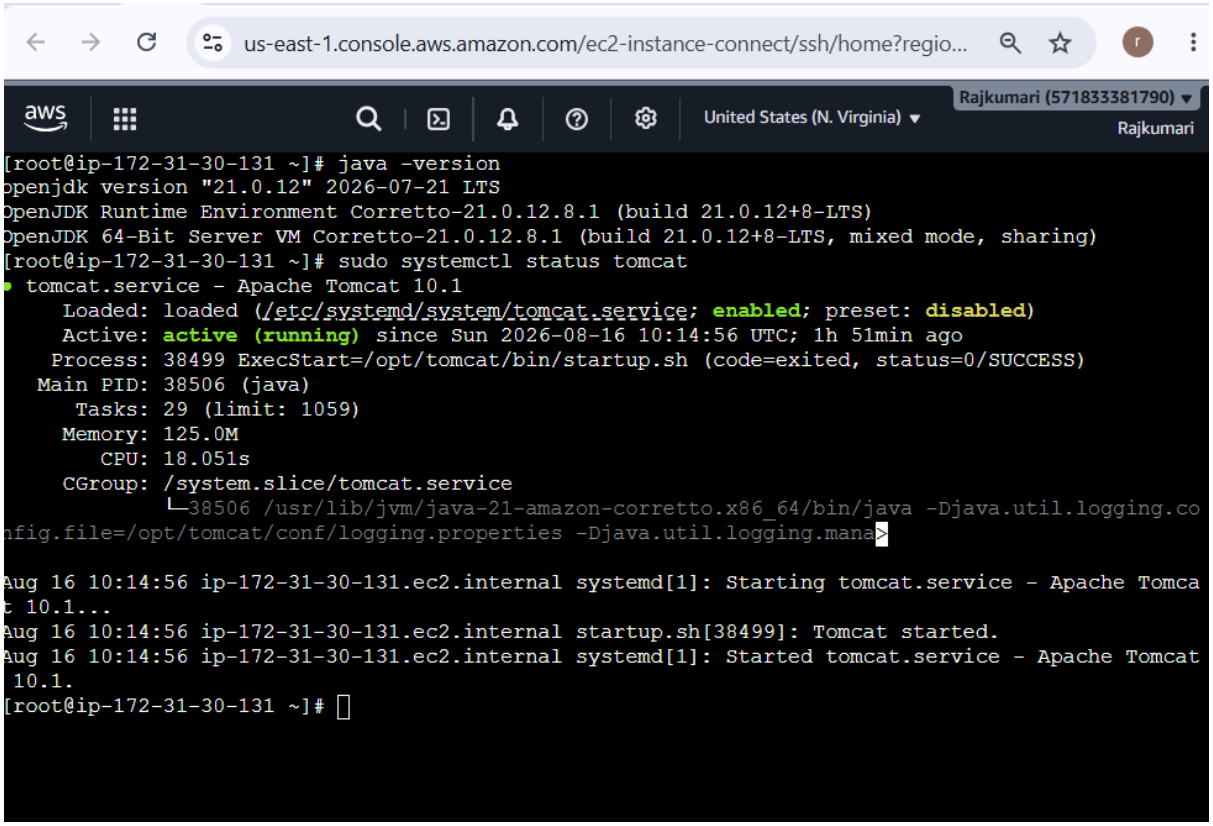
For automated WAR deployment, Jenkins can use Tomcat's Manager text interface.

Test from Jenkins:

curl http://<TOMCAT-PRIVATE-IP>:8080/manager/text/list

If authentication is required, use your Tomcat Manager credentials.

Tomcat's Manager application supports remote WAR deployment using an HTTP PUT request to /manager/text/deploy?path=/....



Step 15 — Create Tomcat deployment user

On Tomcat:

sudo vi /opt/tomcat/conf/tomcat-users.xml

Add an appropriate Manager role/user for your Tomcat version, for example:

<role rolename="manager-script"/>

<user username="jenkins"

password="YOUR_PASSWORD"

roles="manager-script"/>

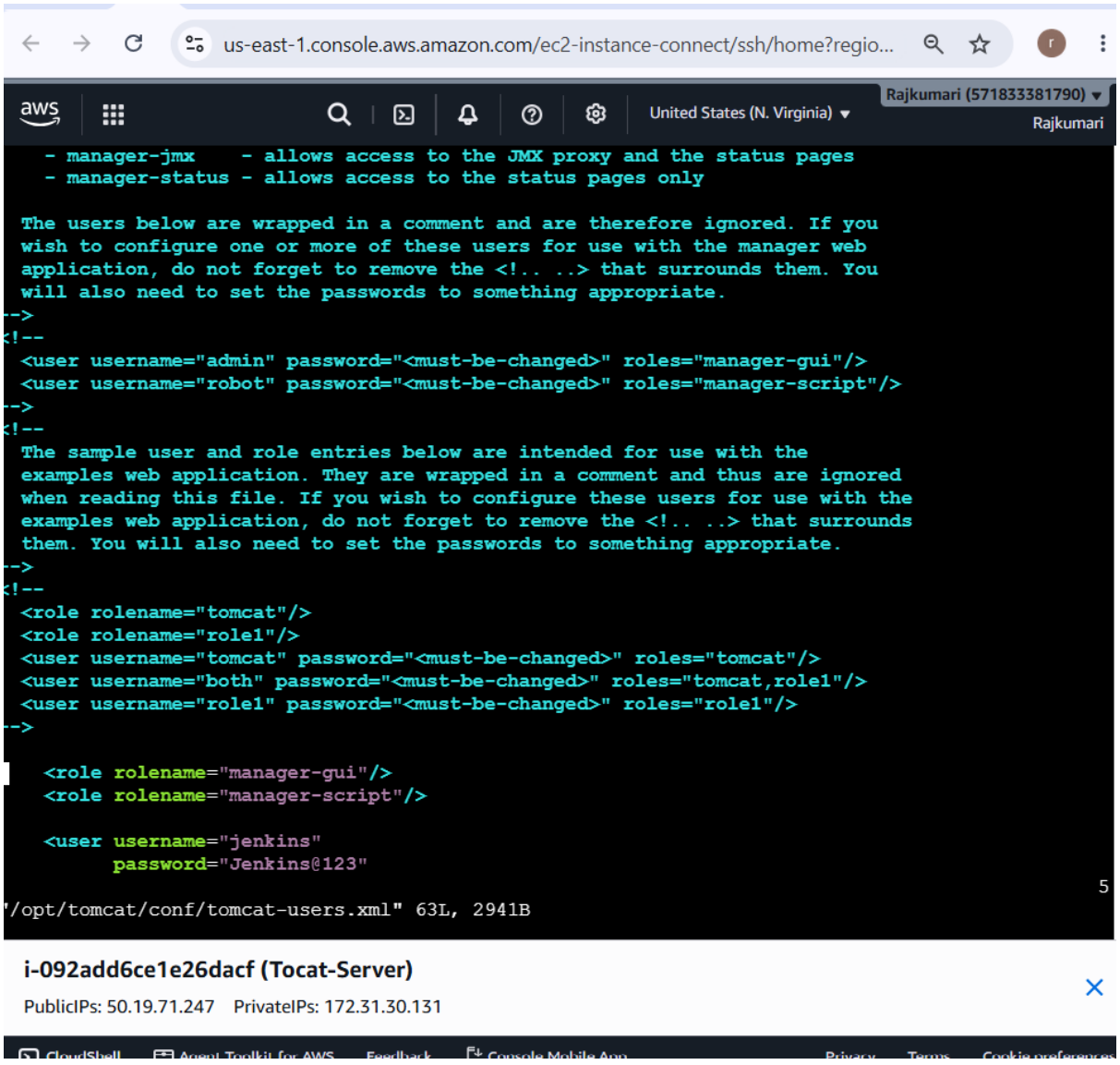
Then restart Tomcat:

sudo systemctl restart tomcat

Verify:

sudo systemctl status tomcat

Note: Tomcat Manager access restrictions may also need adjustment so that your Jenkins server can reach the Manager application.



Step 16 — Create Tomcat Jenkins credential

Go to:

Jenkins → Manage Jenkins → Credentials → Global → Add Credentials

Create:

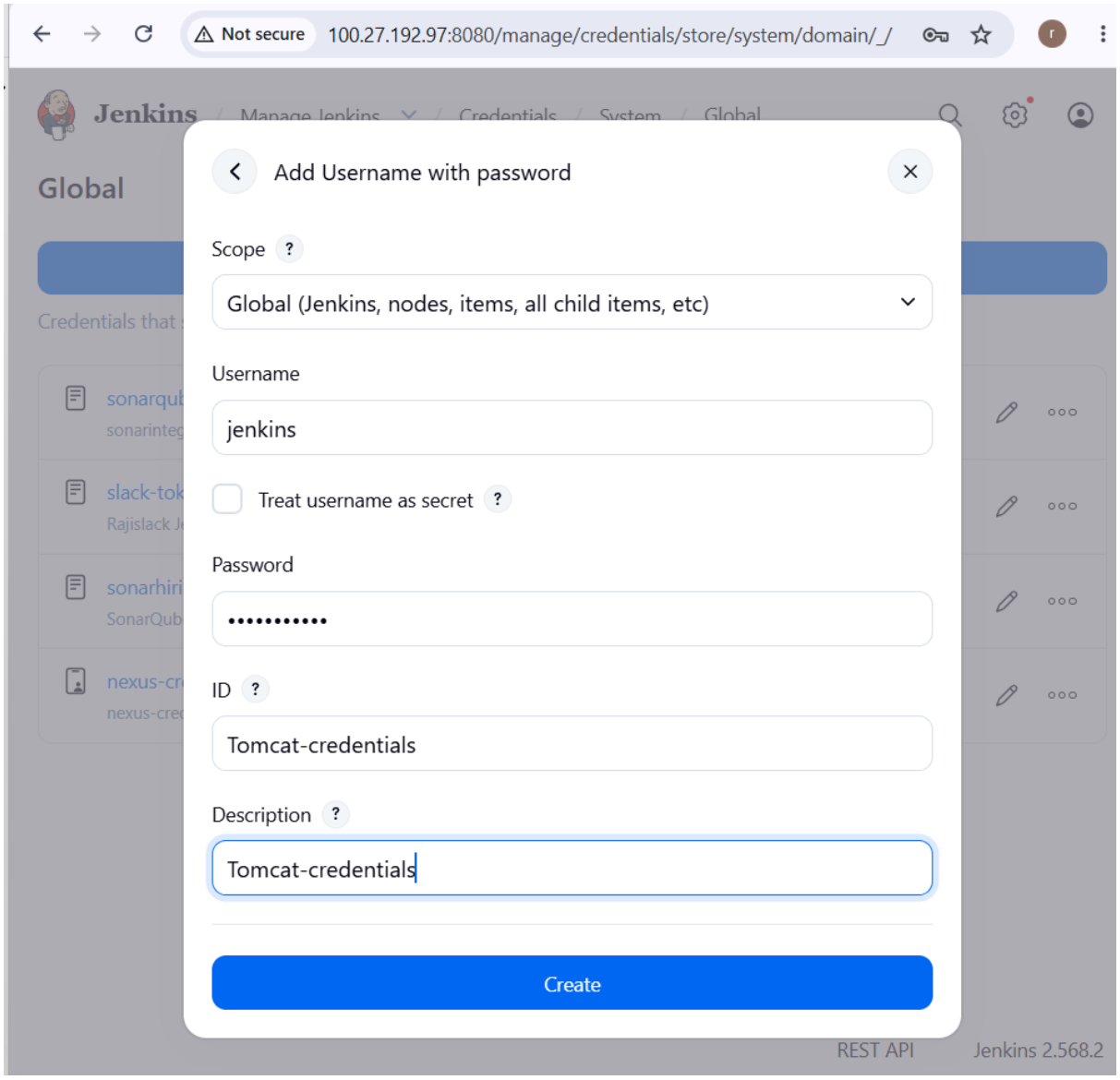
Kind: Username with password

Username: jenkins

Password: YOUR_TOMCAT_PASSWORD

ID: tomcat-credentials

Save it.



Part 6 — Configure Slack

Step 17 — Configure your existing Slack workspace

Use the workspace you recovered:

Workspace: rajislack

Channel: #jenkins-alerts-notification

In Jenkins, configure the Slack notification plugin under:

Manage Jenkins → System

Configure the Slack workspace/token credential.

Your pipeline will use:

```
slackSend(
   channel: '#jenkins-alerts-notification'
)
```

You can test Slack before running the entire pipeline.

Part 7 — Create Jenkins Pipeline

Step 18 — Create the Pipeline job

Go to:

Jenkins Dashboard → New Item

Enter:

sabear-simplecustomerapp-feature-1.1

Select:

Pipeline

Click:

OK

Then scroll to:

Pipeline → Definition

Select:

Pipeline script

We are intentionally using a **Declarative Pipeline**, not a Scripted Pipeline. Jenkins requires a Declarative Pipeline to have the pipeline {} top-level structure.

Step 10 — Check Nexus after successful deployment

After Jenkins reports:

BUILD SUCCESS

open:

Nexus → Repositories → sabear_simplecutomerapp

You should see something similar to:

```
com/
 └── javatpoint/
     └── SimpleCustomerApp/
         └── 8-SNAPSHOT/
             ├── SimpleCustomerApp-8-...war
             ├── SimpleCustomerApp-8-...pom
             └── maven-metadata.xml
```

That confirms your **Nexus Artifactory stage is working**.

Pipeline in Declarative

```
pipeline {
   agent any

   tools {
      maven 'Maven'
   }

   environment {
   NEXUS_URL = 'http://172.31.30.117:8081/repository/sabear_simplecutomerapp/'
   TOMCAT_URL = 'http://172.31.30.131:8080'
```

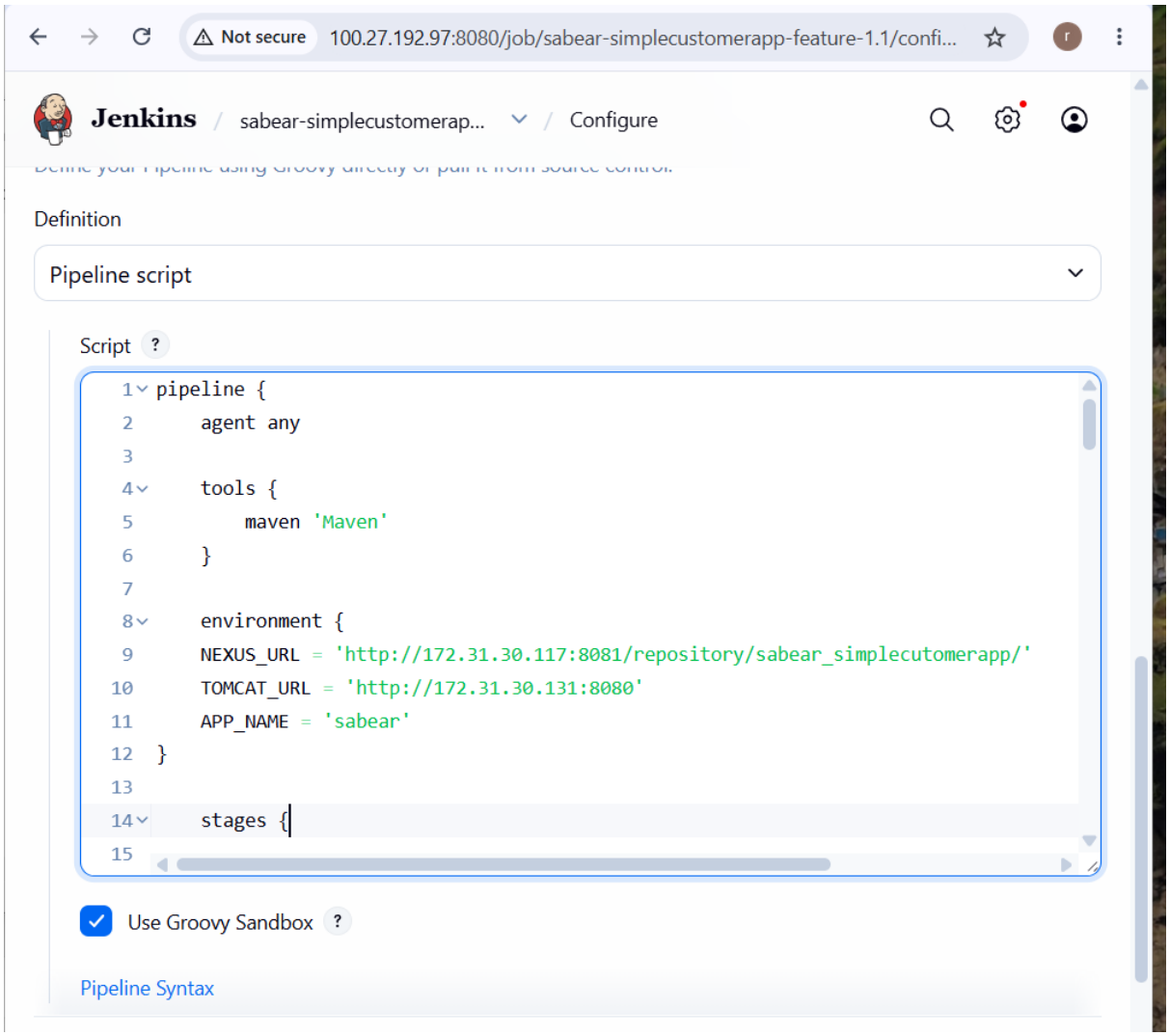
```
    APP_NAME = 'sabear'
}

    stages {

        stage('Git Clone') {
            steps {
                git(
                    branch: 'feature-1.1',
                    url: 'https://github.com/rajkumari-hub/sabear_simplecutomerapp.git'
                )

                sh '''
                    echo "===== Git Information ====="
                    git branch
                    git log -1 --oneline
                '''
            }
        }

        stage('SonarQube Integration') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
                mvn clean verify org.sonarsource.scanner.maven:sonar-maven-
plugin:sonar \
```

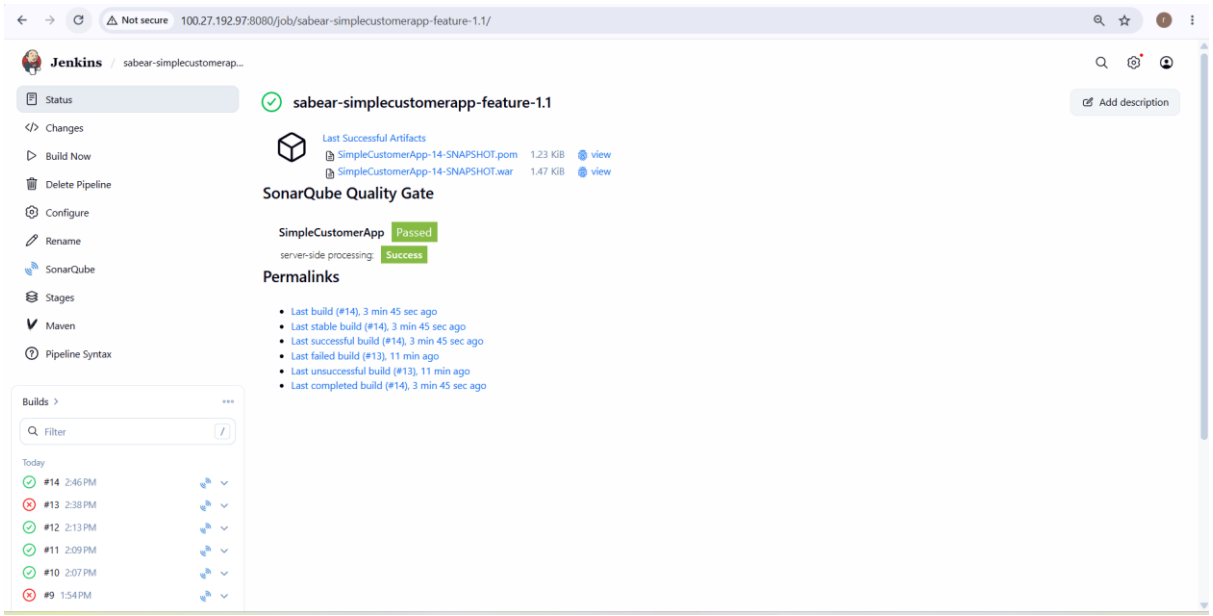
```
            -DBUILD_NUMBER=${BUILD_NUMBER}
        '''
    }
  }
}


    stage('Maven Compilation') {
      steps {
        sh '''
          mvn clean package -DskipTests
        '''
      }
    }

    stage('Nexus Artifactory') {
      steps {
        withMaven(
          mavenSettingsConfig: 'nexus-settings'
        ) {
          sh '''
            mvn deploy -DskipTests
          '''
        }
      }
    }
```

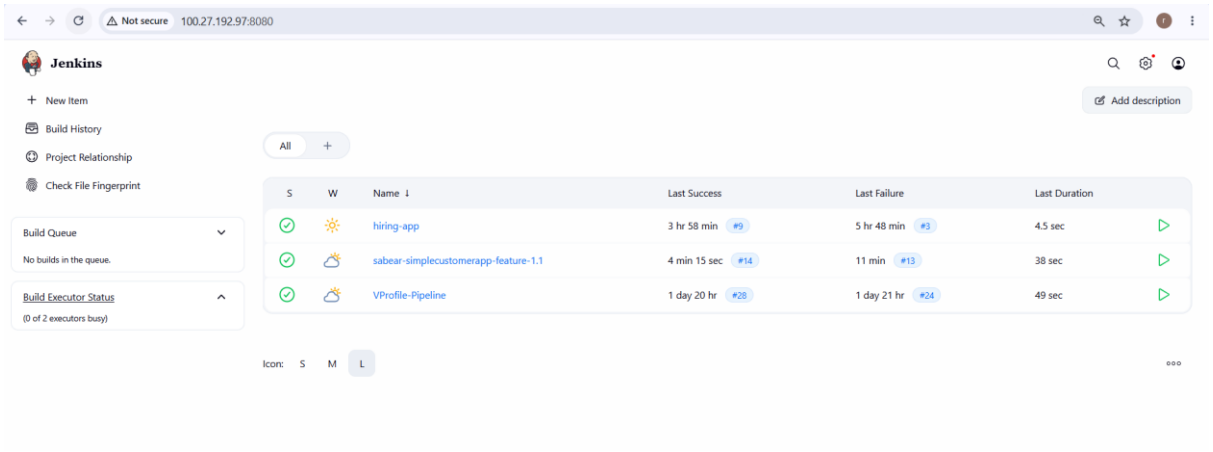
```
        stage('Slack Notification') {
            steps {
                slackSend(
                    channel: '#jenkins-alerts-notification',
                    color: 'good',
                    message: "Nexus deployment completed: ${JOB_NAME}
#${BUILD_NUMBER}"
                )
            }
        }

        stage('Deploy On Tomcat') {
            steps {

                withCredentials([
                    usernamePassword(
                        credentialsId: 'tomcat-credentials',
                        usernameVariable: 'TOMCAT_USER',
                        passwordVariable: 'TOMCAT_PASSWORD'
                    )
                ]) {

                    sh '''
                        echo "===== WAR FILE ====="
                        ls -lh target/*.war
```

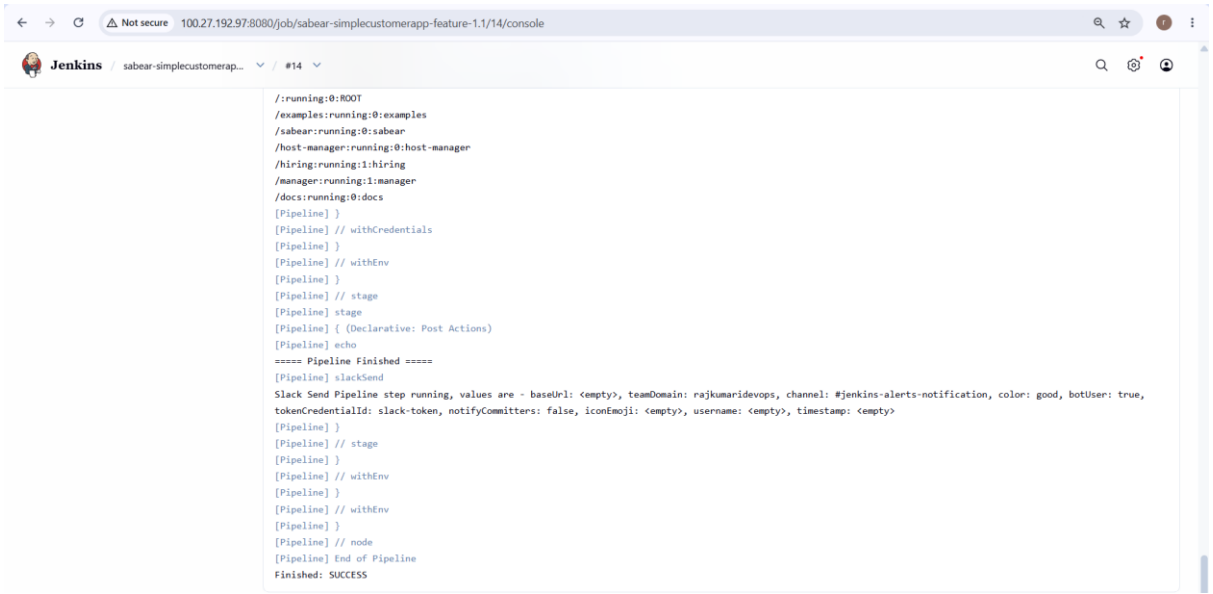
```
                echo "===== Deploying to Tomcat ====="

                curl -v \
                -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
                -T target/*.war \
"$TOMCAT_URL/manager/text/deploy?path=/$APP_NAME&update=true"
            '''
          }
        }
      }
    }

    post {

      success {
        slackSend(
          channel: '#jenkins-alerts-notification',
          color: 'good',
          message: "SUCCESS: ${JOB_NAME} #${BUILD_NUMBER} - Application deployed successfully to Tomcat."
        )
      }

      failure {
        slackSend(
          channel: '#jenkins-alerts-notification',
```

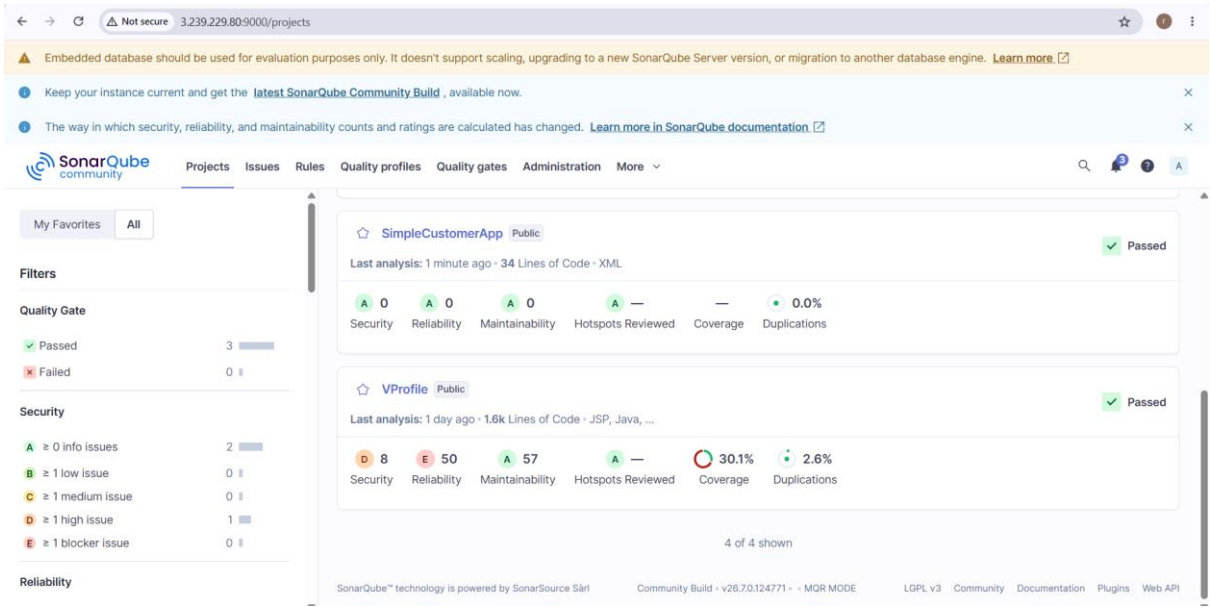
```
            color: 'danger',
            message: "FAILED: ${JOB_NAME} #${BUILD_NUMBER} - Pipeline failed. Check Jenkins console."
        )
    }

    always {
        echo "===== Pipeline Finished ====="
    }
  }
}
```

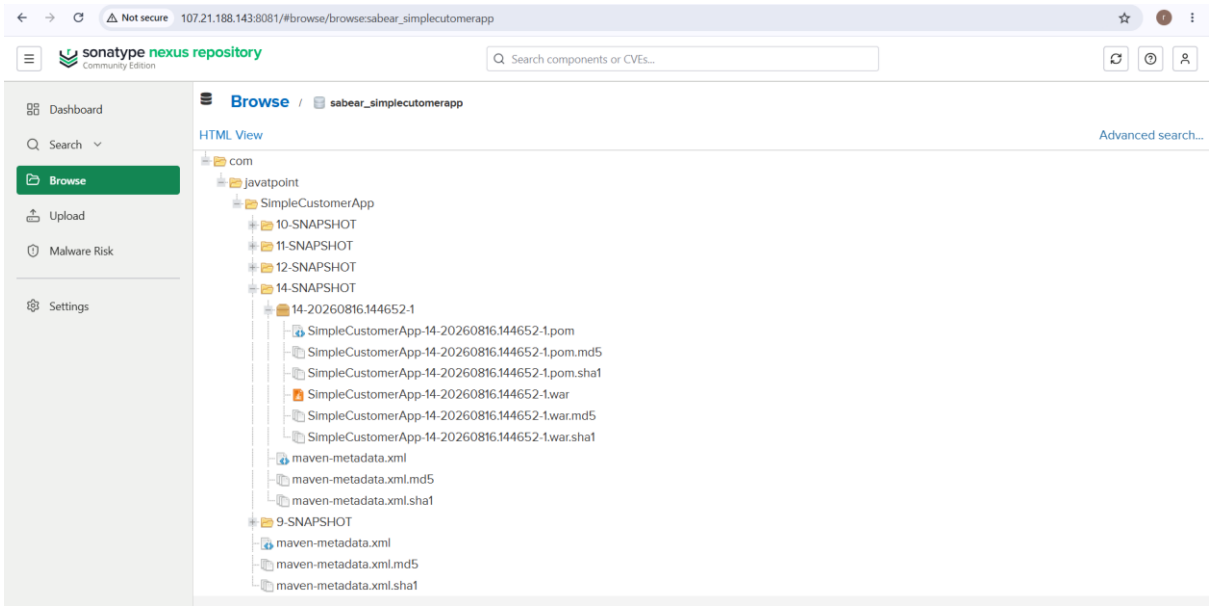


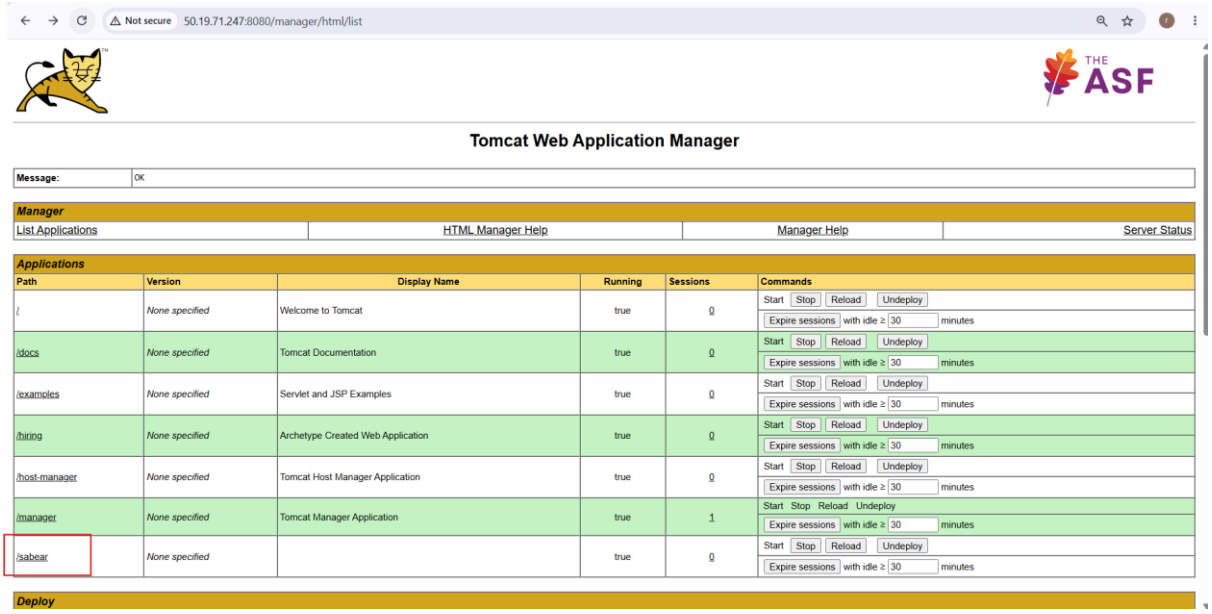

Not secure 100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1/
Jenkins sabear-simplecustomerap...
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
SonarQube
Stages
Maven
Pipeline Syntax
Builds
Filter
Today
#14 2:46 PM
#13 2:38 PM
#12 2:13 PM
#11 2:09 PM
#10 2:07 PM
#9 1:54 PM
sabear-simplecustomerapp-feature-1.1
Add description
Last Successful Artifacts
SimpleCustomerApp-14-SNAPSHOT.pom 1.23 KiB view
SimpleCustomerApp-14-SNAPSHOT.war 1.47 KiB view
SonarQube Quality Gate
SimpleCustomerApp Passed
server-side processing: Success
Permalinks
Last build (#14), 3 min 45 sec ago
Last stable build (#14), 3 min 45 sec ago
Last successful build (#14), 3 min 45 sec ago
Last failed build (#13), 11 min ago
Last unsuccessful build (#13), 11 min ago
Last completed build (#14), 3 min 45 sec ago

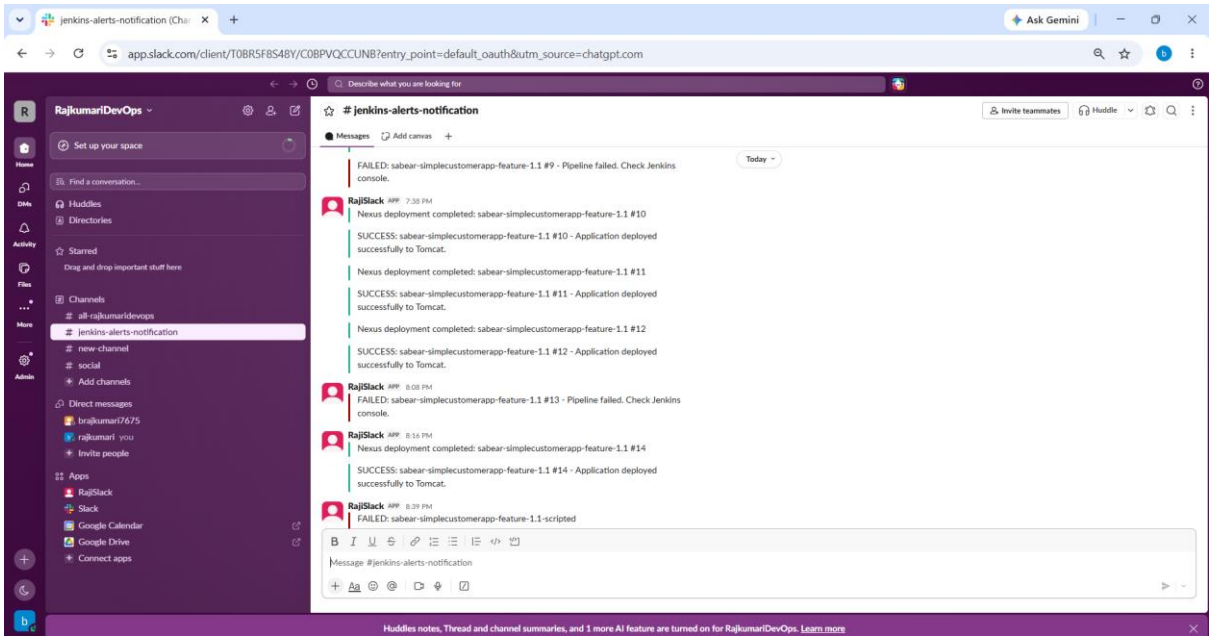

Not secure 100.27.192.97:8080
Jenkins
New Item
Build History
Project Relationship
Check File Fingerprint
Build Queue
No builds in the queue.
Build Executor Status
(0 of 2 executors busy)
Add description
All
S W Name ↓ Last Success Last Failure Last Duration
hiring-app 3 hr 58 min #9 5 hr 48 min #3 4.5 sec
sabear-simplecustomerapp-feature-1.1 4 min 15 sec #14 11 min #13 38 sec
VProfile-Pipeline 1 day 20 hr #28 1 day 21 hr #24 49 sec
Icon: S M L


Not secure 100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1/14/console
Jenkins sabear-simplecustomerap... #14
/:running:0:ROOT
/examples:running:0:examples
/sabear:running:0:sabear
/host-manager:running:0:host-manager
/hiring:running:1:hiring
/manager:running:1:manager
/docs:running:0:docs
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
===== Pipeline Finished =====
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: rajkumaridevops, channel: #jenkins-alerts-notification, color: good, botUser: true, tokenCredentialId: slack-token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure 3.239.229.80:9000/projects
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
Keep your instance current and get the latest SonarQube Community Build, available now.
The way in which security, reliability, and maintainability counts and ratings are calculated has changed. Learn more in SonarQube documentation
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
My Favorites All
Filters
Quality Gate
Passed 3
Failed 0
Security
A ≥ 0 info issues 2
B ≥ 1 low issue 0
C ≥ 1 medium issue 0
D ≥ 1 high issue 1
E ≥ 1 blocker issue 0
Reliability
SimpleCustomerApp Public
Passed
Last analysis: 1 minute ago • 34 Lines of Code • XML
A 0 Security
A 0 Reliability
A 0 Maintainability
A — Hotspots Reviewed
— Coverage
0.0% Duplications
VProfile Public
Passed
Last analysis: 1 day ago • 1.6k Lines of Code • JSP, Java, ...
D 8 Security
E 50 Reliability
A 57 Maintainability
A — Hotspots Reviewed
30.1% Coverage
2.6% Duplications
4 of 4 shown
SonarQube™ technology is powered by SonarSource Sàrl
Community Build • v26.7.0.124771 • • MQR MODE
LGPL v3 Community Documentation Plugins Web API


Not secure 107.21.188.143:8081/#browse/browse:sabear_simplecutomerapp
sonatype nexus repository
Community Edition
Search components or CVEs...
Dashboard
Search
Browse
Upload
Malware Risk
Settings
Browse / sabear_simplecutomerapp
HTML View
Advanced search...
com
javatpoint
SimpleCustomerApp
10-SNAPSHOT
11-SNAPSHOT
12-SNAPSHOT
14-SNAPSHOT
14-20260816.144652-1
SimpleCustomerApp-14-20260816.144652-1.pom
SimpleCustomerApp-14-20260816.144652-1.pom.md5
SimpleCustomerApp-14-20260816.144652-1.pom.sha1
SimpleCustomerApp-14-20260816.144652-1.war
SimpleCustomerApp-14-20260816.144652-1.war.md5
SimpleCustomerApp-14-20260816.144652-1.war.sha1
maven-metadata.xml
maven-metadata.xml.md5
maven-metadata.xml.sha1
9-SNAPSHOT
maven-metadata.xml
maven-metadata.xml.md5
maven-metadata.xml.sha1





3) Setup a Jenkins CICD pipeline using Scripted pipeline using feature-1.1 branch.

Source Code:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages:

- # Git Clone
- # SonarQube Integration
- # Maven Compilation
- # Nexus Artifactory
- # Slack Notification
- # Deploy On tomcat

1. **Create the Jenkins Pipeline job**

Go to:

Jenkins → New Item

Enter:

sabear-simplecustomerapp-feature-1.1-scripted

Select:

Pipeline

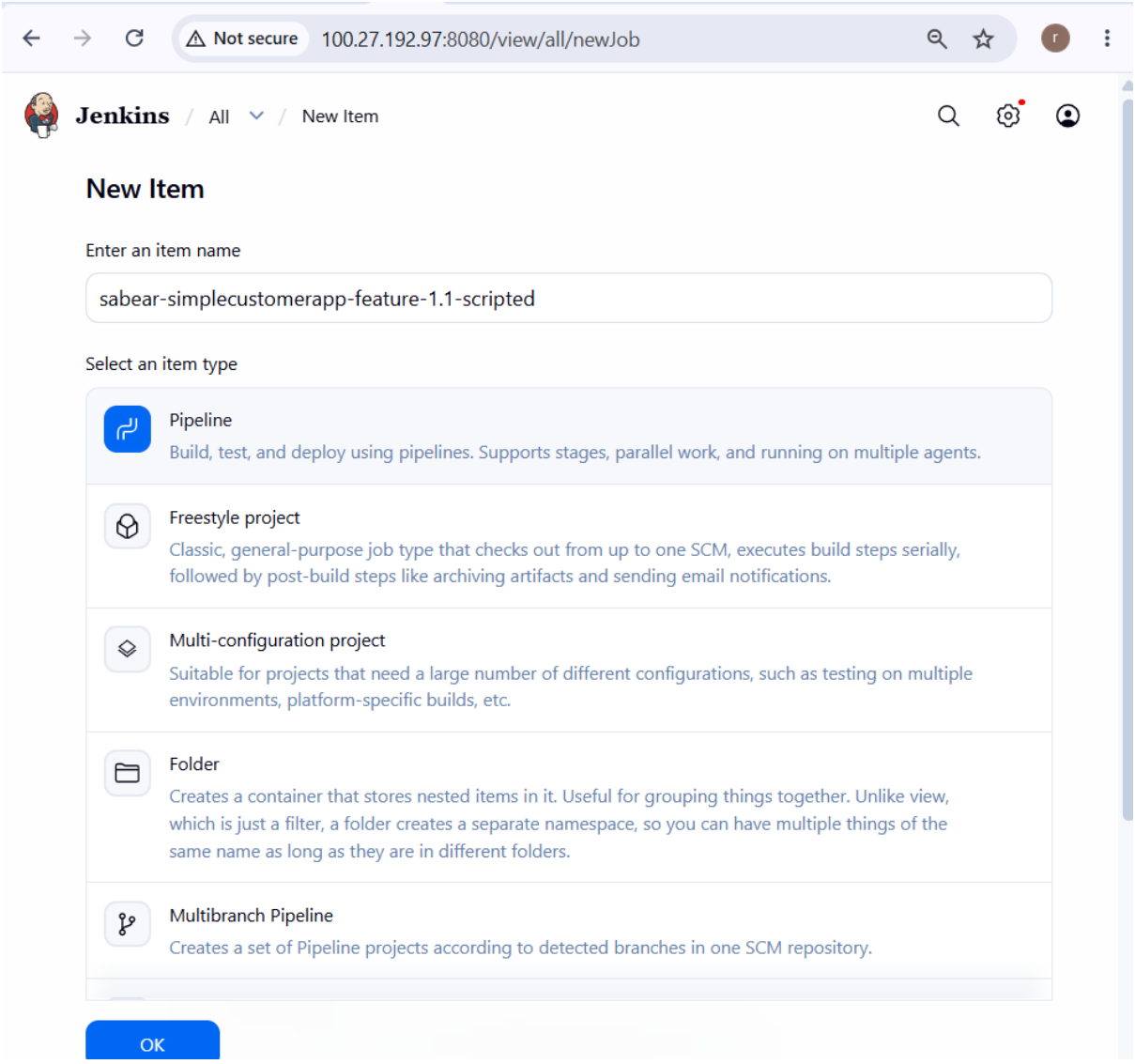
Click **OK**.

Scroll to **Pipeline**.

Set:

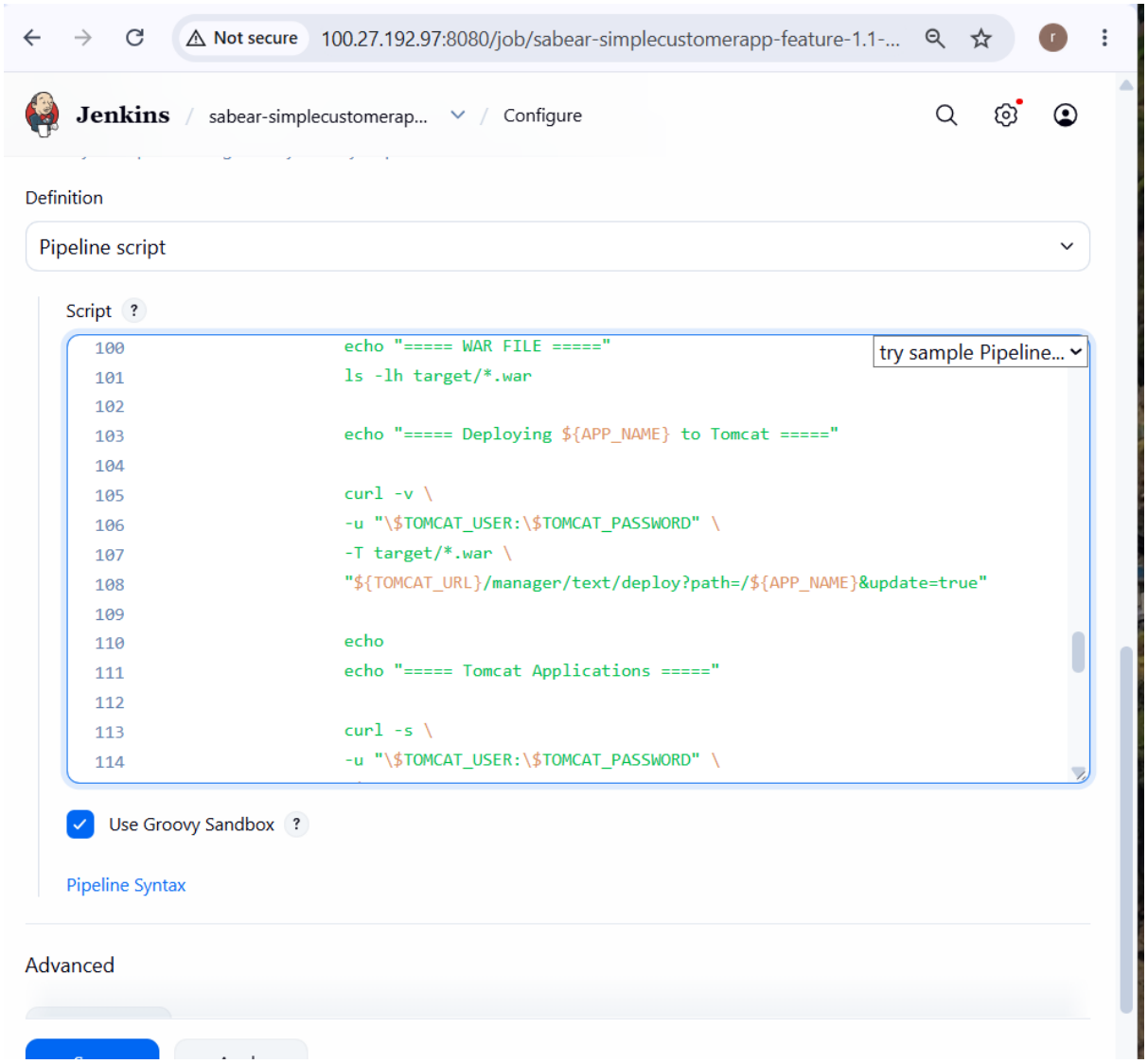
Definition: Pipeline script

Then paste the following script.



2. Complete Scripted Pipeline

Use this version:



3. Important: use your actual Git repository

Your assignment gives:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

But your current work has been pushed to:

https://github.com/rajkumari-hub/sabear_simplecutomerapp.git

Therefore, because your Jenkins pipeline needs to use the repository containing your corrected code, I used:

git branch: 'feature-1.1',

url: 'https://github.com/rajkumari-hub/sabear_simplecutomerapp.git'

If your assignment specifically requires the original betawins repository, change that URL back.

4. Check your Jenkins credentials

You previously had this exact error:

ERROR: Could not find credentials entry with ID 'tomcat-credentials'

So **before running this job**, make sure this credential exists.

Go to:

Jenkins → Manage Jenkins → Credentials → System → Global credentials

Create:

Kind: Username with password

Username: jenkins

Password: Jenkins@123

ID: tomcat-credentials

Description: Tomcat deployment credentials

Your Tomcat server already confirmed these credentials work:

curl -u 'jenkins:Jenkins@123' \

http://172.31.30.131:8080/manager/text/list

and returned:

/sabear:running:0:sabear

So this credential configuration is correct.

5. Check Maven/Nexus configuration

Your pom.xml contains:

```
<distributionManagement>
   <snapshotRepository>
      <id>nexus</id>
      <url>http://172.31.30.117:8081/repository/sabear_simplecutomerapp/</url>
   </snapshotRepository>
</distributionManagement>
```

Therefore Jenkins must have Nexus authentication configured for:

server ID = nexus

inside Maven settings.xml.

The Pipeline Maven Integration plugin's withMaven step configures Maven for Pipeline execution and can use Maven settings configured in Jenkins.

If your Nexus upload is already working from the previous job, **don't change it**.

6. Your WAR must contain the application

This is particularly important because you previously had the 404 problem.

Your current correct WAR showed:

index.html

Welcome.jsp

Customer.jsp

WEB-INF/web.xml

css/

images/

So after Maven compilation, Jenkins should show:

===== Generated WAR =====

-rw-r--r-- ... target/sabear.war

You can also verify from Jenkins:

unzip -l target/*.war | head -50

You should see things such as:

index.html

Welcome.jsp

Customer.jsp

WEB-INF/web.xml

images/

css/

7. Tomcat deployment path

Your deployment is:

WAR:

target/sabear.war

Tomcat:

172.31.30.131:8080

Context:

sabear

Therefore the deployment command becomes:

```
curl -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
-T target/sabear.war \
"http://172.31.30.131:8080/manager/text/deploy?path=/sabear&update=true"
```

The resulting application URL is:

http://50.19.71.247:8080/sabear/

The important distinction is:

Tomcat server private IP

↓

172.31.30.131:8080

Public browser IP

↓

50.19.71.247:8080

Application context

↓

/sabear

8. One correction to your stage order

Your assignment explicitly says:

Git Clone

SonarQube Integration

Maven Compilation

Nexus Artifactory

Slack Notification

Deploy On tomcat

The script above follows **exactly that order**.

However, in a production CI/CD pipeline, I would normally send the success Slack notification **after deployment**, because then Slack means the complete deployment succeeded.

But for your assignment, keep the order exactly as given.

---

9. What you should see in Jenkins

When you click **Build Now**, the Pipeline should show:

Git Clone

✓

↓

SonarQube Integration

✓

↓

Maven Compilation

✓

↓

Nexus Artifactory

✓

↓

Slack Notification

✓

↓

Deploy On Tomcat

✓

At the end:

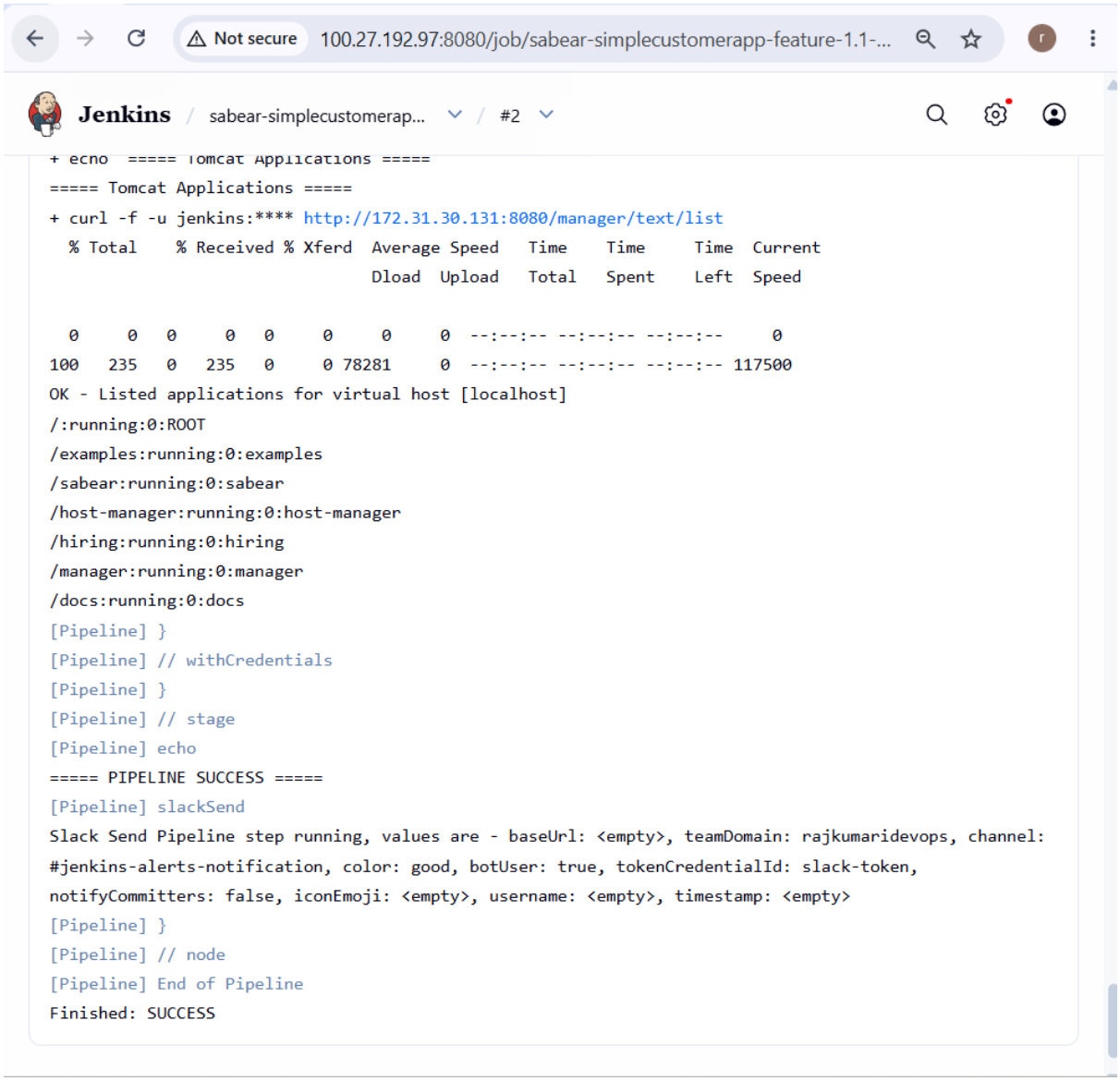
Finished: SUCCESS

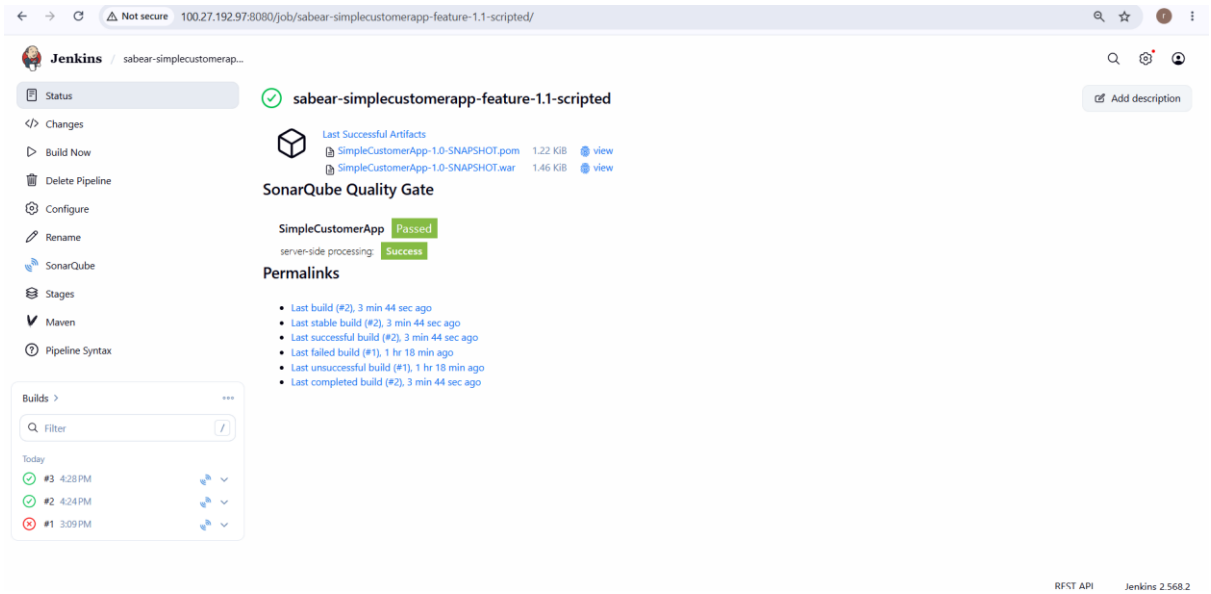
And Tomcat should show:

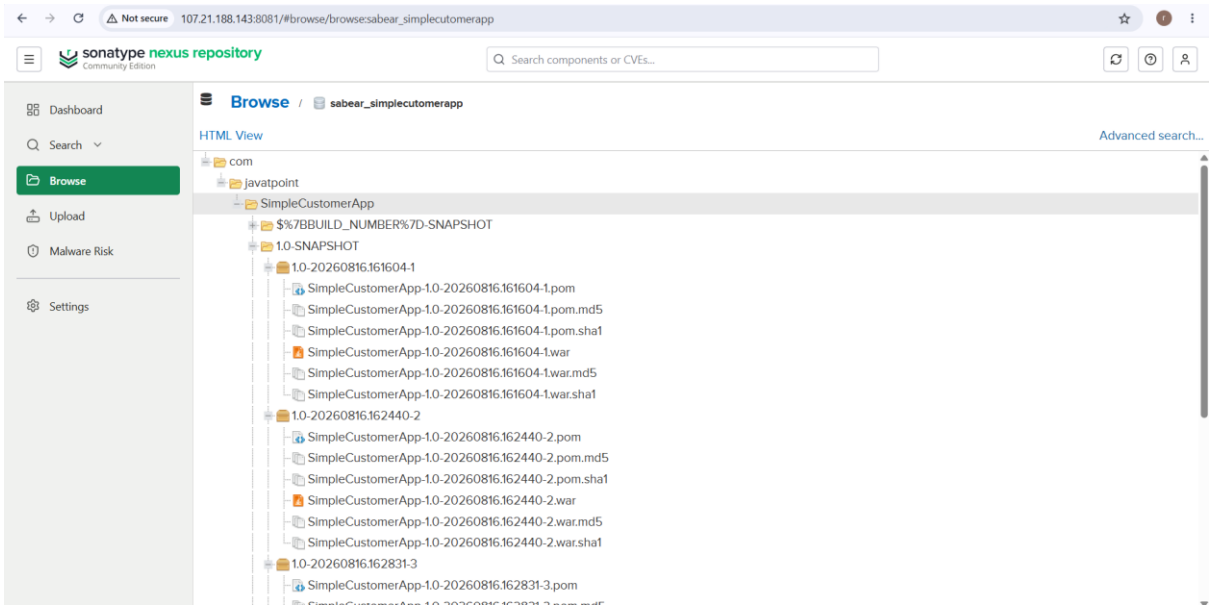
/sabear:running:0:sabear

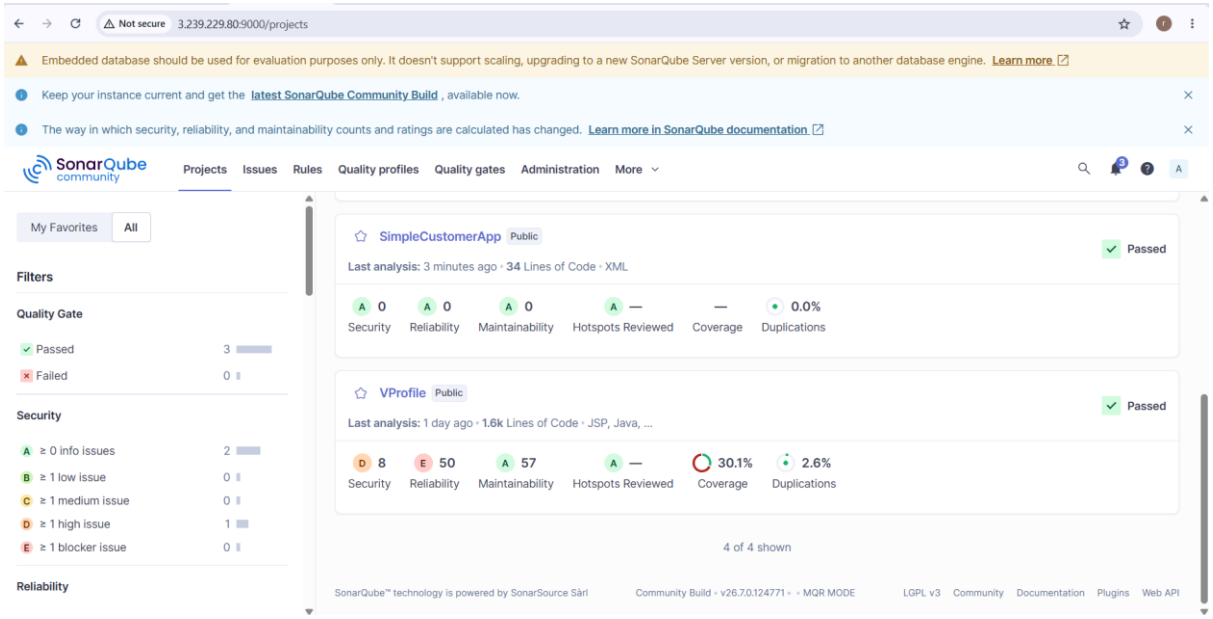
Then open:

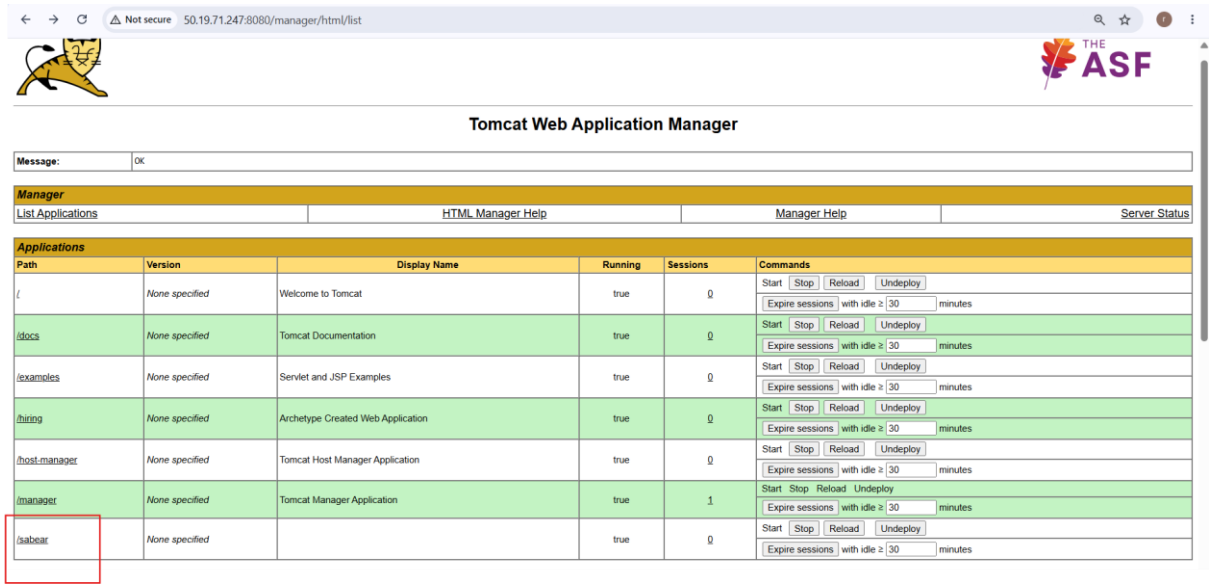
http://50.19.71.247:8080/sabear/

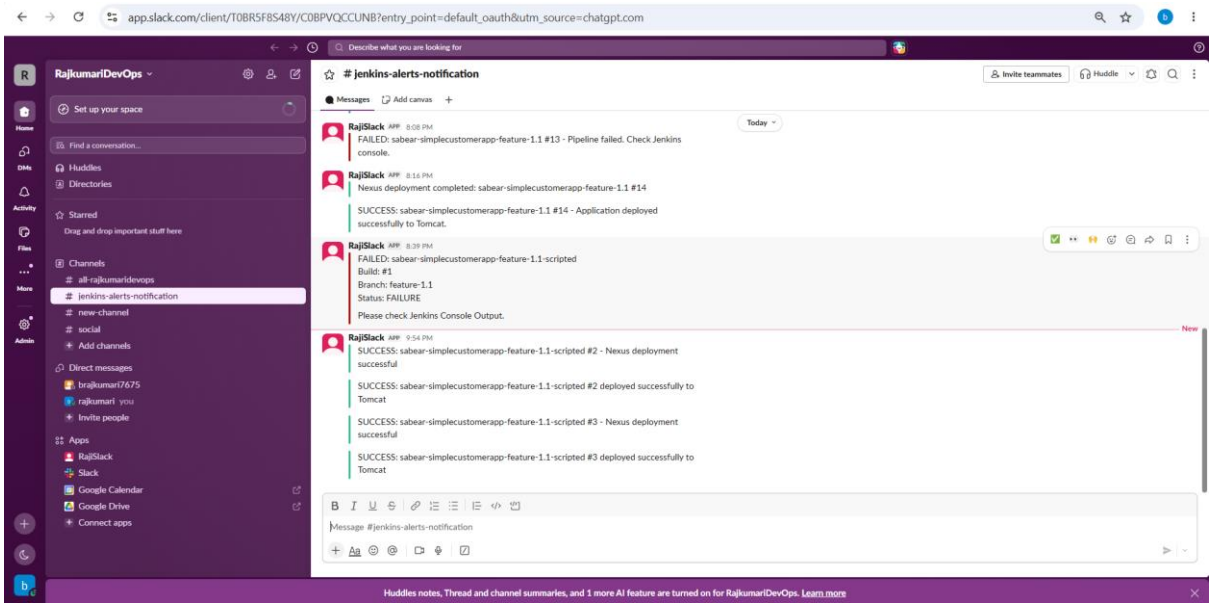

Not secure
100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1-...
Jenkins
sabear-simplecustomerap...
#2
+ echo ===== Tomcat Applications =====
===== Tomcat Applications =====
+ curl -f -u jenkins:**** http://172.31.30.131:8080/manager/text/list
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0
100 235 0 235 0 0 78281 0 --:--:-- --:--:-- --:--:-- 117500
OK - Listed applications for virtual host [localhost]
/:running:0:ROOT
/examples:running:0:examples
/sabear:running:0:sabear
/host-manager:running:0:host-manager
/hiring:running:0:hiring
/manager:running:0:manager
/docs:running:0:docs
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] echo
===== PIPELINE SUCCESS =====
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: rajkumaridevops, channel:
#jenkins-alerts-notification, color: good, botUser: true, tokenCredentialId: slack-token,
notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS


Not secure
100.27.192.97:8080/job/sabear-simplecustomerapp-feature-1.1-scripted/
Jenkins
sabear-simplecustomerap...
Status
Changes
Build Now
Delete Pipeline
Configure
Rename
SonarQube
Stages
Maven
Pipeline Syntax
Builds
Filter
Today
#3 4:28 PM
#2 4:24 PM
#1 3:09 PM
sabear-simplecustomerapp-feature-1.1-scripted
Add description
Last Successful Artifacts
SimpleCustomerApp-1.0-SNAPSHOT.pom 1.22 KiB view
SimpleCustomerApp-1.0-SNAPSHOT.war 1.46 KiB view
SonarQube Quality Gate
SimpleCustomerApp Passed
server-side processing: Success
Permalinks
Last build (#2), 3 min 44 sec ago
Last stable build (#2), 3 min 44 sec ago
Last successful build (#2), 3 min 44 sec ago
Last failed build (#1), 1 hr 18 min ago
Last unsuccessful build (#1), 1 hr 18 min ago
Last completed build (#2), 3 min 44 sec ago
REST API
Jenkins 2.568.2


Not secure 107.21.188.143:8081/#browse/browse:sabear_simplecutomerapp
sonatype nexus repository
Community Edition
Search components or CVEs...
Dashboard
Search
Browse
Upload
Malware Risk
Settings
Browse / sabear_simplecutomerapp
HTML View
Advanced search...
com
javatpoint
SimpleCustomerApp
$%7BBUILD_NUMBER%7D-SNAPSHOT
1.0-SNAPSHOT
1.0-20260816.161604-1
SimpleCustomerApp-1.0-20260816.161604-1.pom
SimpleCustomerApp-1.0-20260816.161604-1.pom.md5
SimpleCustomerApp-1.0-20260816.161604-1.pom.sha1
SimpleCustomerApp-1.0-20260816.161604-1.war
SimpleCustomerApp-1.0-20260816.161604-1.war.md5
SimpleCustomerApp-1.0-20260816.161604-1.war.sha1
1.0-20260816.162440-2
SimpleCustomerApp-1.0-20260816.162440-2.pom
SimpleCustomerApp-1.0-20260816.162440-2.pom.md5
SimpleCustomerApp-1.0-20260816.162440-2.pom.sha1
SimpleCustomerApp-1.0-20260816.162440-2.war
SimpleCustomerApp-1.0-20260816.162440-2.war.md5
SimpleCustomerApp-1.0-20260816.162440-2.war.sha1
1.0-20260816.162831-3
SimpleCustomerApp-1.0-20260816.162831-3.pom


Not secure 3.239.229.80:9000/projects
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
Keep your instance current and get the latest SonarQube Community Build, available now.
The way in which security, reliability, and maintainability counts and ratings are calculated has changed. Learn more in SonarQube documentation
SonarQube community
Projects Issues Rules Quality profiles Quality gates Administration More
My Favorites All
Filters
Quality Gate
Passed 3
Failed 0
Security
≥ 0 info issues 2
≥ 1 low issue 0
≥ 1 medium issue 0
≥ 1 high issue 1
≥ 1 blocker issue 0
Reliability
SimpleCustomerApp Public
Passed
Last analysis: 3 minutes ago · 34 Lines of Code · XML
0 Security
0 Reliability
0 Maintainability
— Hotspots Reviewed
— Coverage
0.0% Duplications
VProfile Public
Passed
Last analysis: 1 day ago · 1.6k Lines of Code · JSP, Java, ...
8 Security
50 Reliability
57 Maintainability
— Hotspots Reviewed
30.1% Coverage
2.6% Duplications
4 of 4 shown
SonarQube™ technology is powered by SonarSource Sàrl
Community Build · v26.7.0.124771 · · MQR MODE
LGPL v3 Community Documentation Plugins Web API


Not secure 50.19.71.247:8080/manager/html/list
THE ASF
Tomcat Web Application Manager
Message: OK
Manager
List Applications HTML Manager Help Manager Help Server Status
Applications
Path Version Display Name Running Sessions Commands
/ None specified Welcome to Tomcat true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs None specified Tomcat Documentation true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples None specified Servlet and JSP Examples true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiring None specified Archetype Created Web Application true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager None specified Tomcat Host Manager Application true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager None specified Tomcat Manager Application true 1 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/sabear None specified true 0 Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes



Step 8 — Run the pipeline

Click:

Build Now

The expected result is:

Git Clone ✅

SonarQube Integration ✅

Maven Compilation ✅

Nexus Artifactory ✅

Slack Notification ✅

Deploy On Tomcat ✅

At the end you should see:

===== PIPELINE SUCCESS =====

Finished: SUCCESS

4) Write sample skeleton of pipelines.

5) Create a parameterized job in Jenkins.

Source Code:

https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git

4) Write sample skeleton of pipelines

For this assignment, you can create **sample skeletons** without connecting them to a real application.

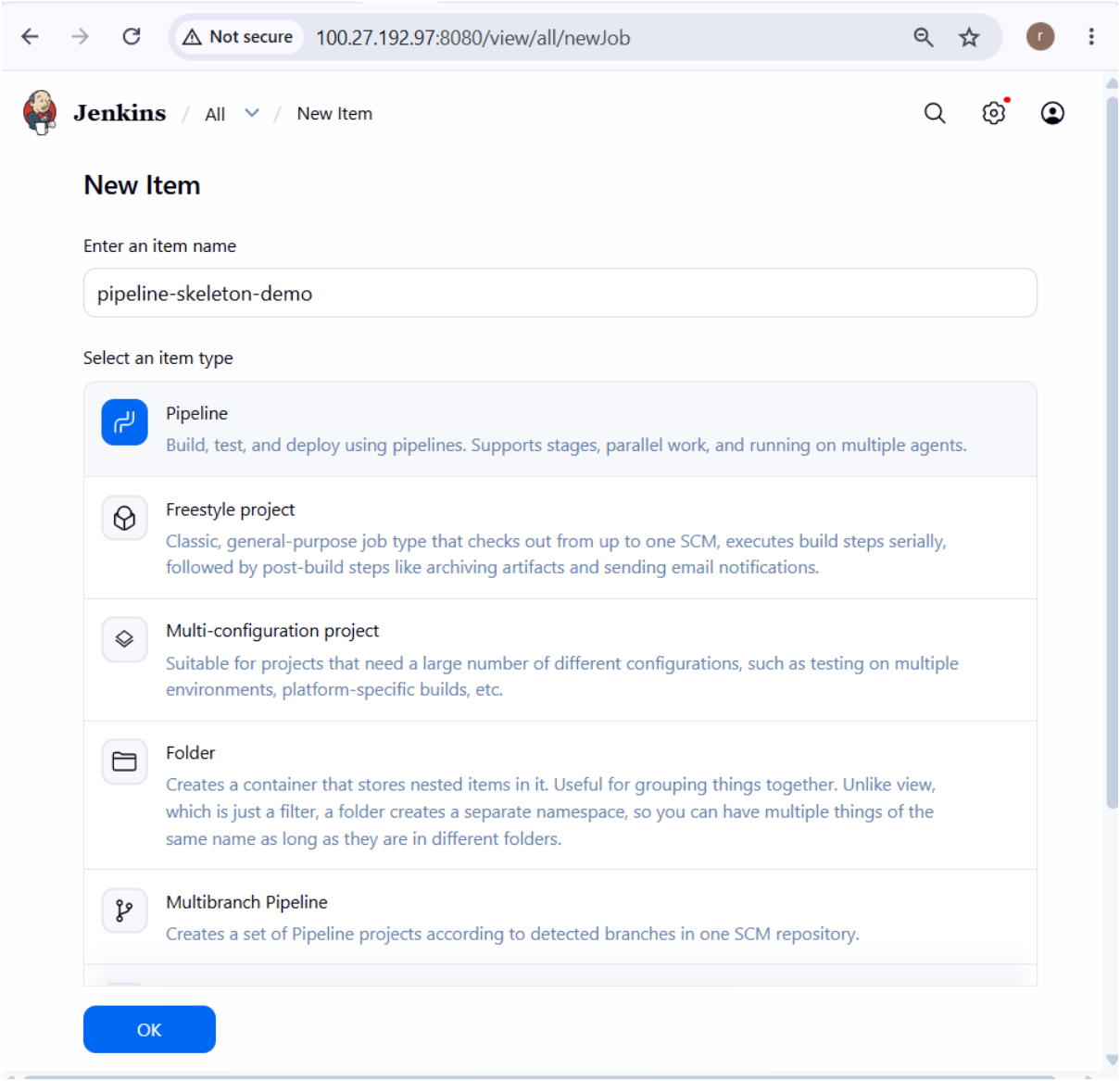
Create a Jenkins **Pipeline** job called:

pipeline-skeleton-demo

Go to:

Jenkins Dashboard → New Item → pipeline-skeleton-demo → Pipeline → OK

In the **Pipeline → Script** section, you can demonstrate the following.



A. Declarative Pipeline skeleton

```
pipeline {

    agent any

    stages {

        stage('Git Clone') {
```

```
        steps {
            echo 'Cloning source code...'
        }
    }

    stage('Build') {
        steps {
            echo 'Building application...'
        }
    }

    stage('Test') {
        steps {
            echo 'Running tests...'
        }
    }

    stage('Deploy') {
        steps {
            echo 'Deploying application...'
        }
    }
}

post {
    success {
```

```
            echo 'Pipeline completed successfully'
        }

        failure {
            echo 'Pipeline failed'
        }
    }
}
```

This demonstrates the standard Declarative structure:

```
pipeline
  ├── agent
  ├── stages
  │     ├── stage
  │     ├── stage
  │     ├── stage
  │     └──── stage
  └──── post
```

Jenkins documents this as the basic Declarative Pipeline structure.

B. Scripted Pipeline skeleton

Since your previous assignment already used Scripted Pipeline, this is important for your interview/practical assignment.

```
node {

    stage('Git Clone') {
```

```
        echo 'Cloning source code...'
    }

    stage('Build') {
        echo 'Building application...'
    }

    stage('Test') {
        echo 'Running tests...'
    }

    stage('Deploy') {
        echo 'Deploying application...'
    }

}
```

The basic difference is:

Declarative:

```
pipeline {
    agent any
    stages {
        ...
    }
}
```

Scripted:

```
node {
```

```
    stage('Build') {
        ...
    }
}
```

Jenkins describes node as the fundamental Scripted Pipeline construct, while pipeline is specific to Declarative syntax.

C. Simple Maven Pipeline skeleton

You can also show a more realistic skeleton:

```
node {

    stage('Git Clone') {
        git branch: 'master',
            url: 'https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git'
    }

    stage('Maven Build') {
        sh 'mvn clean package'
    }

    stage('Test') {
        sh 'mvn test'
    }

    stage('Deploy') {
        echo 'Deploying application...'
```

```
    }
}
```

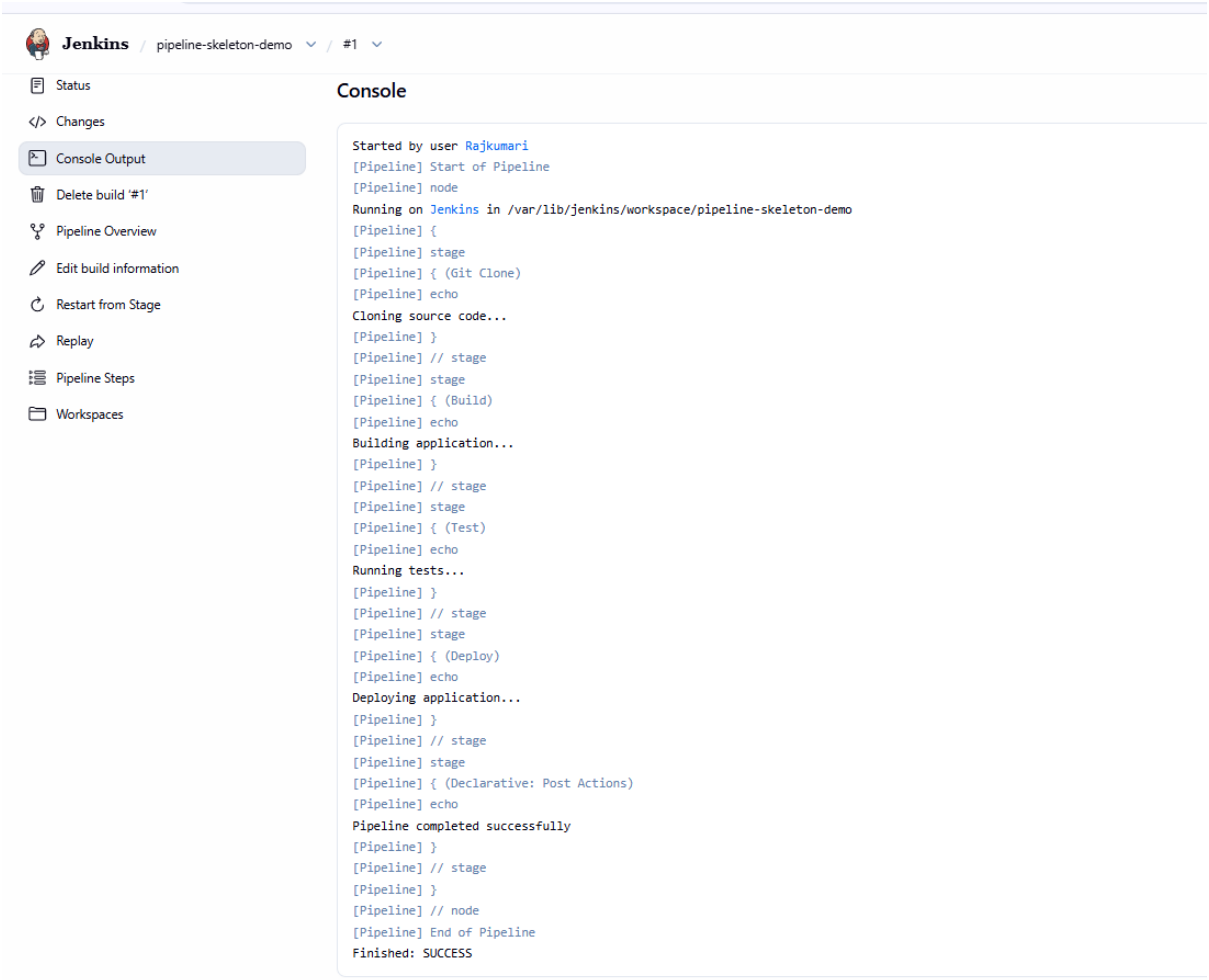
For your assignment, this is a good **sample skeleton** because it shows how a real Java/Maven application would flow through Jenkins.

1. Declarative Pipeline Skeleton

Create a Jenkins **Pipeline** job and use:

groovy

```groovy
pipeline {

    agent any

    stages {

        stage('Git Clone') {
            steps {
                echo 'Cloning source code...'
            }
        }

        stage('Build') {
            steps {
                echo 'Building application...'
            }
        }

        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application...'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully'
        }

        failure {
            echo 'Pipeline failed'
        }
    }
}
```

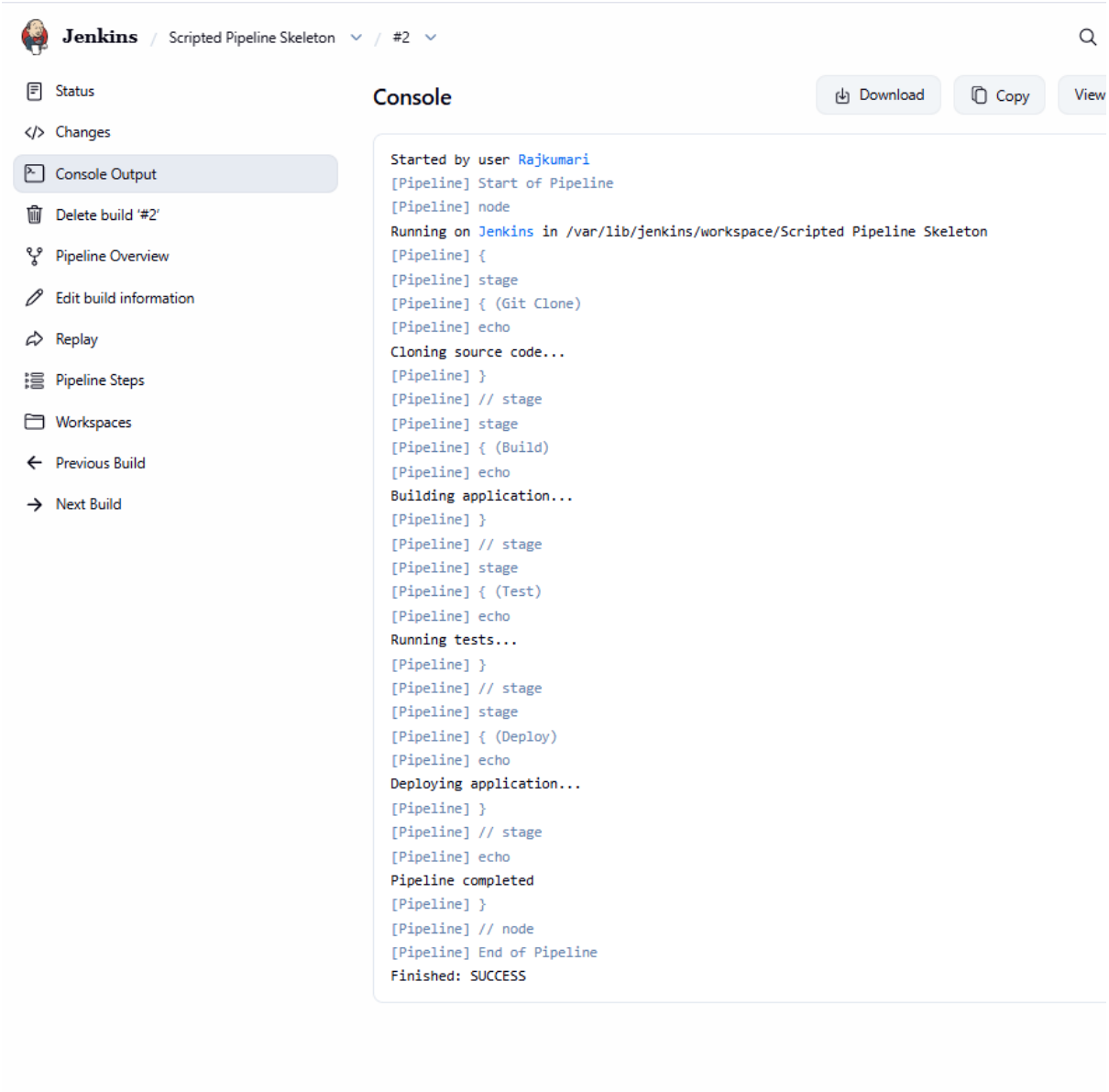


2. Scripted Pipeline Skeleton

This is the style you used for your **feature-1.1** assignment.

groovy

```groovy
node {

    stage('Git Clone') {
        echo 'Cloning source code...'
    }

    stage('Build') {
        echo 'Building application...'
    }

    stage('Test') {
        echo 'Running tests...'
    }

    stage('Deploy') {
        echo 'Deploying application...'
    }

    echo 'Pipeline completed'
}
```

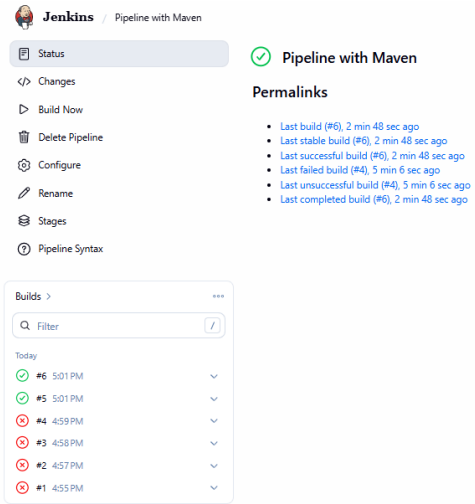

Jenkins / Scripted Pipeline Skeleton / #2
Status
Changes
Console Output
Delete build '#2'
Pipeline Overview
Edit build information
Replay
Pipeline Steps
Workspaces
Previous Build
Next Build
Console
Download
Copy
View
Started by user Rajkumari
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Scripted Pipeline Skeleton
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] echo
Cloning source code...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Running tests...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] echo
Pipeline completed
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

3. Pipeline with Maven

You can also show a practical Maven pipeline skeleton:

groovy

```groovy
pipeline {

    agent any

    tools {
        maven 'Maven'
    }

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/example/project.git'
            }
        }

        stage('Maven Build') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application...'
            }
        }
    }
}
```



5) Create a Parameterized Jenkins Job

Now create a **separate Jenkins job** for this source code:

https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git

I recommend creating:

spring3-mvc-parameterized

Step 1 — Create the job

Go to:

Jenkins Dashboard → New Item

Enter:

spring3-mvc-parameterized

Select:

Pipeline

Click:

OK

Step 2 — Enable parameters

In the job configuration, find:

General

Enable:

☑ This project is parameterized

Then click:

Add Parameter → String Parameter

Create:

Parameter 1

Name: ENVIRONMENT

Default Value: dev

Description: Deployment environment

Add another:

Parameter 2

Name: BRANCH

Default Value: master

Description: Git branch to build

Add another:

Parameter 3

Name: MESSAGE

Default Value: Hello Jenkins

Description: Message to display during the build

Your parameters will look like:

ENVIRONMENT = dev

BRANCH = master

MESSAGE = Hello Jenkins

Jenkins parameters are supplied when triggering the build and are accessible through the params object.

Step 3 — Add the Pipeline

Scroll down to:

Pipeline → Definition

Select:

Pipeline script

Paste this **Scripted Pipeline**:

node {

```
stage('Parameters') {

    echo "===== BUILD PARAMETERS ====="

    echo "Environment : ${params.ENVIRONMENT}"
    echo "Branch      : ${params.BRANCH}"
    echo "Message     : ${params.MESSAGE}"
}


stage('Git Clone') {

    echo "===== GIT CLONE ====="

    git branch: "${params.BRANCH}",
        url: 'https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git'
}


stage('Build') {

    echo "===== MAVEN BUILD ====="

    sh 'mvn clean package -DskipTests'
}
```

```
stage('Test') {

    echo "===== MAVEN TEST ====="

    sh 'mvn test'
}



stage('Display Message') {

    echo "===== USER MESSAGE ====="

    echo "${params.MESSAGE}"
}



stage('Deployment') {

    echo "===== DEPLOYMENT ====="

    echo "Application will be deployed to ${params.ENVIRONMENT}"
}



echo "===== PIPELINE COMPLETED ====="
```

}

Save the job.

Step 4 — Build with Parameters

Now click:

Build with Parameters

You should get a screen similar to:

ENVIRONMENT [dev]

BRANCH [master]

MESSAGE [Hello Jenkins]

[Build]

For the first test, use:

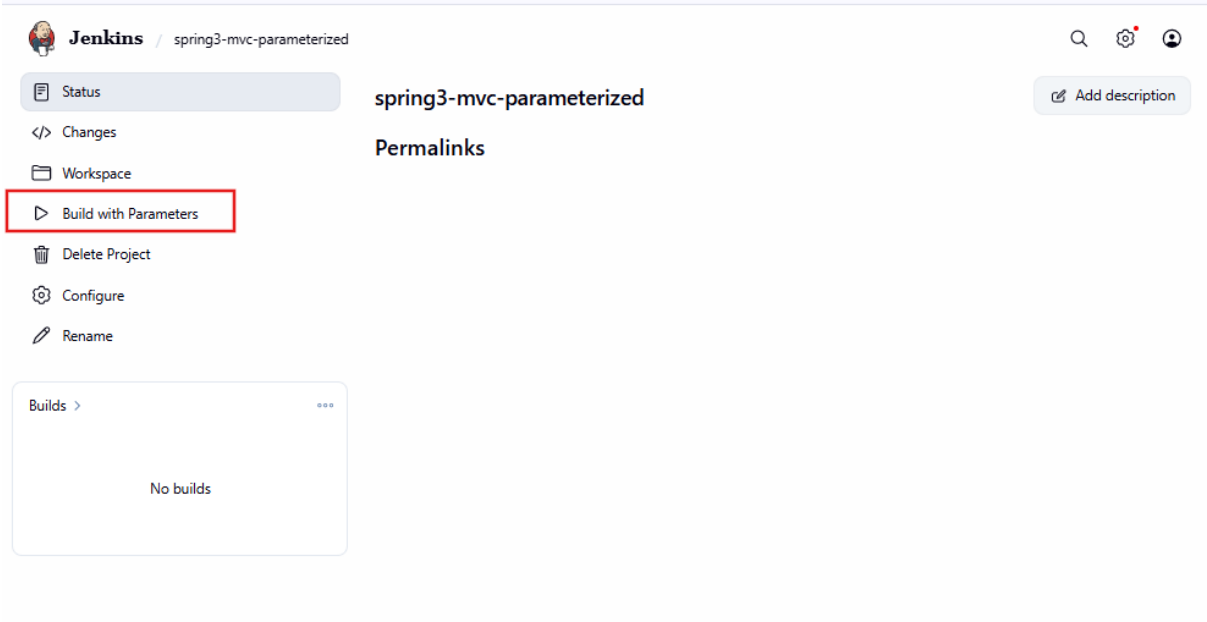
ENVIRONMENT = dev

BRANCH = master

MESSAGE = Testing parameterized Jenkins job

Click:

Build



Step 5.1: Run the parameterized job

Click:

Build with Parameters

You should see:

ENVIRONMENT

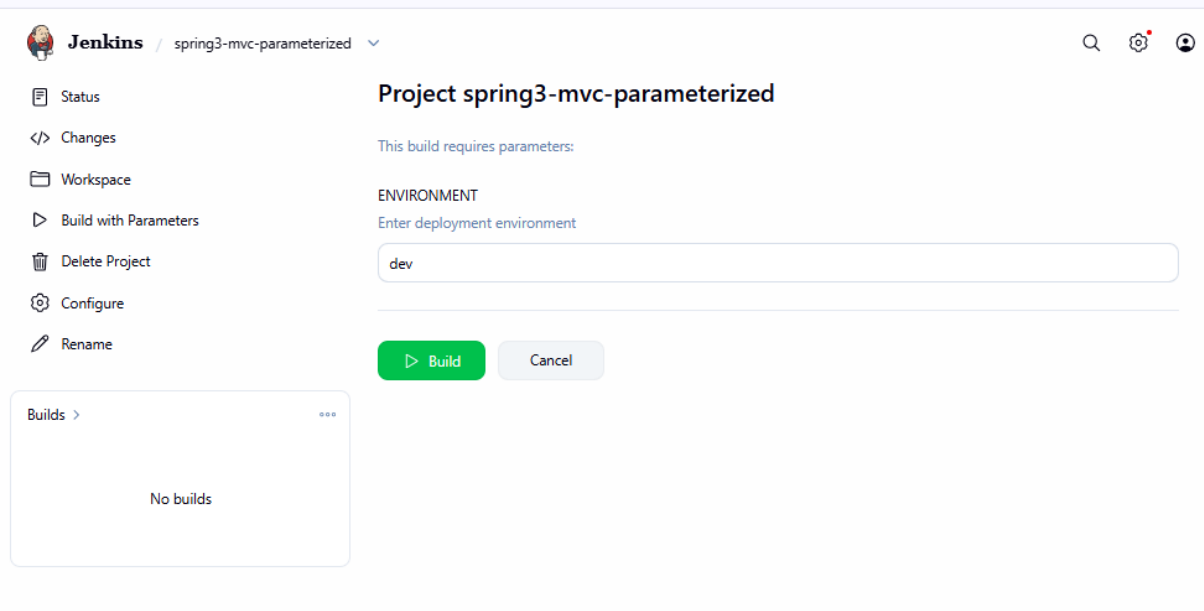
[dev]

Enter:

dev

Click:

Build



Step 5 — Check Console Output

Go to:

Build #1 → Console Output

You should see:

===== BUILD PARAMETERS =====

Environment : dev

Branch : master

Message : Testing parameterized Jenkins job

===== GIT CLONE =====

===== MAVEN BUILD =====

===== MAVEN TEST =====

===== USER MESSAGE =====

Testing parameterized Jenkins job

===== DEPLOYMENT =====

Application will be deployed to dev

===== PIPELINE COMPLETED =====

Finished: SUCCESS

Important: check the repository branch first

Before you use master, verify what branches actually exist in the repository:

git ls-remote --heads https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git

If it shows:

refs/heads/master

then keep:

BRANCH = master

If it shows:

refs/heads/main

then use:

BRANCH = main

Better parameterized version for your assignment

If your instructor wants to see a **real parameterized Maven job**, I would use these parameters:

Parameter	Type	Example
BRANCH	String	master

Parameter	Type	Example
ENVIRONMENT	Choice	dev
MAVEN_GOAL	Choice	package
MESSAGE	String	Testing Jenkins

For ENVIRONMENT, you can provide:

dev

test

prod

For MAVEN_GOAL:

package

verify

install

Then your Maven stage can use:

```
stage('Maven Build') {

   sh "mvn clean ${params.MAVEN_GOAL} -DskipTests"
}
```

That demonstrates the actual purpose of parameterization: **the same Jenkins job can behave differently based on values supplied when the build starts.**

Your Assignment 4 + 5 final structure

```
Assignment 4
 |
 └── Sample Pipeline Skeleton
    ├── Declarative Pipeline
    ├── Scripted Pipeline
```

└── Maven Pipeline Skeleton

Assignment 5

|

└── spring3-mvc-parameterized

|

├── BRANCH

├── ENVIRONMENT

├── MESSAGE

|

├── Git Clone

├── Maven Build

├── Maven Test

├── Display Parameters

└── Deployment Skeleton

This is a clean way to demonstrate both assignments without interfering with your existing **sabear → SonarQube → Nexus → Slack → Tomcat** pipeline.

Step 8: Test another parameter

Run:

Build with Parameters

Enter:

ENVIRONMENT = test

Click **Build**.

Console should show:

Environment selected: test

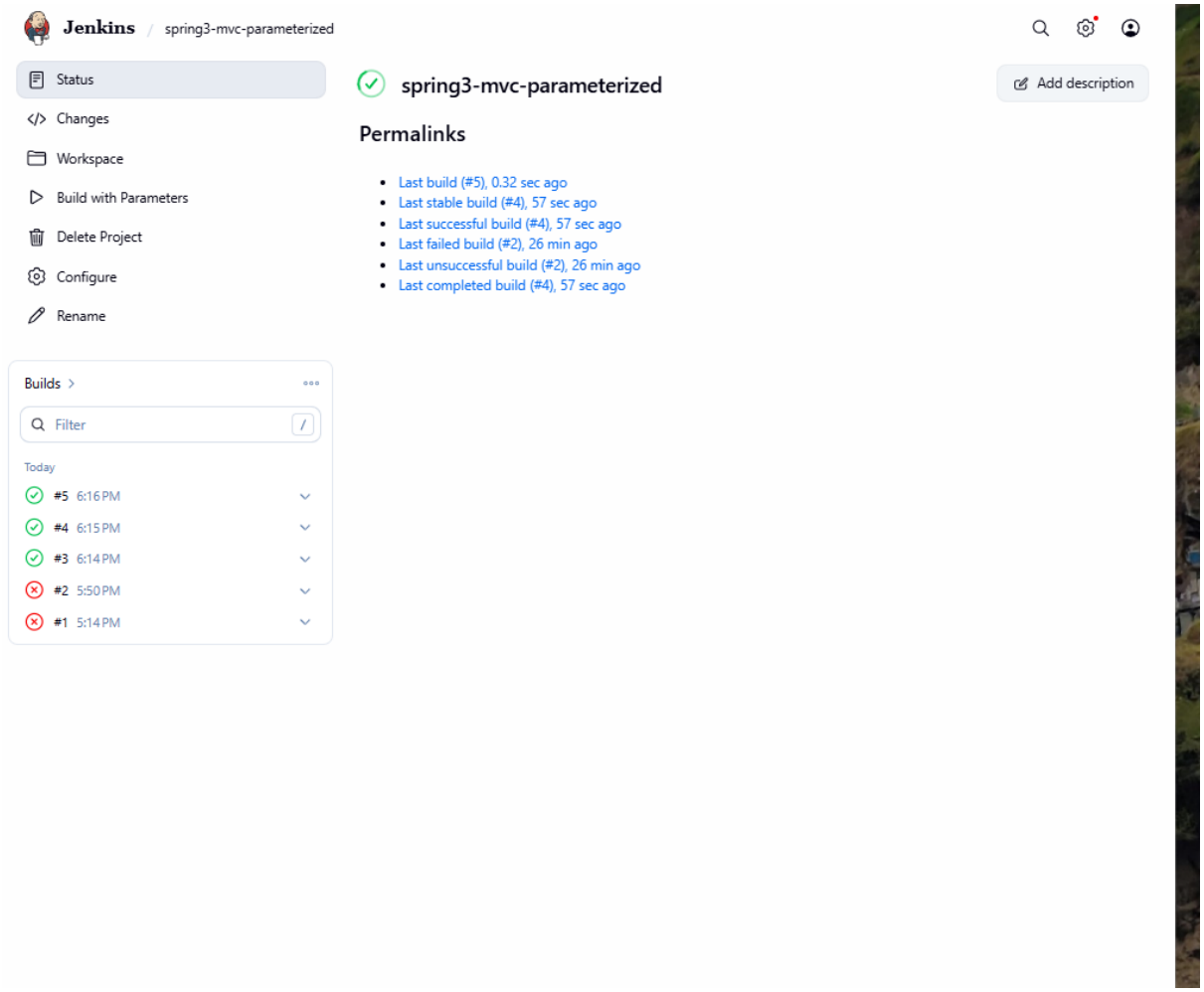
Run it again with:

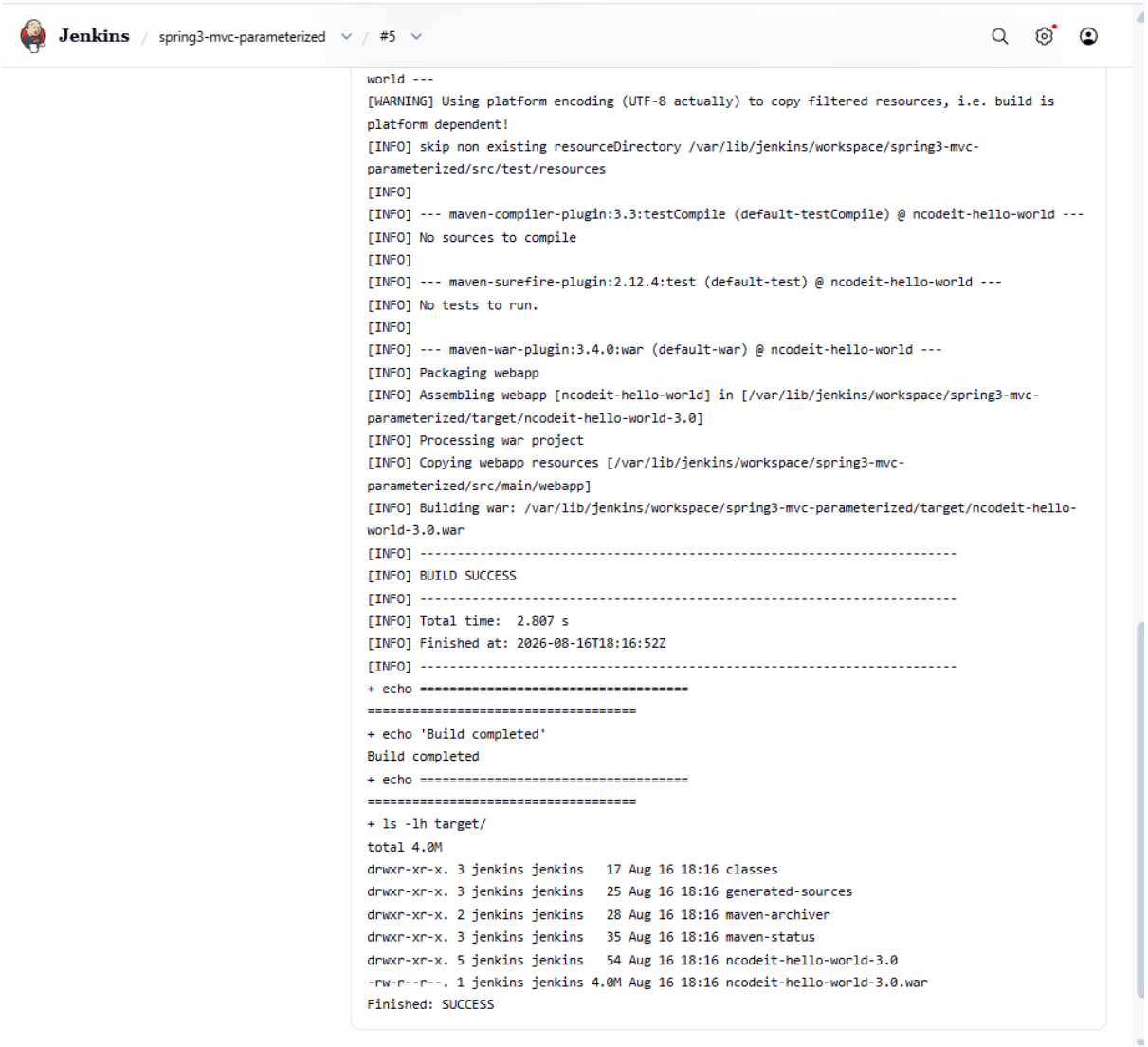
ENVIRONMENT = production

You should see:

Environment selected: production

This demonstrates that the Jenkins job is actually **parameterized**.





6) Setup one slave machine for Jenkins.

Jenkins Official Documentation

Recommended

- Pipeline: https://www.jenkins.io/doc/book/pipeline/

- Environment Variables: https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#using-environment-variables

- Parameters: https://www.jenkins.io/doc/book/pipeline/syntax/#parameters

- Handling credentials in Pipeline: https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#handling-credentials

Part 1 — Create the Agent EC2

Step 1 — Open AWS

Go to:

AWS Console → EC2 → Instances → Launch instance

Use:

Setting	Value
Name	jenkins-agent
AMI	Amazon Linux 2023
Instance type	t2.micro or t3.micro
Key pair	Your existing key pair
Network	Same VPC as Jenkins
Subnet	Preferably same/public subnet
Auto-assign Public IP	Enabled

For your assignment, a small Amazon Linux instance is enough.

Part 2 — Security Group

Your agent must allow SSH from the Jenkins controller.

Go to the agent's:

Security → Security Groups → Inbound rules

Add:

Type: SSH

Protocol: TCP

Port: 22

Source: Jenkins Server Security Group

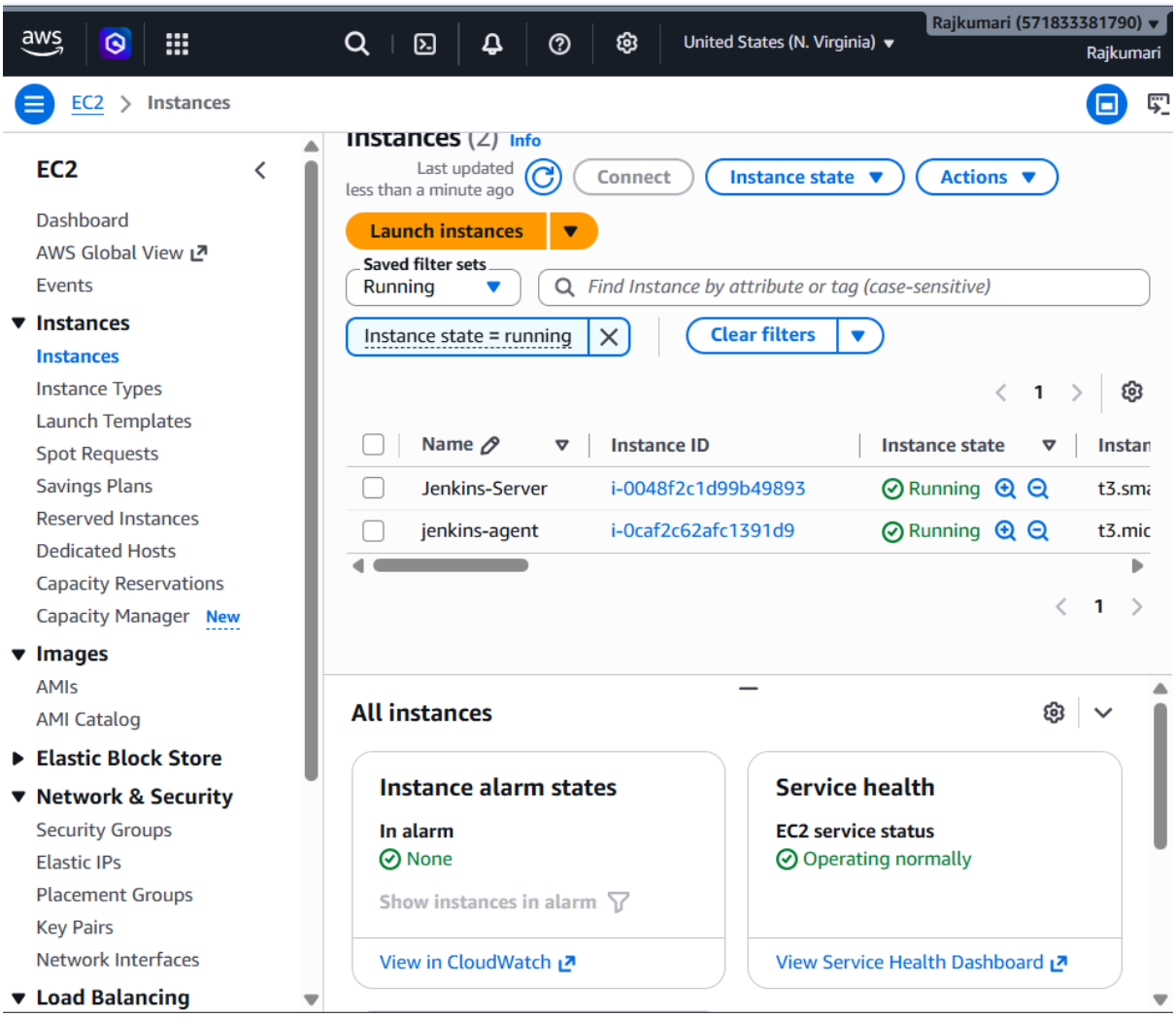
If you cannot reference the Jenkins security group, temporarily use:

Source: Jenkins Server private IP/32

For example:

172.31.x.x/32

Do not use 0.0.0.0/0 for SSH in a real environment.



Part 3 — Connect to the New Agent

SSH into the new EC2:

ssh -i your-key.pem ec2-user@AGENT_PUBLIC_IP

Then become root:

sudo su -

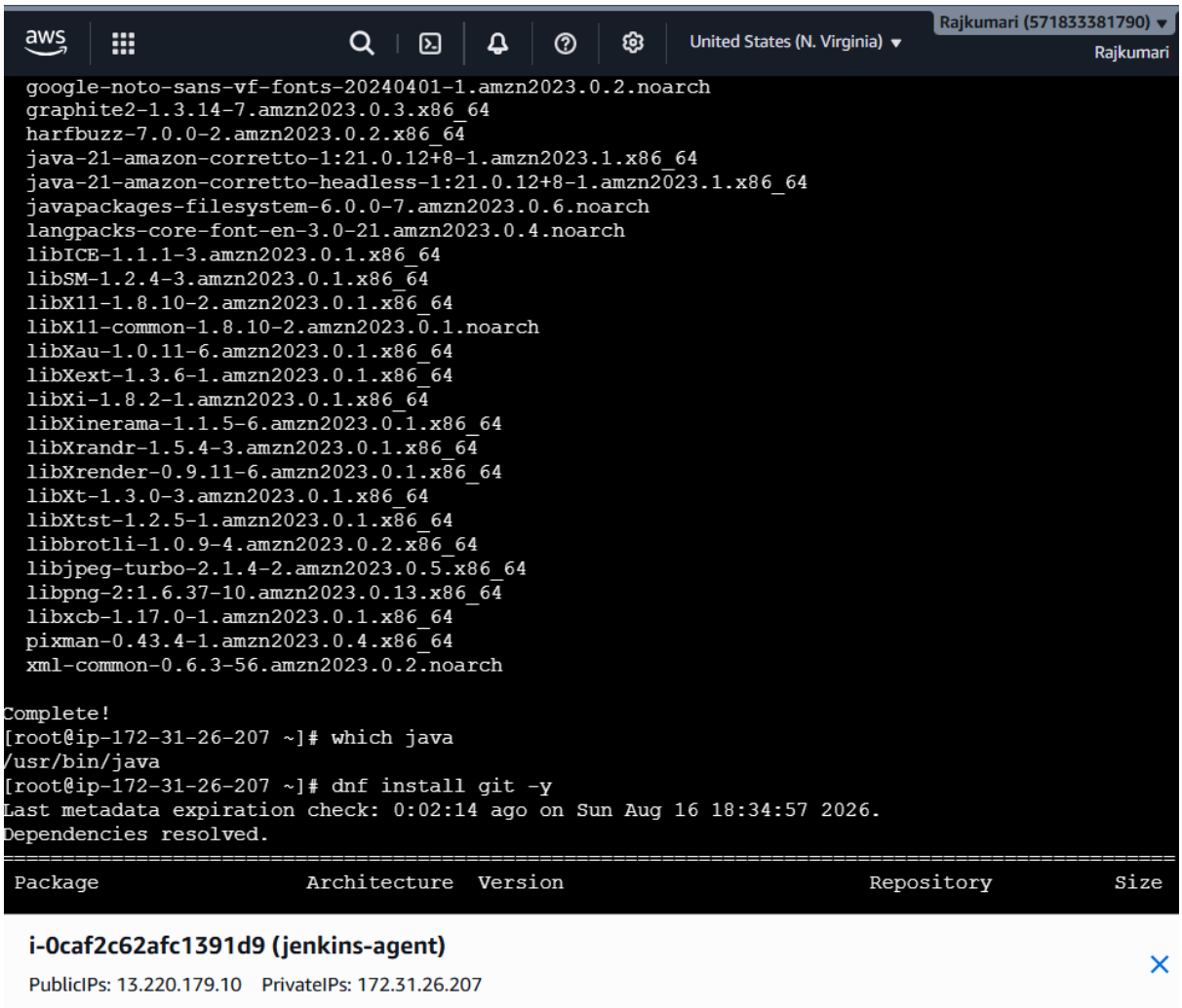
Check hostname:

hostname

You should see your agent machine hostname.

Part 4 — Install Java on Agent

Jenkins agents require Java. Jenkins documentation describes agents as Java processes that connect to the controller.
Since you're already using Java 17 on your Jenkins server, use Java 17 on the agent too.
Run:
dnf install java-17-amazon-corretto -y
Check:
java -version
You should see Java 17.
Also check:
which java
For example:
/usr/bin/java



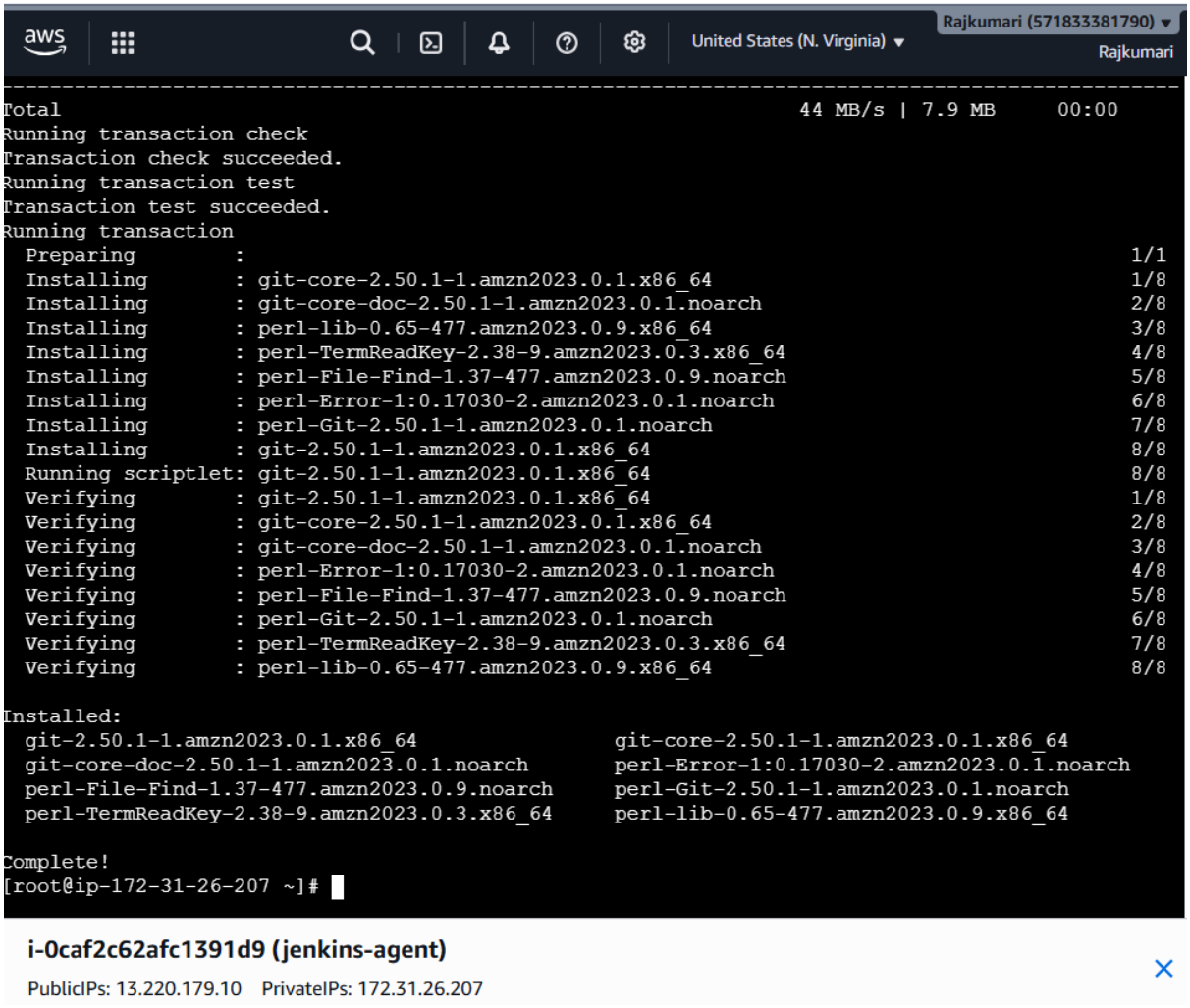
Part 5 — Install Git

Run:

dnf install git -y

Verify:

git –version



Part 6 — Install Maven

Because your agent will eventually perform builds, install Maven:

dnf install maven -y

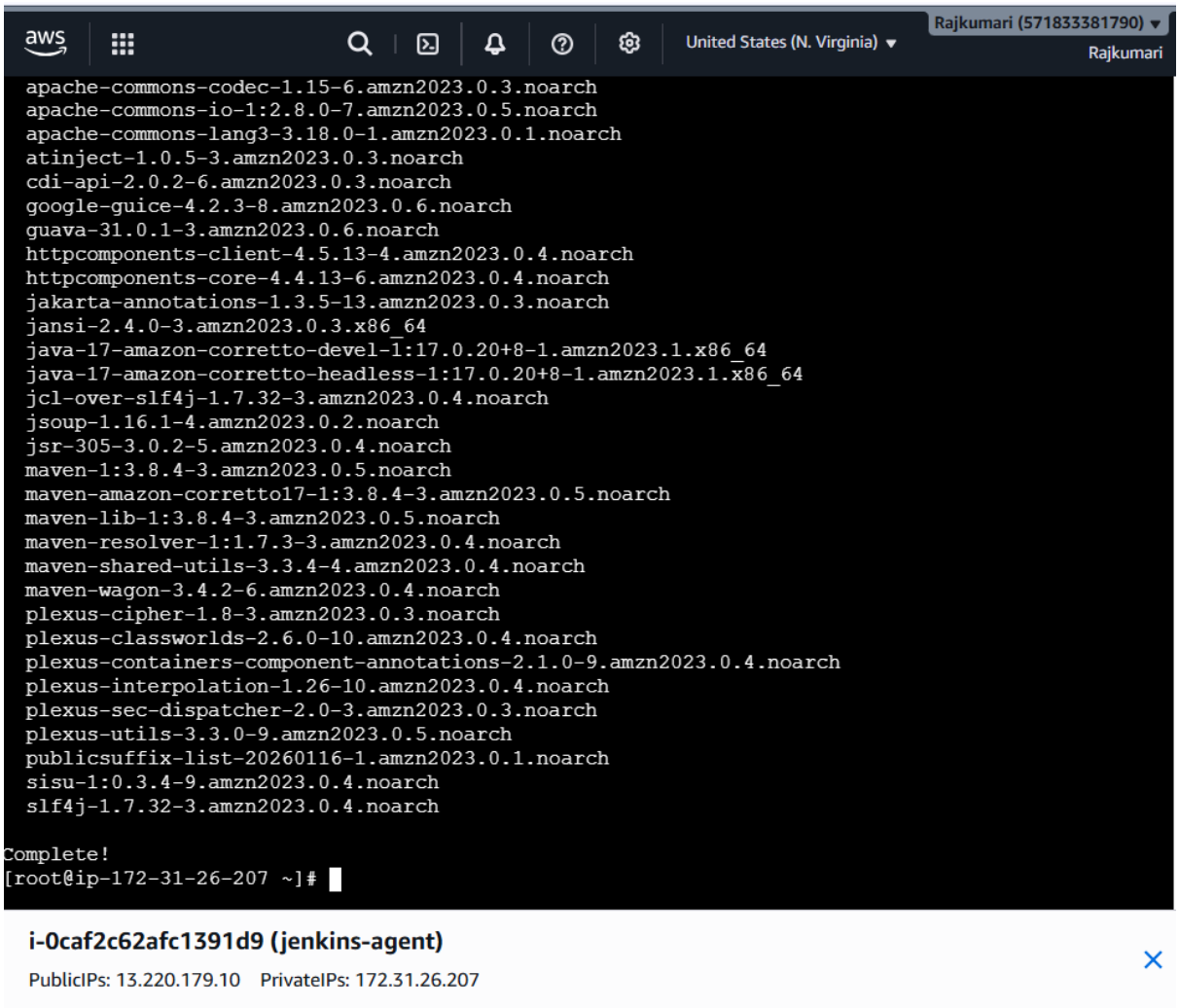
Verify:

mvn -version

You should see something similar to:

Apache Maven 3.x

Java version: 17



Part 7 — Create Jenkins Agent User

This is a good practice because you shouldn't run Jenkins builds as root.

On the **agent server**:

useradd -m -s /bin/bash jenkins

Set a password:

passwd jenkins

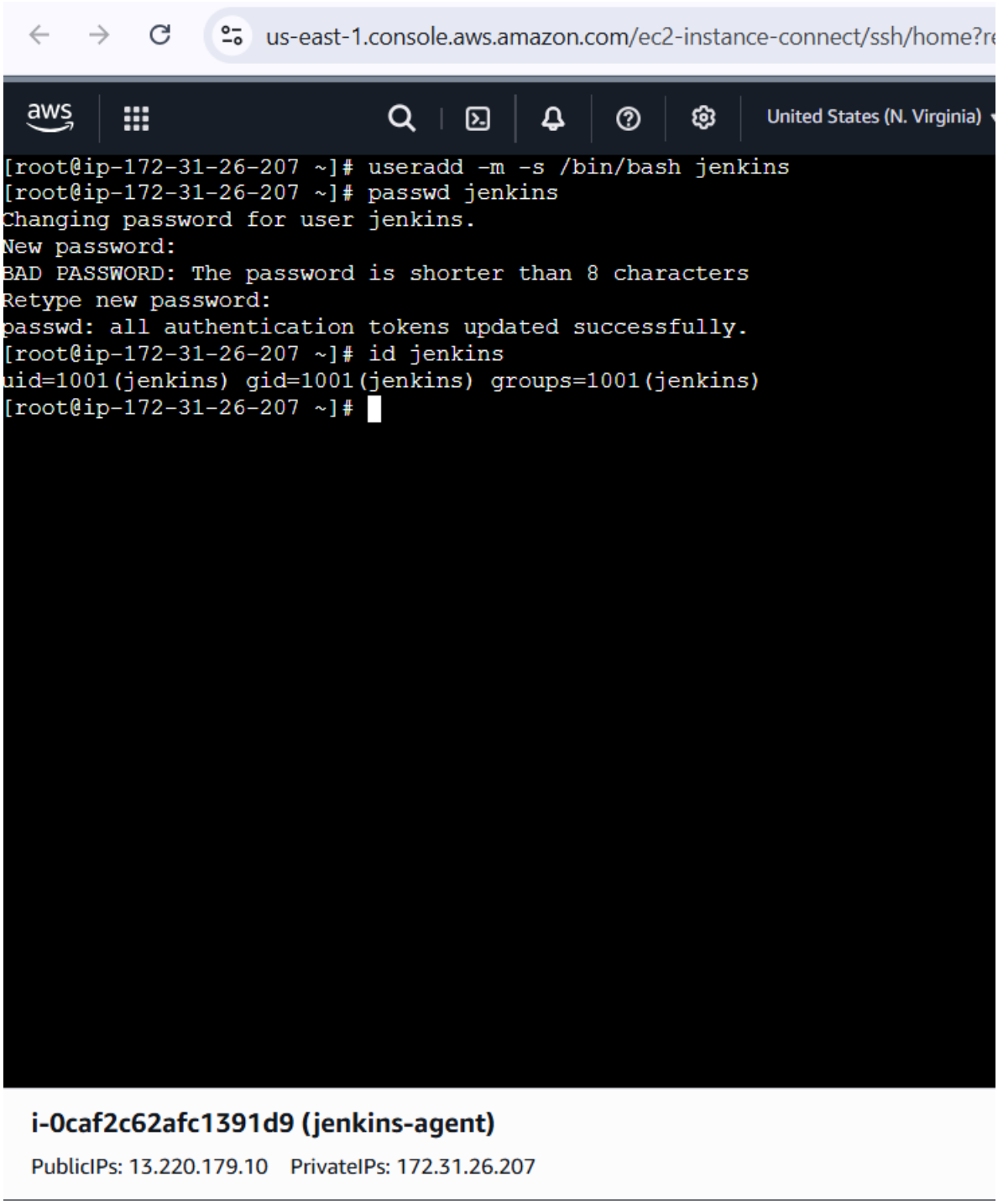
You can use a temporary password for this assignment.

Then:

id jenkins

You should see:

uid=... (jenkins)



Part 8 — Prepare Agent Directory

Run:

mkdir -p /home/jenkins/agent

Change ownership:

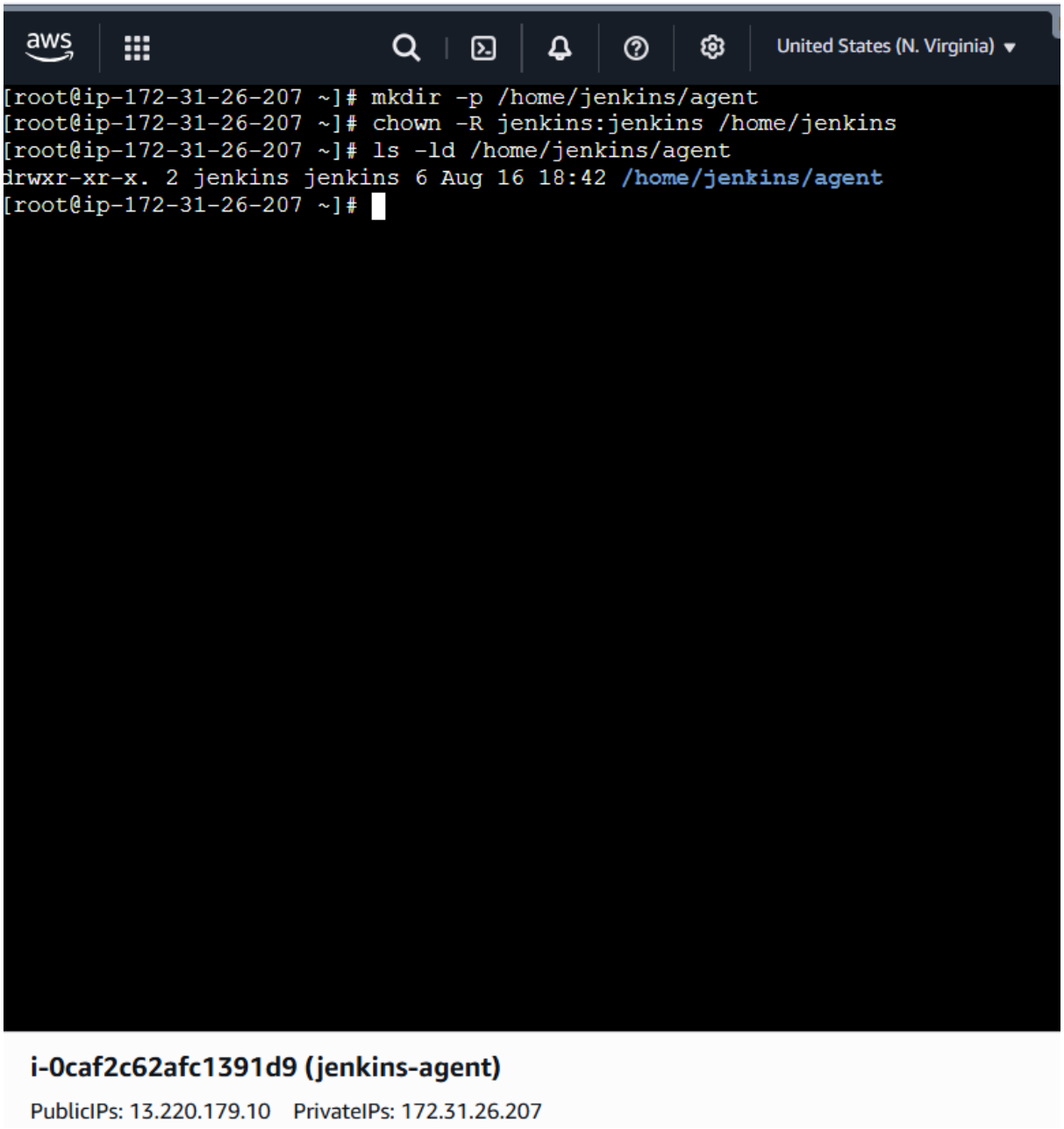
chown -R jenkins:jenkins /home/jenkins

Check:

ls -ld /home/jenkins/agent

It should show:

jenkins jenkins



Part 9 — Verify SSH Connectivity from Jenkins Controller

Now go to your **Jenkins controller server**.

You need to test:

ssh jenkins@AGENT_PRIVATE_IP

If this asks for a password, enter the password you created.

For example:

ssh jenkins@172.31.20.100

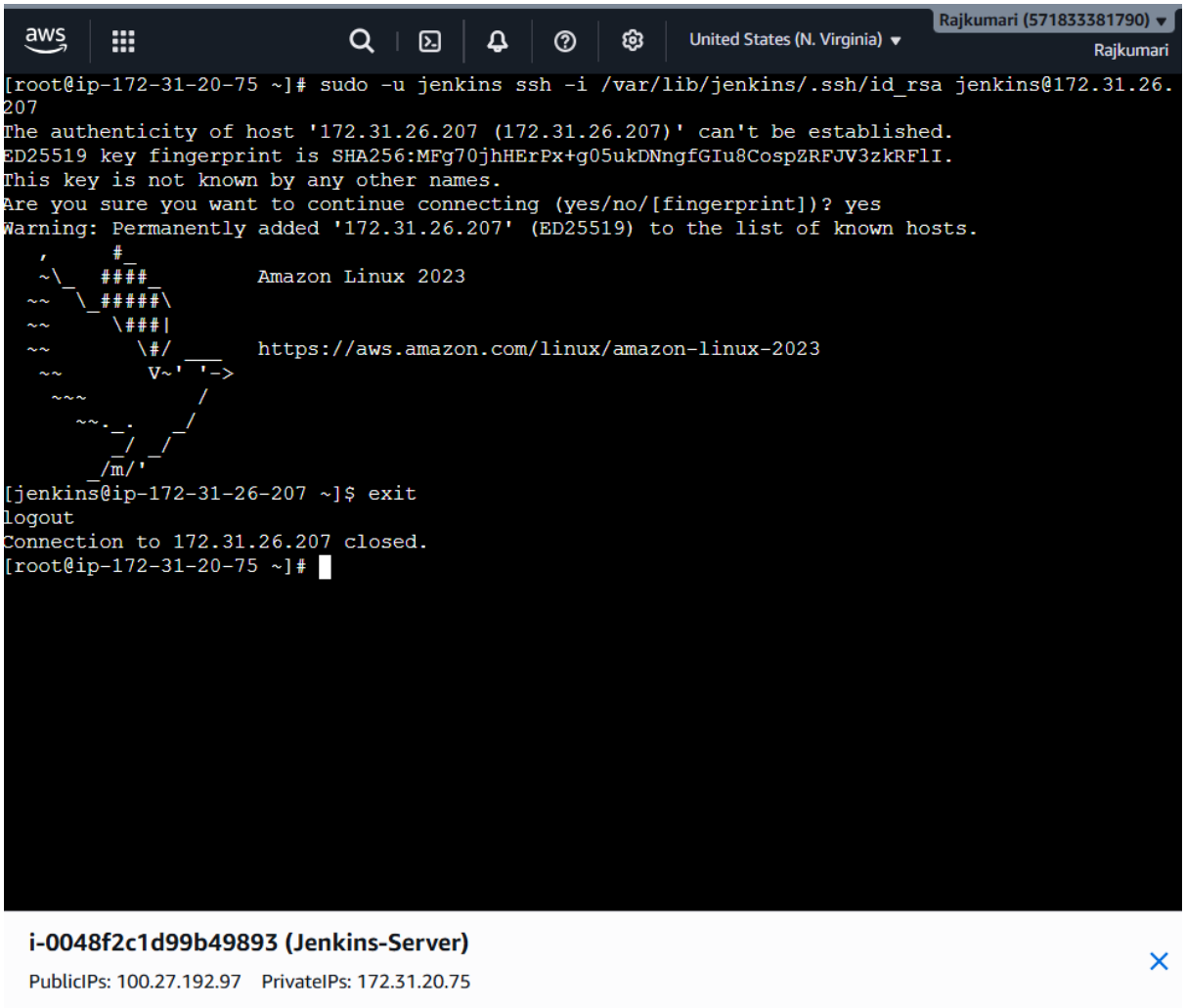
If you get:

Last login...

then SSH connectivity works.

Exit:

exit



Part 10 — Configure Agent in Jenkins

Now open Jenkins:

Jenkins Dashboard

↓

Manage Jenkins

↓

Nodes

Depending on your Jenkins version, this may appear as:

Manage Jenkins

→ Nodes

Jenkins' official agent setup follows the same basic process: create a new node, configure its remote root, labels, usage and launch method.

Part 11 — Create New Node

Click:

New Node

Enter:

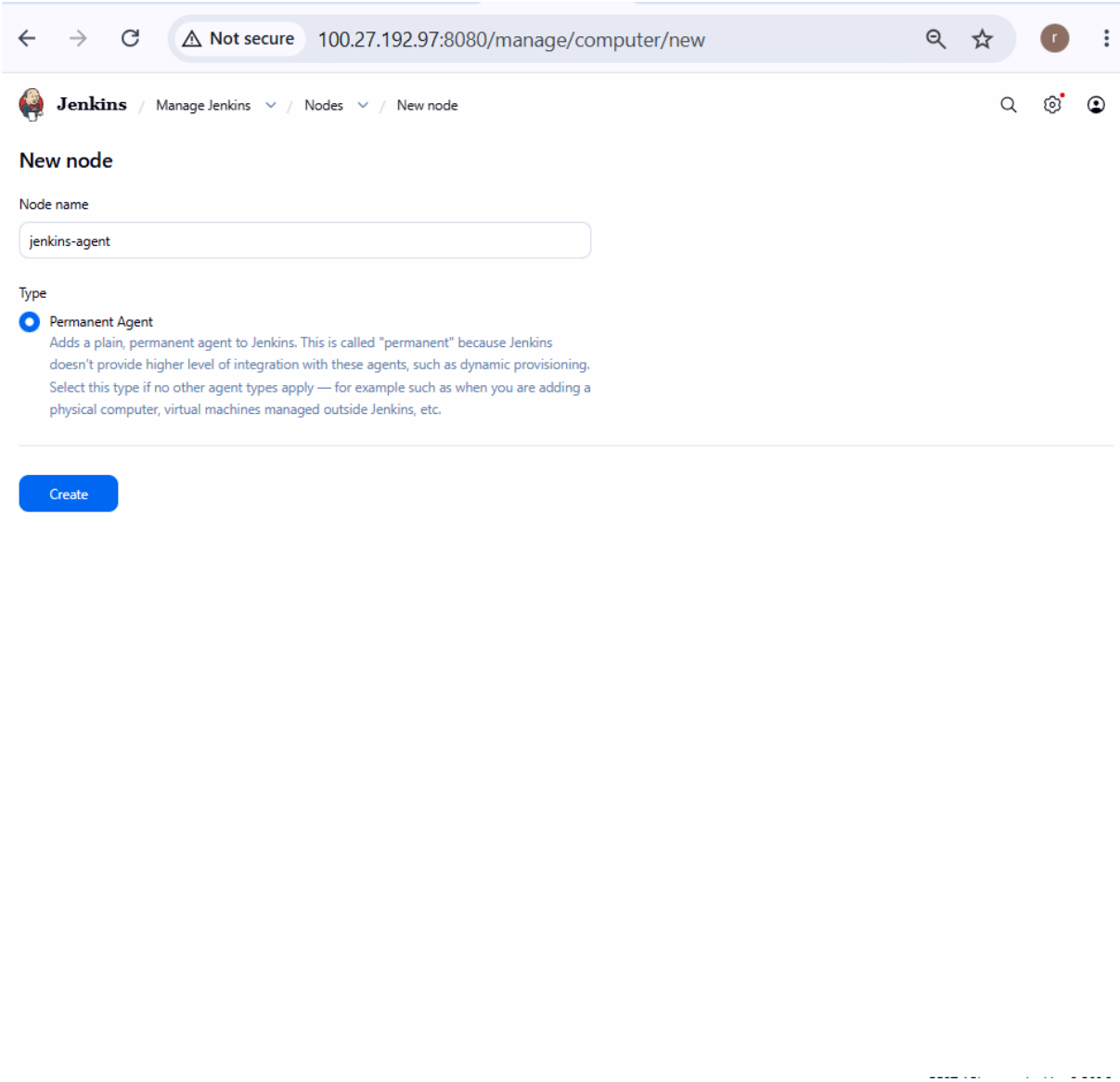
jenkins-agent

Select:

Permanent Agent

Click:

Create



Part 12 — Configure Node

You'll get the agent configuration page.

Name

jenkins-agent

Description

You can enter:

Jenkins build agent for DevOps assignments

Number of executors

Use:

1

One executor is a safe starting configuration for an agent.

Remote root directory

Enter:

/home/jenkins/agent

This is where Jenkins will keep its agent workspace/files.

Labels

Enter:

devops-agent

This is **very important**.

Your Pipeline can later specifically request:

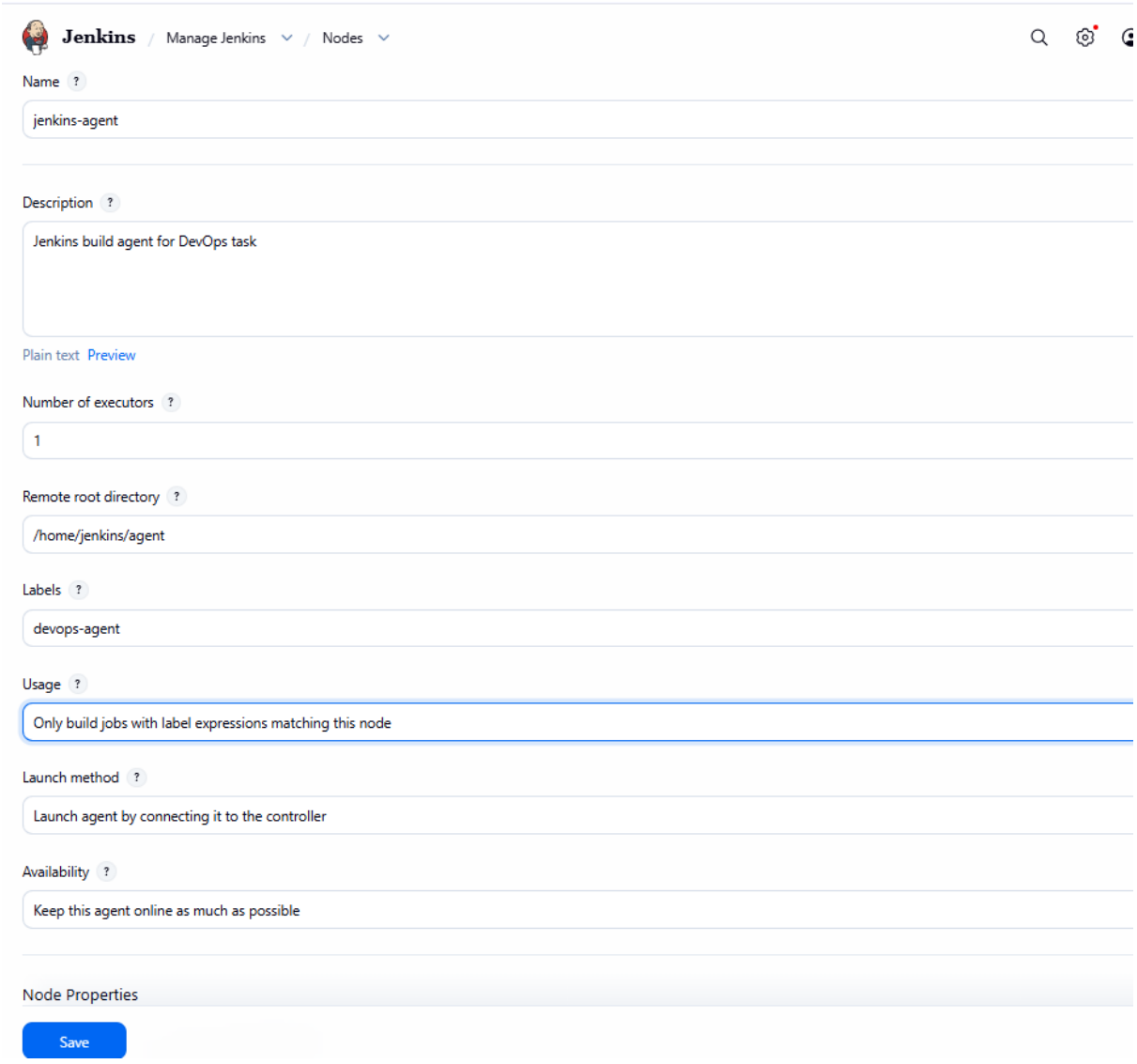
```
agent {
    label 'devops-agent'
}
```

Usage

Select:

Only build jobs with label expressions matching this node

This makes it easy to demonstrate that the job is running specifically on your new agent.



Part 13 — Launch Method

For your assignment, use:

Launch agents via SSH

Then configure:

Host

Enter the **private IP address** of your agent.

For example:

172.31.20.100

Use your actual private IP.

Credentials

Click:

Add → Jenkins

You need credentials that allow Jenkins controller to SSH into the agent.

Part 14 — Create SSH Credentials

If you're using username/password for the assignment:

Select:

Kind:

Username with password

Username:

jenkins

Password:

<password you created on agent>

ID:

jenkins-agent-ssh

Description:

SSH credentials for Jenkins agent

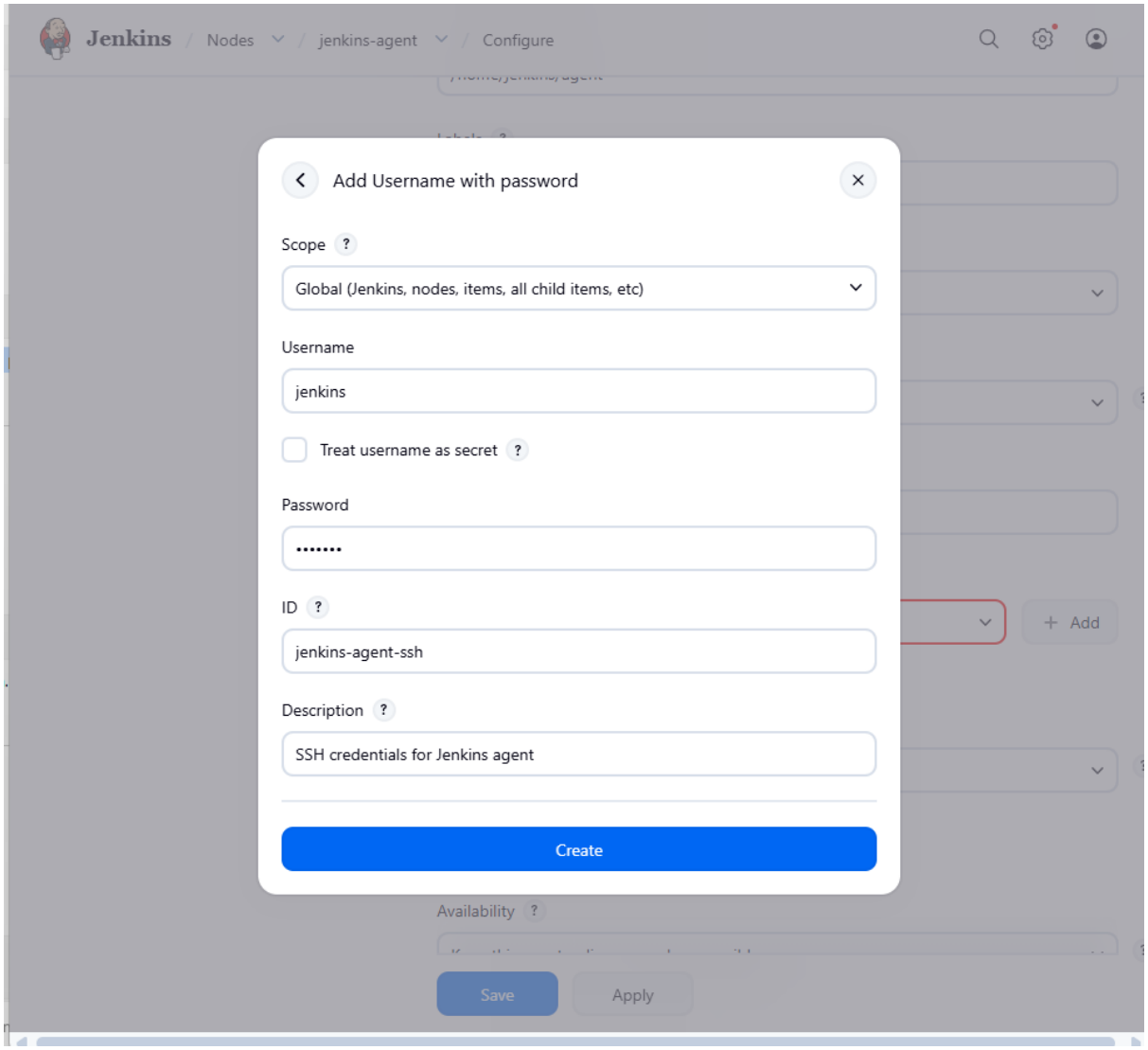
Click:

Add

Then select:

jenkins-agent-ssh

Jenkins supports storing credentials such as username/password and SSH private keys in its credential system rather than hard-coding them into jobs.



Part 15 — Host Key Verification

For a lab assignment, if you see:

Host Key Verification Strategy

you can select:

Non verifying Verification Strategy

However, for production environments, don't disable host-key verification. Proper host-key verification is safer.

Part 16 — Save

Click:

Save

Your node may initially show:

jenkins-agent

Offline

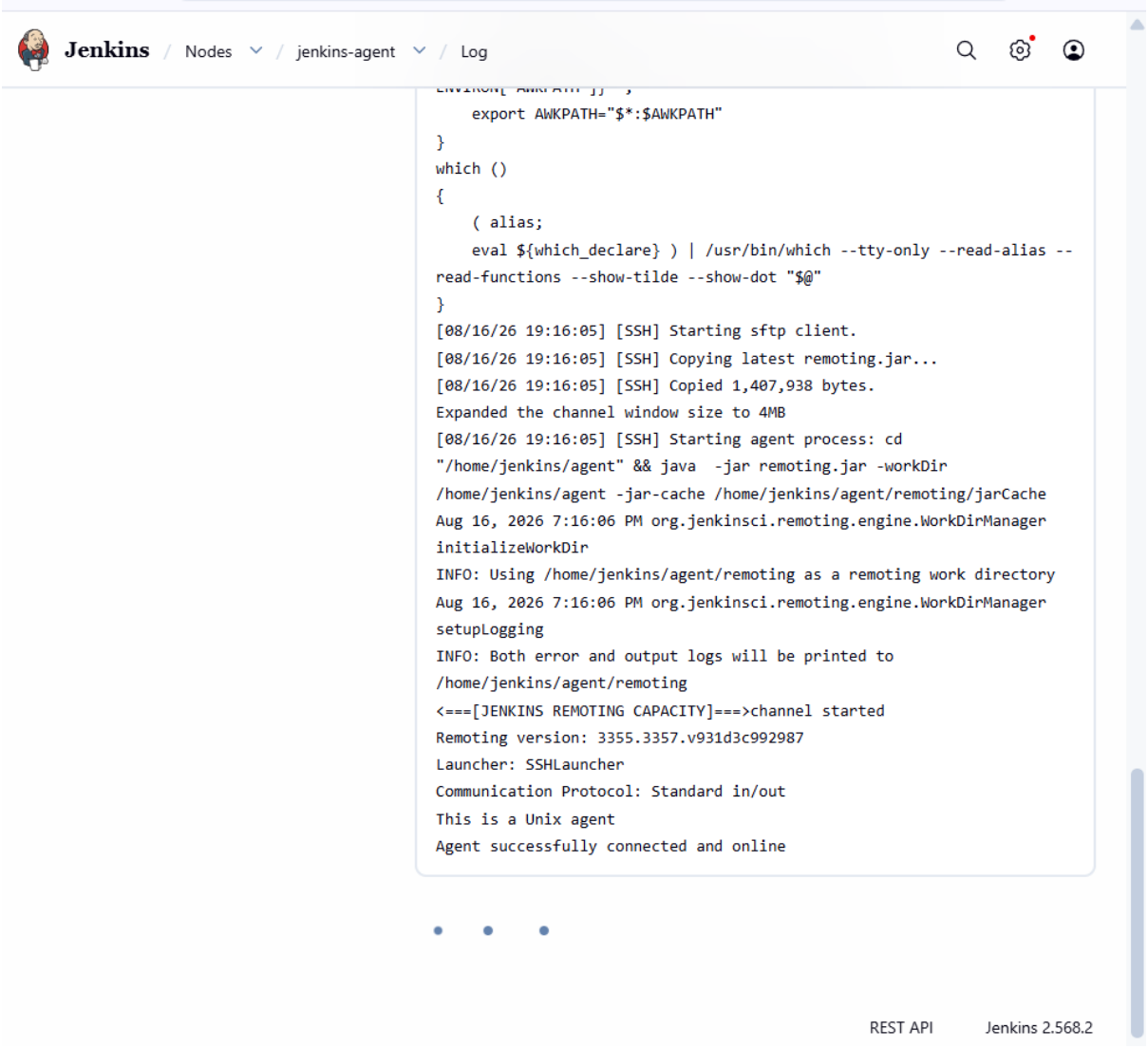
That's okay for a moment.

Jenkins should attempt to connect.

Official Jenkins documentation says a successfully launched agent should eventually show:

Agent successfully connected and online

in its log.



Part 18 — Verify the Agent is Online

Go to:

Manage Jenkins → Nodes

You should see:

jenkins-agent

Online

You have now completed the basic **slave/agent setup**.

Part 19 — Create a Test Freestyle Job

Now we need to prove that the agent actually works.

Create:

Jenkins Dashboard → New Item

Name:

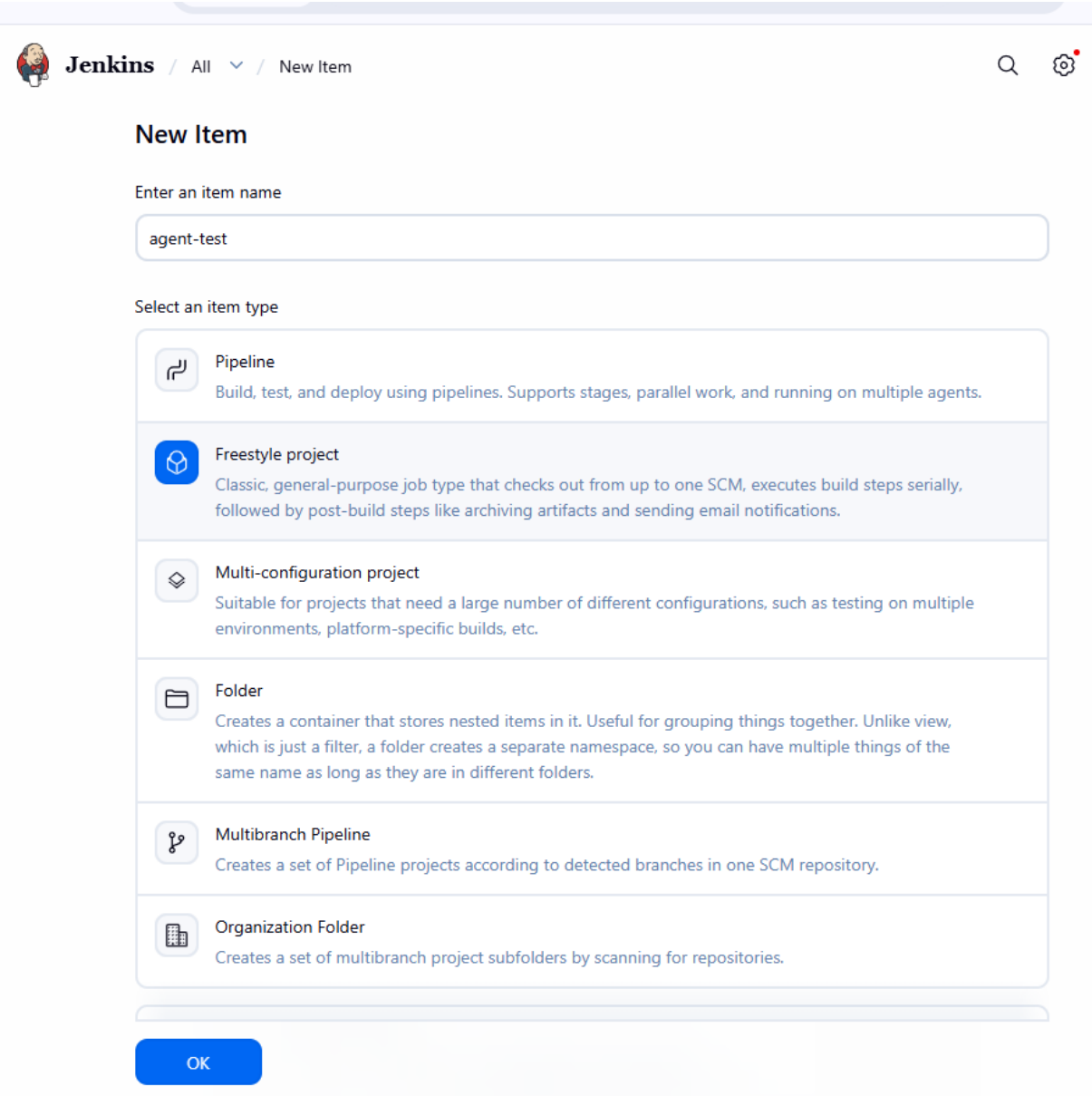
agent-test

Select:

Freestyle project

Click:

OK



Part 20 — Restrict Job to Agent

Inside the job configuration, find:

General

Enable:

Restrict where this project can be run

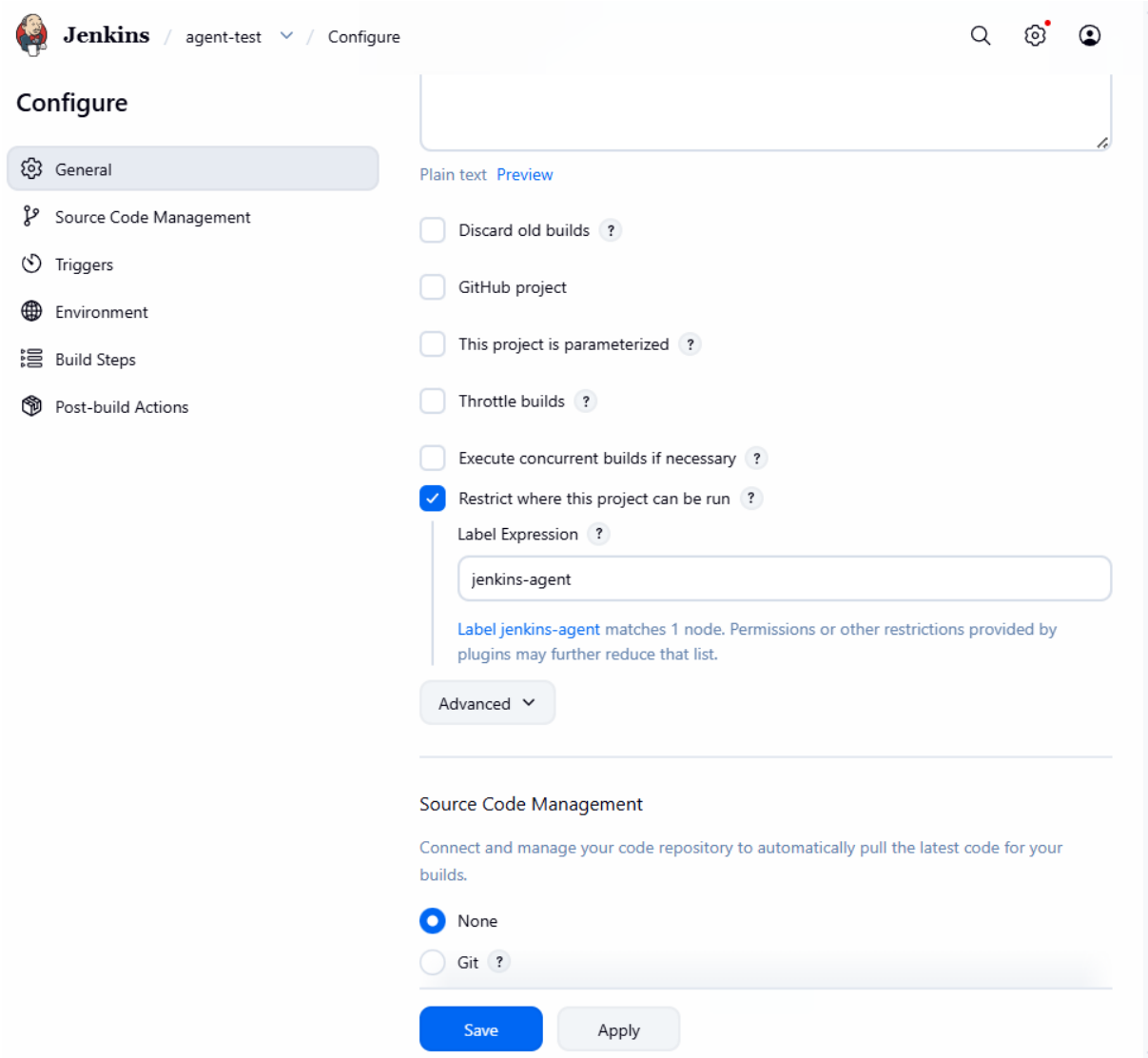
Label Expression:

devops-agent

This tells Jenkins:

Run this job only on the node having the devops-agent label.

Jenkins' official agent tutorial uses the same concept: assign a label to the agent and restrict the job to that label.



Part 21 — Add Execute Shell

Under:

Build Steps → Execute shell

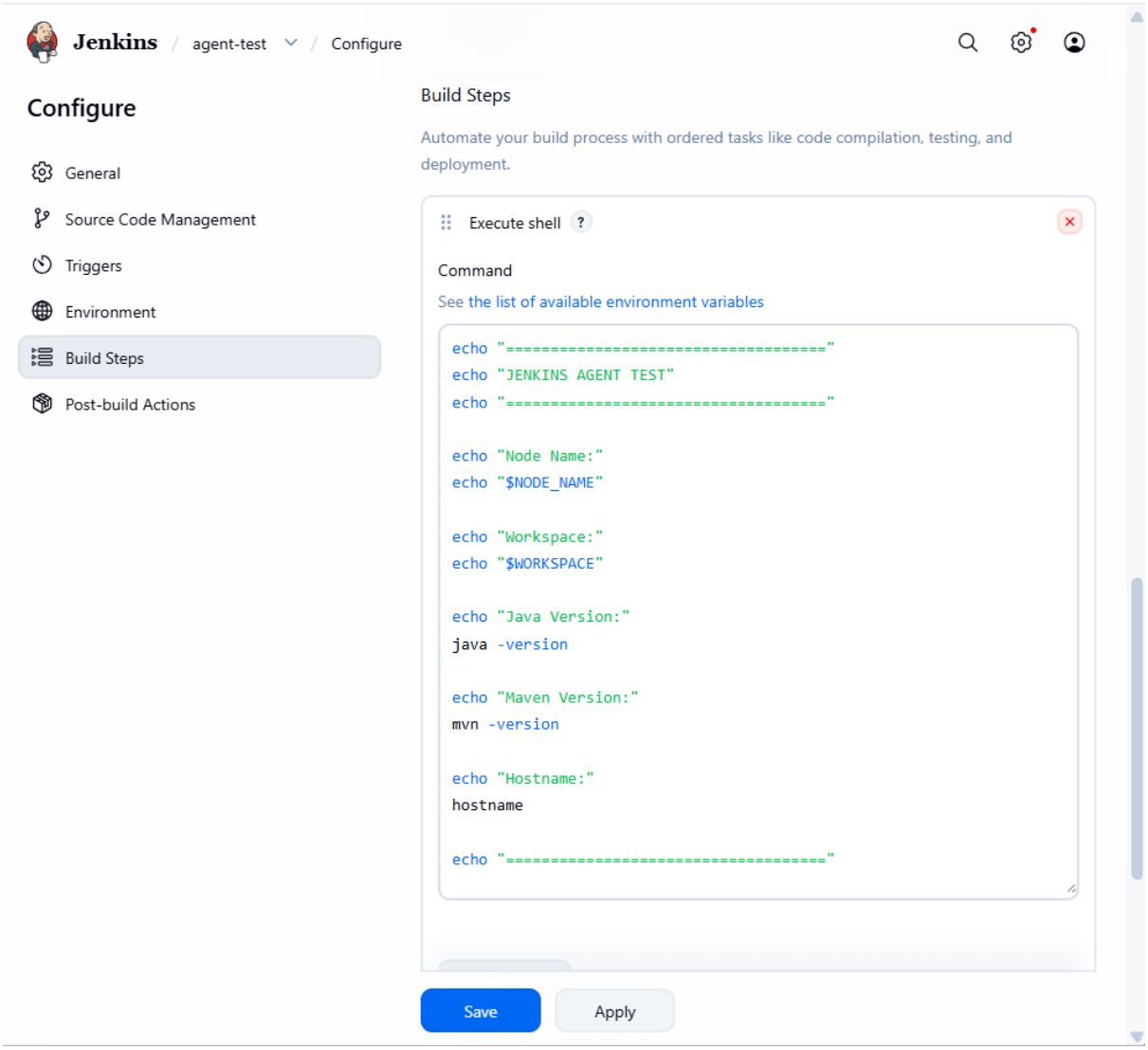
Enter:

```
echo "=================================="
echo "JENKINS AGENT TEST"
echo "=================================="
```

```
echo "Node Name:"
echo "$NODE_NAME"

echo "Workspace:"
echo "$WORKSPACE"

echo "Java Version:"
java -version

echo "Maven Version:"
mvn -version

echo "Hostname:"
hostname

echo "========================================"
```

Click:

Save



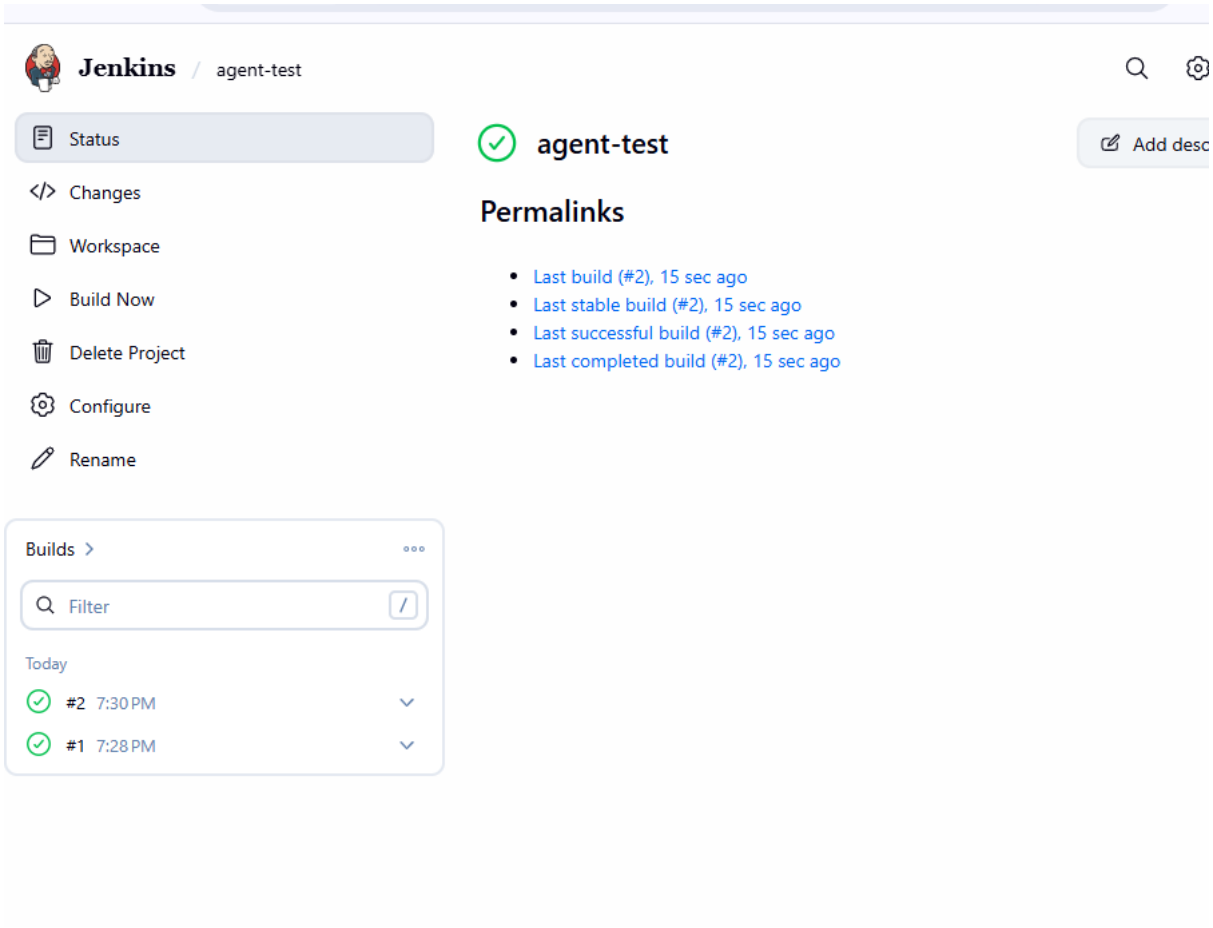
Part 22 — Build the Test Job

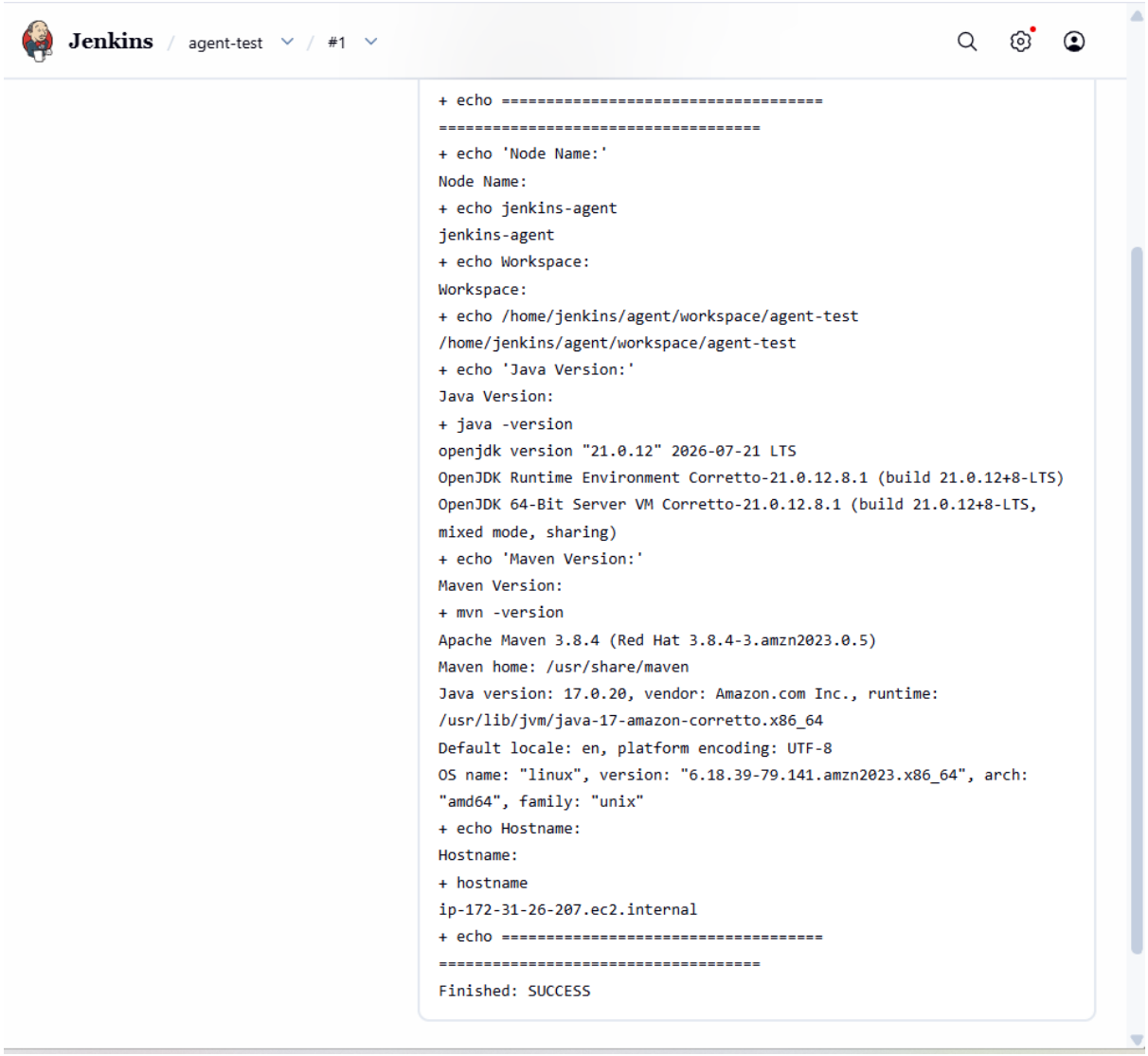
Click:

Build Now

Open:

Console Output


Jenkins / agent-test
Status
Changes
Workspace
Build Now
Delete Project
Configure
Rename
Builds
Filter
/
Today
#2 7:30 PM
#1 7:28 PM
agent-test
Add desc
Permalinks
Last build (#2), 15 sec ago
Last stable build (#2), 15 sec ago
Last successful build (#2), 15 sec ago
Last completed build (#2), 15 sec ago



Part 23 — Test With Your Maven Application

Now we can use your assignment project.

Create another job or modify the test job.

Use your repository:

https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git

Branch:

*/master

Restrict it to:

devops-agent

Then Execute Shell:

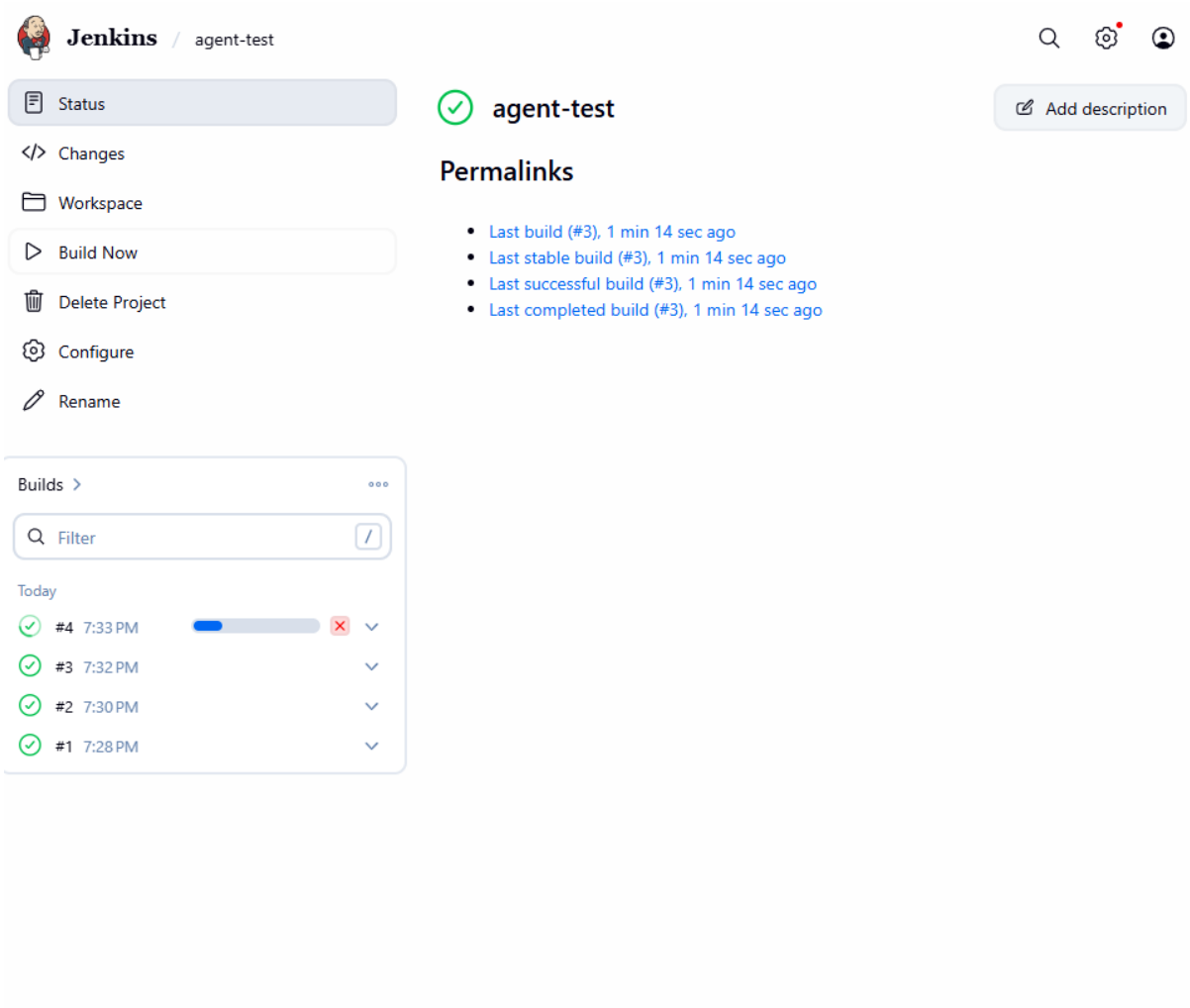
```
echo "Running build on:"
echo "$NODE_NAME"

java -version
mvn -version

mvn clean package

echo "WAR files:"
ls -lh target/*.war
```

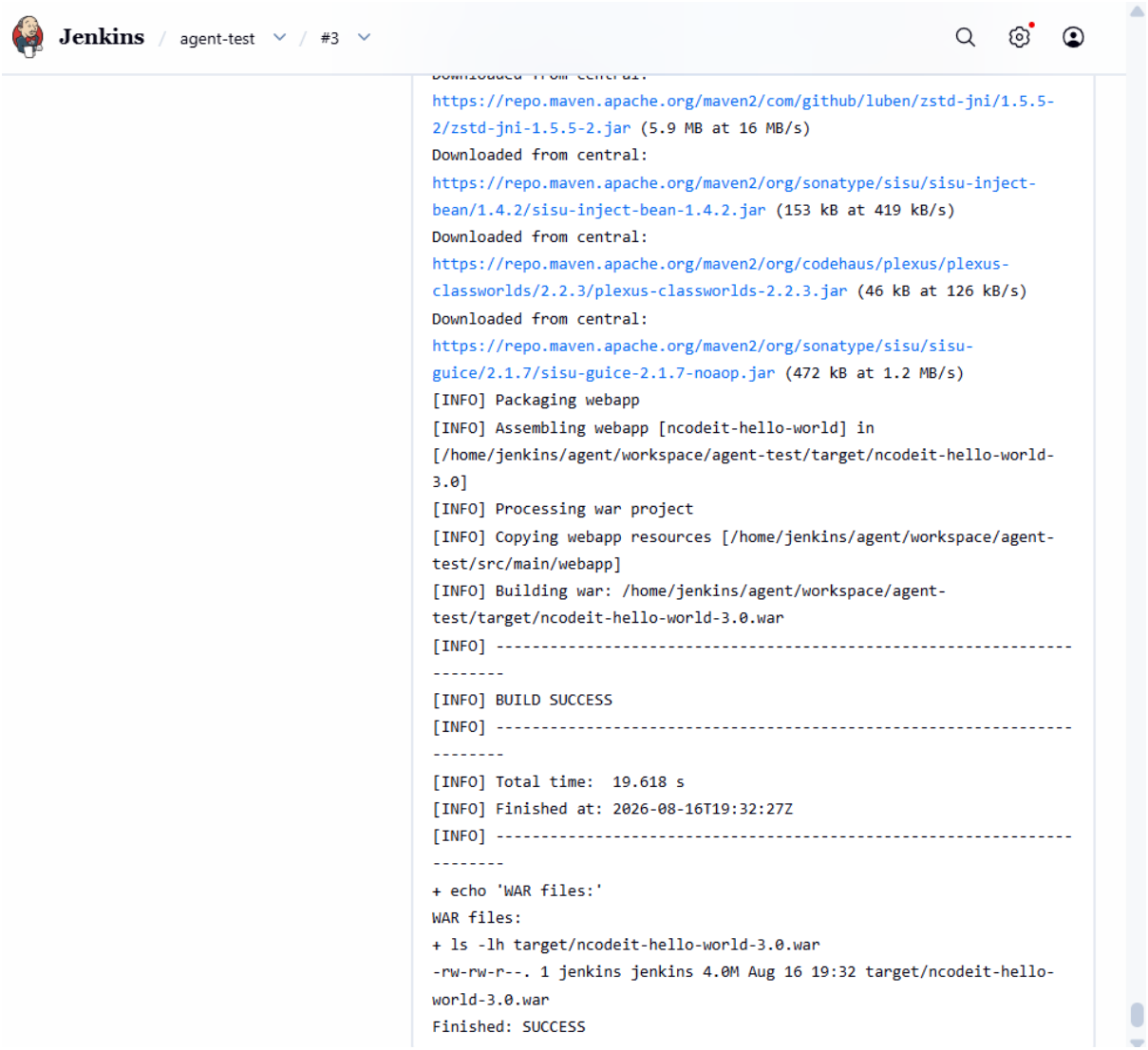
This should produce:

```
[INFO] BUILD SUCCESS
```

and:

```
target/ncodeit-hello-world-3.0.war
```


Jenkins / agent-test
Status
Changes
Workspace
Build Now
Delete Project
Configure
Rename
Builds
Filter
/
Today
#4 7:33 PM
#3 7:32 PM
#2 7:30 PM
#1 7:28 PM
agent-test
Add description
Permalinks
Last build (#3), 1 min 14 sec ago
Last stable build (#3), 1 min 14 sec ago
Last successful build (#3), 1 min 14 sec ago
Last completed build (#3), 1 min 14 sec ago



Part 24 — Pipeline Test

Because your assignment also gives the Jenkins Pipeline documentation, you can demonstrate the agent with a Pipeline.

Create:

New Item → Pipeline

Name:

agent-pipeline

Under **Pipeline → Definition**, select:

Pipeline script

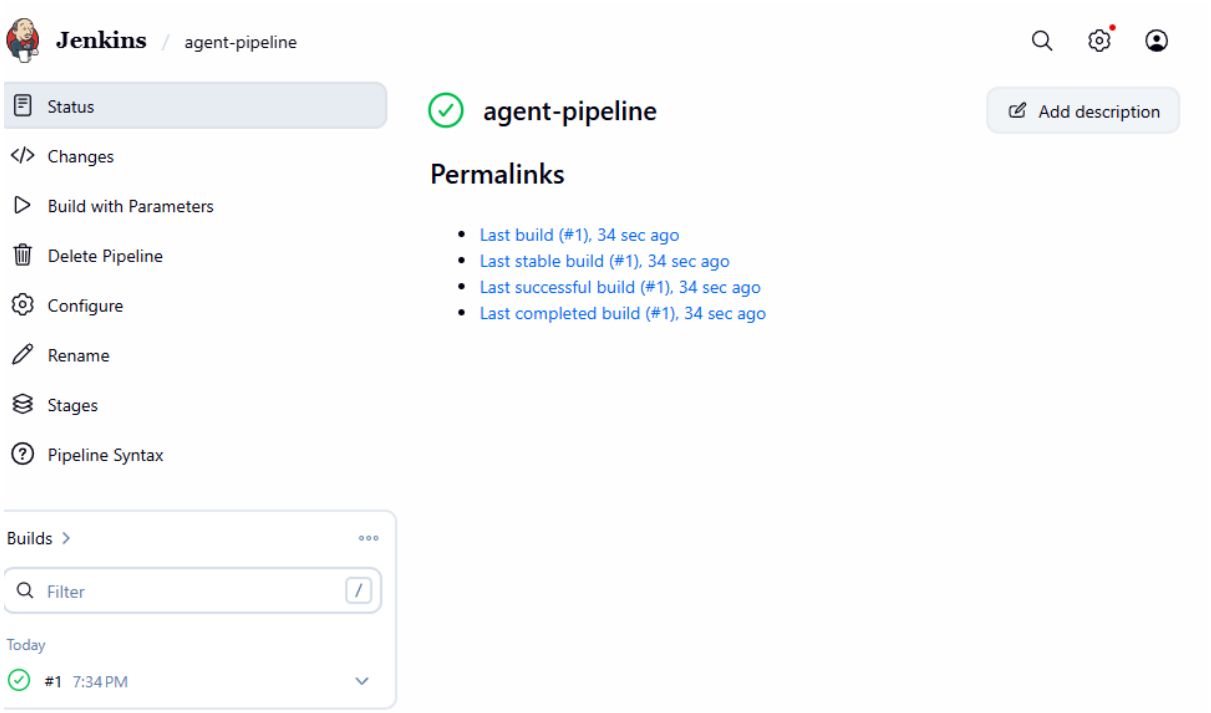
Use:

```
pipeline {
    agent {
        label 'devops-agent'
    }

    environment {
        APP_NAME = 'spring3-mvc'
        BUILD_ENV = 'dev'
    }

    parameters {
        choice(
            name: 'ENVIRONMENT',
            choices: ['dev', 'test', 'prod'],
            description: 'Select deployment environment'
        )

        string(
            name: 'VERSION',
            defaultValue: '3.0',
            description: 'Application version'
        )
    }

    stages {
```

```
    stage('Agent Information') {
        steps {
            echo "Running on node: ${env.NODE_NAME}"
            echo "Workspace: ${env.WORKSPACE}"
            echo "Environment: ${params.ENVIRONMENT}"
            echo "Version: ${params.VERSION}"
        }
    }

    stage('Git Clone') {
        steps {
            git branch: 'master',
                url: 'https://github.com/rajkumari-hub/spring3-mvc-maven-xml-hello-world-1.git'
        }
    }

    stage('Maven Build') {
        steps {
            sh 'java -version'
            sh 'mvn -version'
            sh 'mvn clean package'
        }
    }

    stage('Verify WAR') {
```

```
        steps {
            sh 'ls -lh target/*.war'
        }
    }
}

post {
    success {
        echo 'Build completed successfully on Jenkins agent.'
    }

    failure {
        echo 'Build failed.'
    }
}
}
```

Jenkins Pipeline supports agent, environment, and parameters directly in Declarative Pipeline syntax. Parameters are available through params, while environment variables are available through env.



Part 25 — Build Pipeline

Click:

Build with Parameters

Select:

ENVIRONMENT = dev

VERSION = 3.0

Click:

Build

Your console should show:

Running on node: jenkins-agent

Then:

Environment: dev

Version: 3.0

Then:

[INFO] BUILD SUCCESS

Finally:

Build completed successfully on Jenkins agent.

About the credentials assignment

Your supplied documentation also mentions:

Handling credentials in Pipeline.

For this assignment, don't put passwords or PATs directly into the Jenkinsfile. Jenkins recommends storing credentials in Jenkins and referring to them by credential ID.

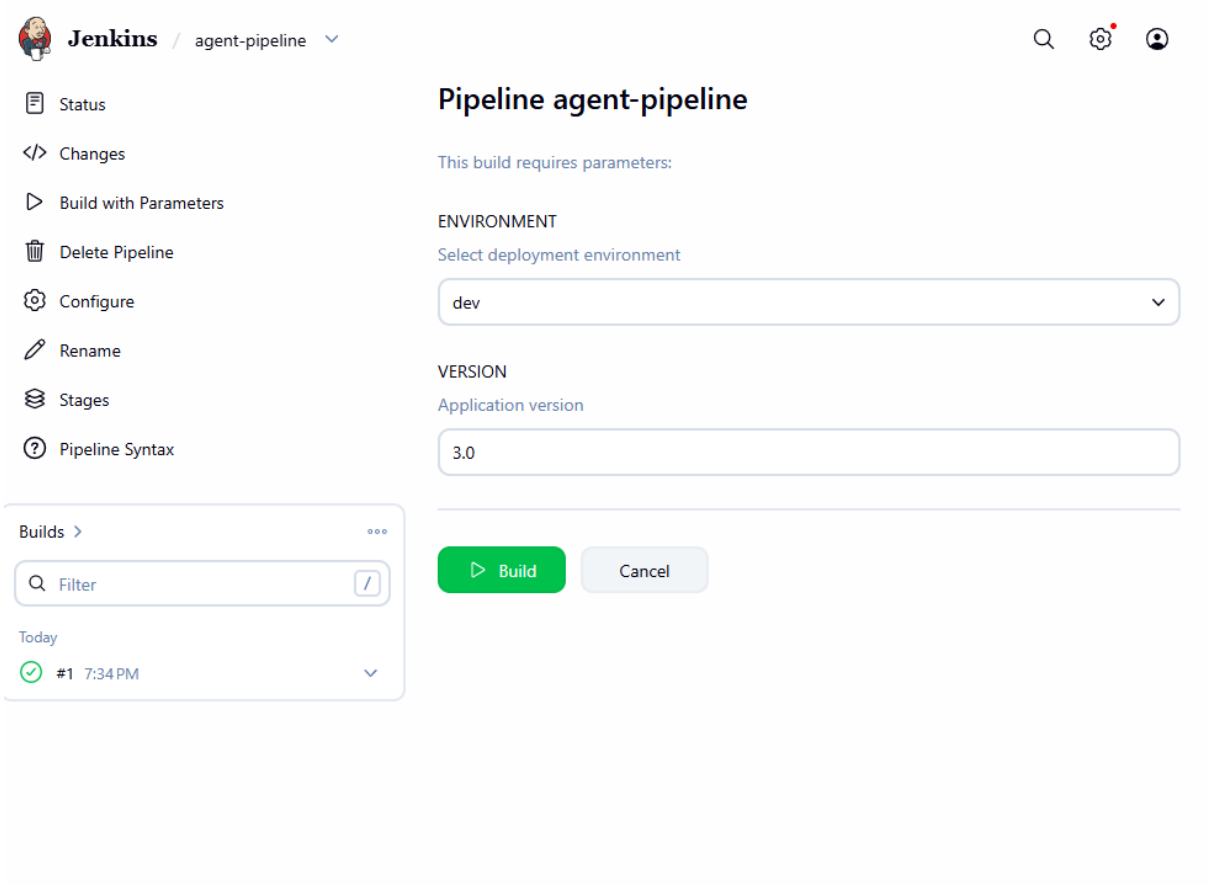
For example:

```
environment {
   GITHUB_CREDS = credentials('github-credentials')
}
```

Then Jenkins exposes the appropriate credential variables to the Pipeline.

Do not do this:

```
GITHUB_PASSWORD = 'mypassword123'
```


Jenkins / agent-pipeline
Status
Changes
Build with Parameters
Delete Pipeline
Configure
Rename
Stages
Pipeline Syntax
Builds
Filter
/
Today
#1 7:34 PM
Pipeline agent-pipeline
This build requires parameters:
ENVIRONMENT
Select deployment environment
dev
VERSION
Application version
3.0
Build
Cancel

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.979 s
[INFO] Finished at: 2026-08-16T19:37:17Z
[INFO] ------------------------------------------------------------------------
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify WAR)
[Pipeline] sh
+ ls -lh target/ncodeit-hello-world-3.0.war
-rw-rw-r--. 1 jenkins jenkins 4.0M Aug 16 19:37 target/ncodeit-hello-world-3.0.war
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Build completed successfully on Jenkins agent.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```